

Aegis

AEGIS: Technical Design & Evaluation Report

A Mobile App for Local Disaster Preparedness and Response

Asim Naseer
https://github.com/awsimn/Aegis

Final Project Report 1
Developing a Mobile App for Local Disaster Preparedness and Response 1
0.1 Project Template Declaration 3
INTRODUCTION (995 words) 3
1.1 Background / Context 3
1.2 Problem Definition 3
1.3 Project Objectives and Motivation 3
1.4 Scope 4
1.5 Ethical Considerations 5
LITERATURE REVIEW (1974 words) 7
2.1 The role of mobile applications 7
2.2 Gamification in Mobile Applications 8
2.3 Mobile User Experience for Crisis Apps 8
2.4 Existing Disaster Management Apps (Comparative Review) 10
2.5 Machine Learning Models 11
DESIGN (1998 words) 13
3.1 Requirements Analysis 13
3.2 Design Philosophy 14
3.3 User Group Analysis 15
3.4 Use Cases 17
3.5 Database Design 19
3.6 Wireframes 20
3.7 Test Strategy 21
IMPLEMENTATION (2433 words) 23
4.1 Technical Architecture 23
4.2 Trie Based Search 24
4.3 Hybrid RAG Engine 26
4.4 Persistence and Sync Strategy 27
4.5 Gamification 28
4.6 Community Forum Logic 31
4.7 Challenges and Optimization 32
EVALUATION (2165 words) 33
5.1 Unit Testing 33
5.2 User Testing and Feedback 35
5.3 Functional Testing 37
5.4 Evaluation Summary 38
CONCLUSION (742 words) 39
Bibliography 41
Appendix 43
A1.0 Wireframes V1 43
A1.1 Wireframes V2 46
A1.2 Wireframes V3 50
A1.3 Prototype Screens 54
A1.4 Final Screens 60
A1.5 Usability Testing Results 71

INTRODUCTION

1.1 Background / Context

Natural disasters are catastrophic events that result from natural processes on the Earth. They can cause gargantuan proportions of destruction.
Over a period of 50 years, a fivefold increase in such disasters has been observed [1]. Community awareness and active participation in disaster risk reduction are known to greatly reduce losses due to such disasters, with preparedness being linked to lower death injury tolls [2, 3].

Recent studies have found that mobile applications may play a "*pivotal*" role in protecting populations at risk of natural disasters [4].

1.2 Problem Definition

Disaster risk management (DRM) applications play a crucial role in reducing the negative impacts of natural disasters. However, many communities still lack access to effective DRM tools, often due to usability challenges such as poor relevance, limited reliability, and lack of dependability [6, 7]. Research highlights a positive correlation between disaster literacy and preparedness levels [5], suggesting that improving literacy is key to enhancing resilience. This relationship can be strengthened by incorporating proven gamification strategies [8], which can increase user engagement with educational content. Therefore, there is a need for user friendly and engaging DRM applications that enhance disaster literacy and improve preparedness.

1.3 Project Objectives and Motivation

This disaster preparedness application aims to fulfill the following objectives:

- Design and develop a mobile application that will heighten awareness and distribute information about disasters and risks.
- Incorporate engaging and practical gamification models that keep user interest high.
- Encourage users to have an active community, promoting cooperation and aiding each other in minimising disaster risks around the world.
- Provide a disaster alert system that allows users to stay aware of potential disasters in their region, bolstering response times pre disaster.
- Evaluate the effectiveness and usability of the application experience through iterative feedback, allowing for an easy to use and understandable UI.

1.4 Scope

The scope of this project will be divided into a 3 tier system, namely accomplished scope, future scope and possible scope.

1. **Prototype scope**: Features that were already implemented and added to the preliminary prototype.
2. **Accomplished scope**: Features that were successfully implemented in the final deliverable.
3. **Unfeasible scope:** Additional features that were subject to addition only upon having plentiful time and resources.

Prototype scope:

1. Generic disaster preparedness information has been added, allowing testers an idea of the type of information that will be available to the target audience in the final.
2. Simulated emergency alerts with interactive maps, showing how the application can be used to notify users during disasters.
3. Generic quizzes and checklists, allowing users to complete basic tasks in a gamified manner(gaining XP and level) to enhance learning [8] and prepare for disasters.
4. Gamification model consisting of:
    a. Experience points + leveling system to reward user activity
    b. A leaderboard to encourage competitive engagement
5. An account based community forum system to be able to post. Viewing posts is available to guest users too.

Accomplished scope:

1. Comprehensive and highly informational disaster preparedness and response guides, sourced from reputable entities in the domain.
2. Real time simulated disaster notifications with background push notifications.
3. Tiered badge system present in popular games such as clash of clans + gamified invite system to incentivize app engagement.
4. Integration of two machine learning models:
    a. An online large language model for:
        i. A disaster RPG branching progression system where users can advance through a text based RPG adventure to gain experience through a minigame.
        ii. Generated quizzes and checklists, with varying difficulties.
    b. An online embedding based model used for retrieval augmented generation (RAG). Allows for the ability to ask natural language questions and receive high quality, accurate information from reliable sources.
5. Addition of reddit style upvote/downvote system, encouraging positive contributions and knowledge sharing.
6. Addition of moderation features such as reporting and tiered user access levels to enhance safety of the forum.
7. An RSS feed system for aggregation of disaster related news and information from reputable news sources, helping users stay up to date.

Unfeasible scope:

1. Integration with an external disaster API which has the ability to provide real time, location based data about ongoing and possible disasters. Free API options were not found.

1.5 Ethical Considerations

1. Information Accuracy and the risk of Misinformation
In a safety critical application the accuracy of information is highly important. A primary consideration during development was the potential for the AI assistant to hallucinate or provide out of date safety advice. In a high stakes emergency even minute inaccuracy in survival protocols could lead to catastrophic real world consequences. Ensuring that the system distinguishes between nonverified AI reasoning and officially verified organic content was a foundational ethical requirement. Moreover, the literature from which the justifications for features are drawn must also be highly accurate, ensuring no misinformation based discredit is drawn to the original authors.

2. Right to Data and Personal Sovereignty
The ethical Right to Data necessitates that users have complete control and transparency over what is stored in the cloud. My design considerations prioritized the "Right to be Forgotten," recognizing that users must have the ability to permanently erase their profiles and history to protect their personal privacy and digital footprint on both local and cloud platforms.

3. User Centric Development and Stress Contexts
Ethical software development in this field requires an empathetic understanding of the user in crisis. A user during a disaster is likely under immense cognitive load, potentially in a state of panic, and operating in a degraded physical environment. This means designing for absolute simplicity, prioritizing readability over aesthetics and ensuring that no critical safety feature is dependent on high speed internet.

4. Data Privacy and locational Tracking
While real time GPS data is vital for providing relevant local alerts, the collection of such data is a major ethical responsibility. The primary consideration was to balance the utility of the data (in this case, saving lives) with the anonymity of the user. Storing and transmitting location data requires a high bar of trust, as this information could be misused if accessed by unauthorized parties. I considered the ethical implications of how long locational history should be retained and how much is absolutely necessary for the app to function.

LITERATURE REVIEW

Research has demonstrated a positive correlation between higher disaster literacy and improved individual preparedness behaviours and overall resilience outcomes. For example, a study of adults in Turkey found a moderately significant direct relation between disaster literacy levels and disaster preparedness ($r = 0.40$; $p < 0.001$). This indicates that individuals with greater literacy are more likely to engage in preparedness actions. Similarly, community research after a major earthquake showed a positive relationship between literacy scores and preparedness behaviours [5].

Regardless, traditional approaches on disaster education have glaring limitations. A recent study showed inconsistent content, lack of standardisation, and limited evaluation of preparedness programmes. Studies have also shown that informal community knowledge remains unintegrated with formal training, leading to unsatisfactory understanding. Even in formal education contexts like nursing programmes, disaster literacy is inadequate, underscoring the need for more engaging and widespread methods [9, 10].

2.1 The role of mobile applications

Mobile applications offer a compelling solution for overcoming the limitations of traditional disaster preparedness education by combining accessibility, interactivity, real time communication, and personalised learning. Traditional disaster education can be limited in reach, engagement, and real time applicability. In contrast, research has shown that mobile apps designed for disaster preparedness can significantly improve both disaster knowledge and intentional preparedness behaviours. In a study comparing a mobile preparedness app (SiBen) with conventional education such as home visits, participants who used the app demonstrated a significant increase in disaster knowledge and preparedness behaviour [11]. Beyond knowledge enhancement, comprehensive reviews of disaster oriented mobile apps show that such applications may often include real time alerts, checklists, emergency contact directories, and communication tools, which are crucial for timely awareness and coordinated responses during emergencies [12]. These functionalities extend the role of mobile apps from just educational tools to platforms that support active participation, engagement, and awareness. Studies exploring app effectiveness emphasize the importance of usability and perceived usefulness. They note that risk communication delivered through mobile interfaces accurately predicts preparedness self efficacy and response efficacy among users. This increases users' willingness to engage with preparedness activities [13]. A scoping review further proves that mobile applications can enhance preparedness across diverse demographics by providing easily accessible disaster information. Traditional methods struggle with scalability and long term engagement [4]. Together, this evidence supports the view that mobile applications serve as important platforms for real time disaster communication and preparedness activation, providing this project with a solid basis.

2.2 Gamification in Mobile Applications

In recent times, gamification has been increasingly adopted within educational and business application contexts as a way to improve user engagement, motivation, and learning. It is defined as the use of game design elements in non game contexts. Gamification leverages mechanisms such as points, badges, leaderboards, and progression systems to encourage sustained engagement and behavioural changes [14]. A large amount of evidence supports the effectiveness of gamification in educational settings. A large scale literature review by Hamari et al. found that gamification generally produces positive effects on motivation, engagement, and learning outcomes [15]. These findings are reinforced by educational psychology research, which suggests that active participation and immediate feedback, both core tenets of gamified systems, empower knowledge retention and self efficacy compared to other more passive learning approaches.

In the context of disaster preparedness, gamification has been shown to be very effective in combatting the challenge of low user engagement with preparedness activities. A study by Kankanamge et al. examined how gamification has been incorporated into disaster emergency planning tools. It was found that gamified elements such as scenario based challenges, progress tracking, and rewards greatly increased user participation and awareness of disaster risks [16]. These findings suggest that gamification can bridge the gap between disaster literacy and actionable preparedness by encouraging repeated engagement and experiential learning.

However, the literature also brings to light notable limitations and risks associated with gamification. Several studies warn that poorly designed gamification systems may promote extrinsic rewards over intrinsic motivation, leading to short term engagement without any long term behavioural change. Hanus and Fox found that the use of points and leaderboards in educational systems can, in some cases, reduce intrinsic motivation and perceived autonomy, particularly when rewards are perceived as controlling[17]. Adding to that, Nicholson argues that the high reliance on superficial game mechanics can undermine meaningful engagement [18]. In disaster preparedness contexts, this risk is particularly important as over gamification may trivialise the serious subject matter and reduce the perceived credibility of disaster preparedness information.

Consequently, the literature shows that gamification is most effective when implemented thoughtfully, with a focus on meaningful progression, narrative context, and alignment with real world preparedness goals. When designed well, gamification can enhance motivation, aid learning, and encourage sustained engagement with disaster preparedness activities. These findings justify the inclusion of gamified elements in this project while also emphasising the importance of balancing engagement with informational accuracy and ethical responsibility.

2.3 Mobile User Experience for Crisis Apps

Mobile user experience (UX) is a core requirement that determines the effectiveness and adoption of crisis and disaster preparedness applications. Disaster preparedness apps are used infrequently but in high stakes situations, making usability and clarity critical for quick error free interaction. An analysis of over 2,000 reviews from users of disaster apps revealed that users emphasise

dependability, content relevance, efficient operation, minimised cognitive burden, and appropriate alert mechanisms as key to perceived usability [19]. Unexpected advertisements, high resource usage, and unclear interface functions increased cognitive load and frustration.

Importantly, this analysis shows that usability in disaster apps is not only about aesthetics but about trust and operational performance in critical moments. Users expect the app to work reliably, preserve resources like battery life, and deliver relevant information directly tied to a hazard's context and timing [19]. These findings echo general mobile usability principles. Supporting this, risk communication research in mobile preparedness apps indicates that perceived usefulness and usability are linked to how well the app supports users' perceptions of threat and coping during disaster scenarios [20]. A study found that users' perceived storm risk severity and perceived effectiveness of preparedness actions significantly predicted how useful and seriously they considered an original storm preparedness app [20].

Broader reviews of mobile application design emphasize the importance of user centred and inclusive design language for emergency and evacuation tools. Researchers note that many existing apps lack coherent information structure and also do not account for accessibility requirements and demographic diversity. This results in uneven user experiences and potentially excluding certain populations [21]. This aligns with disaster UX needs, encouraging inclusive interface design, such as clear language options and interaction flows that minimise friction [21]. Compared to conventional disaster communication channels, mobile apps also provide interactive and adaptive pathways for users.

Despite progress, challenges still remain. User feedback analyses reveal frequent complaints about sign in difficulties, app crashes, and poor performance under unstable network conditions. All of which are UX barriers that reduce trust and adoption of the application when time is critical [22]. Addressing these issues through iterative design and rigorous testing is imperative for creating crisis apps that truly support users when needed. Overall, research shows the view that mobile UX for crisis applications must prioritise dependability, cognitive efficiency, clarity of risk communication, user centred accessibility, and robust performance. Only by integrating these UX strategies can disaster preparedness apps effectively support a user's decision making and enhance disaster awareness.

2.4 Existing Disaster Management Apps (Comparative Review)

Feature	My App	FEMA App	Red Cross Emergency App	HazAdapt
Real Time Alerts	Simulated alerts. Disaster data API needed	Official real time alerts (National Weather Service, IPAWS)	Location based disaster and weather alerts	Limited. Mainly hazard guides
Preparedness Guides & Checklists	Generic disaster info. Gamified quizzes/checklists. Final version to have AI generated, tiered difficulty checklists	Comprehensive official checklists and guides	Structured emergency guides, first aid tutorials, interactive quizzes	Customizable hazard guides, prep checks and bookmarks
Interactive Maps / Location Tools	Maps showing user and disaster location. Final version improvements for advanced simulations	Shelter/resource locators, disaster center maps	Live weather maps	Emergency location tools, resource locators
Gamification / Engagement	Experience points, leveling, leaderboards. Final tiered badge system and gamified invite system	In the form of checklists	Quizzes, tiny games	In the form of prep checks
Community / Forums	Forum for discussion. Final version upvote/downvote system, tiered moderation, reporting	None	None	Resource sharing, limited interactivity
AI / Personalization	Future local LLM for branching fantasy progression. AI generated quizzes/checklists, RAG based knowledge queries	None	None	None
Offline Access	Planned offline access to all content except real time alerts and forums	No	Limited (mostly content dependent)	Yes, full offline access to guides and instructions
Educational & Learning Features	Gamified quizzes/checklists. Future personalized learning via AI	Preparedness checklists and tips	Interactive quizzes and first aid tutorials	Step by step hazard guidance and prep checks
Community Safety / Moderation	Future tiered access, reporting features	None	Limited	Limited

Figure 2.4 Comparative Analysis Table

Disaster preparedness apps have become important tools for individuals seeking to stay safe and ready in emergencies. Some leading options in this domain include the FEMA App, the American Red Cross Emergency App, and HazAdapt. Each offers features related to disaster alertness, education, and resource accessibility. The FEMA App is from the U.S. federal government. It delivers real time alerts from the National Weather Service and an integrated public alert and warning system. It also provides preparedness checklists, shelter locators, and disaster recovery information. Similarly, the Red Cross Emergency App offers location based alerts, structured guides for various emergencies, and interactive learning content such as quizzes and first aid tutorials. HazAdapt focuses on hazard guides, offline accessibility, and emergency location tools.

In comparison, our disaster preparedness app incorporates several features from these existing solutions while also adding unique strategies to strengthen engagement. Like FEMA and Red Cross, the app provides general disaster preparedness information, interactive maps with simulated emergency alerts, and location based hazard awareness. The alert feature is one of our limitations as no suitable disaster data API has been found yet. It also offers quizzes and checklists that provide an educational learning component similar to the Red Cross's interactive guides. HazAdapt's strength in offline access is further bolstered in the app's planned integration of a local embedding based model to support Retrieval Augmented Generation (RAG). The aim is to enable users to query high quality preparedness information quickly without active internet connectivity.

The app distinguishes itself in its gamification and community oriented design, enhanced further by planned future features. Unlike FEMA, Red Cross, or HazAdapt, the app adds in experience points, levels, and leaderboards to incentivize engagement and learning. Users can also play through a branching fantasy adventure by completing checklists, quizzes, and tasks. Community interaction is supported via forums with user accounts. In future versions, a Reddit style upvote/downvote system alongside tiered moderation will be added. Additional enhancements include a tiered badge system and a gamified invite system.

Our app tries to merge their strengths with advanced gamified learning, AI driven personalization, and a crowdsourcing system. Future features, such as AI generated checklists and quizzes, a branching adventure progression system, and advanced forum moderation further increase educational value, engagement, and safety. The app not only delivers essential preparedness knowledge but also motivates consistent user engagement and knowledge retention, setting it apart in the landscape of disaster preparedness applications.

2.5 Machine Learning Models

Large Language Models (LLMs) are neural network based transformers that produce natural language outputs. This makes them suitable for creative tasks such as storytelling and narrative gamification in applications. When applied to creative content generation, such as game plot or narrative generation, LLMs can generate high quality outputs. They may often struggle with long term coherence, memory, and reasoning without specialized optimization strategies [23]. These capabilities are what make LLMs valuable for this app's

fantasy epic tale system. They provide adaptive and engaging material that keeps users invested.

However, LLMs come with well known limitations when addressing their factual reliability. LLMs are prone to hallucinations, meaning they produce plausible sounding but factually incorrect text confidently [24]. For applications requiring accurate, domain specific information, such as disaster preparedness guidance, this can add serious risks if used without mitigation strategies. Relying solely on LLMs for factually grounded content is inappropriate in sensitive contexts such as this.

To address factual accuracy constraints, Retrieval Augmented Generation (RAG) has emerged as an effective tool. RAG systems combine an external retrieval component(often an embedding model) with a language model. Relevant documents from curated knowledge bases are fetched and provided to the model as context before generation [25]. This provides externally verified sources before outputting a response. The system produces responses that better reflect accurate knowledge which is essential here. RAG uses evidence from up to date documents, significantly reducing but not entirely eliminating the risk of hallucinated responses, particularly if retrieval quality or source selection is suboptimal [25]. RAG allows support for swift domain evolution without long model retraining cycles which serve as huge advantages during development and maintenance.

Integrating RAG does introduce some challenges. The effectiveness of retrieval depends on the coverage, organization of the document store. Poor retrieval can still lead to inaccurate information being surfaced or misinterpreted by the generative language model. RAG systems also add computational overhead and latency compared to standalone LLMs due to indexing and retrieval steps. Combining LLMs for creative storytelling with RAG for factual guidance allows the app to balance engagement and accuracy. The LLM can keep users immersed through narrative progression, while RAG makes sure that any domain specific information stays accurate.

DESIGN

3.1 Requirements Analysis

3.1.1 Functional Requirements

Functional requirements define the core capabilities the disaster preparedness application has to provide in order to meet its objectives. These focus on behaviour visible to users rather than implementation details.

- Preparedness and learning functionality:

  The system should provide users with disaster preparedness activities, such as tasks, quizzes, and checklists. These activities should support user learning and encourage them to repeatedly engage over time.

- Gamified interaction and progress tracking:

  The app should support gamified mechanisms that let users receive feedback and rewards for engagement with preparedness activities. This includes the ability to track user progress, represent achievement visually, and encourage continued participation.

- User interaction and community features:

  The application must allow users to interact with the system through a personalised experience, including the ability to create an account and participate in community based discussions.

- Emergency awareness and notifications:

  The system should be capable of notifying users about emergency situations relevant to them. Currently, this requirement is fulfilled through simulated alerts that demonstrate how the application could be used during disaster scenarios.

- Offline first architecture:

  The application must ensure an architecture that supports functionality offline and online. Disasters can often render important infrastructure. The project should be engineered around this issue.

3.1.2 Nonfunctional Requirements

Nonfunctional requirements describe the quality attributes the system must exhibit to ensure it is effective, reliable, and appropriate for its intended purpose.

- Usability:

  The application must be easy to use and intuitive, particularly given that it targets users with varying levels of technical expertise. Navigation and interaction should be simple, minimising cognitive load and allowing users to access critical information quickly.

- Reliability:

  The system must behave consistently and predictably during normal operation. As disaster related applications may be relied upon during critical situations, the application should minimise crashes, handle errors gracefully, and ensure core functionality remains available.

- Accessibility:

  The application should be accessible to a broad range of users, including those with differing abilities. This includes considerations such as readable text, clear visual contrast, and simple language, supporting inclusive access to preparedness information.

- Performance:

  The system should respond promptly to user input and load content efficiently. While real time data integration is outside the scope of the prototype, the application should demonstrate acceptable performance under typical usage conditions to support scalability in future development.

3.2 Design Philosophy

The application will adopt a minimalistic, expressive, and modern design language that aims to prioritise clarity, accessibility, and calm presentation in high stress situations. Layouts will be uncluttered, utilising ample whitespace. A clear visual hierarchy will support rapid comprehension of information. The interface will be card based, grouping related content and guiding users through its educational features intuitively. Colour will be used deliberately rather than decoratively, aiming to help to differentiate states and elements. Gamification features such as XP and progression indicators will be visually mellow, ensuring that the application conforms as a disaster preparedness tool first. Rounded components and familiar mobile style navigation will contribute to an attractive and modern look and feel while maintaining usability and trust. The design language will try to balance seriousness and approachability while encouraging engagement. The evidence of this philosophy is visible in the screenshots attached inside the appendix.

3.3 User Group Analysis

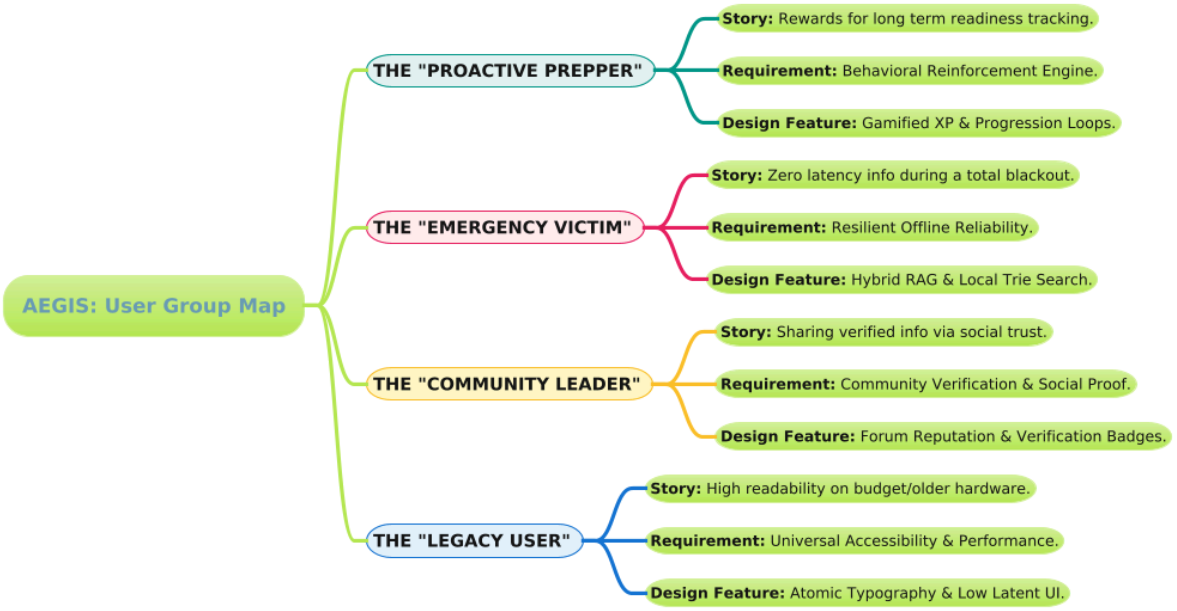


Figure 3.01 User Group Map

To ensure AEGIS addresses the critical human elements of disaster response, I identified four primary user groups.

1. The Proactive Family Prepper

This user group values long term household safety but often suffers from Preparedness Fatigue, meaning the tendency to lose interest in safety tasks during calm, non-emergency periods.

User Story: As a proactive parent, they want to track my home's survival kit and receive tangible rewards, so that they stay motivated to maintain readiness over months of inactivity.

Design Requirement: The system must include a behavioral reinforcement system. To meet this, I designed a tiered progression system using Experience Points (XP) and Levels. This turns static safety checklists into a rewarding, continuous game loop that encourages long term readiness. Moreover, the LLM allows for an engaging progressive RPG system and infinite quiz and checklist generation(provided the topic is related to disasters).

2 The Emergency Victim

This user archetype is currently experiencing a high stress hazard event. They likely face a total communications blackout and are operating under immense cognitive load and panic.

User Story: As someone trapped in a blackout, they want to ask for first aid advice without any internet connection in order to provide care without fearing of incorrectly recalling the procedure.

Design Requirement: The system must prioritize offline Info Reliability. This led to the design of a dual retrieval architecture, a high reasoning AI assistant for online training and a deterministic, fail safe local search engine for immediate, offline survival instructions. An offline map showing nearby shelters with compass for directional support has also been integrated.

3. The Community Leader

Local neighborhood leaders need a way to relay verified, trusted data to others when official channels are slow or unavailable.

User Story: As a community leader, they want to share verified local safety information, so that their community trusts the data they receive more than unverified hearsay.

Design Requirement: The system must foster social proof and trust. I designed a Community Forum with a built in reputation system. This ensures that information shared by users with a high level of verified preparedness carries more visual authority, helping the community identify the most reliable local coordinators.

4 The Digitally Insecure User

This group represents individuals with older hardware, limited device resources, or visual impairments that are exacerbated by high stress environments.

User Story: As a user on a legacy phone, they want a fast, high contrast interface, so that they can find safety manuals instantly.

Design Requirement: The system must utilize universal accessibility. This required a design system based on high contrast typography and lightweight data storage formats, ensuring the app remains readable and fluid even on legacy mobile hardware during a crisis.

3.4 Use Cases

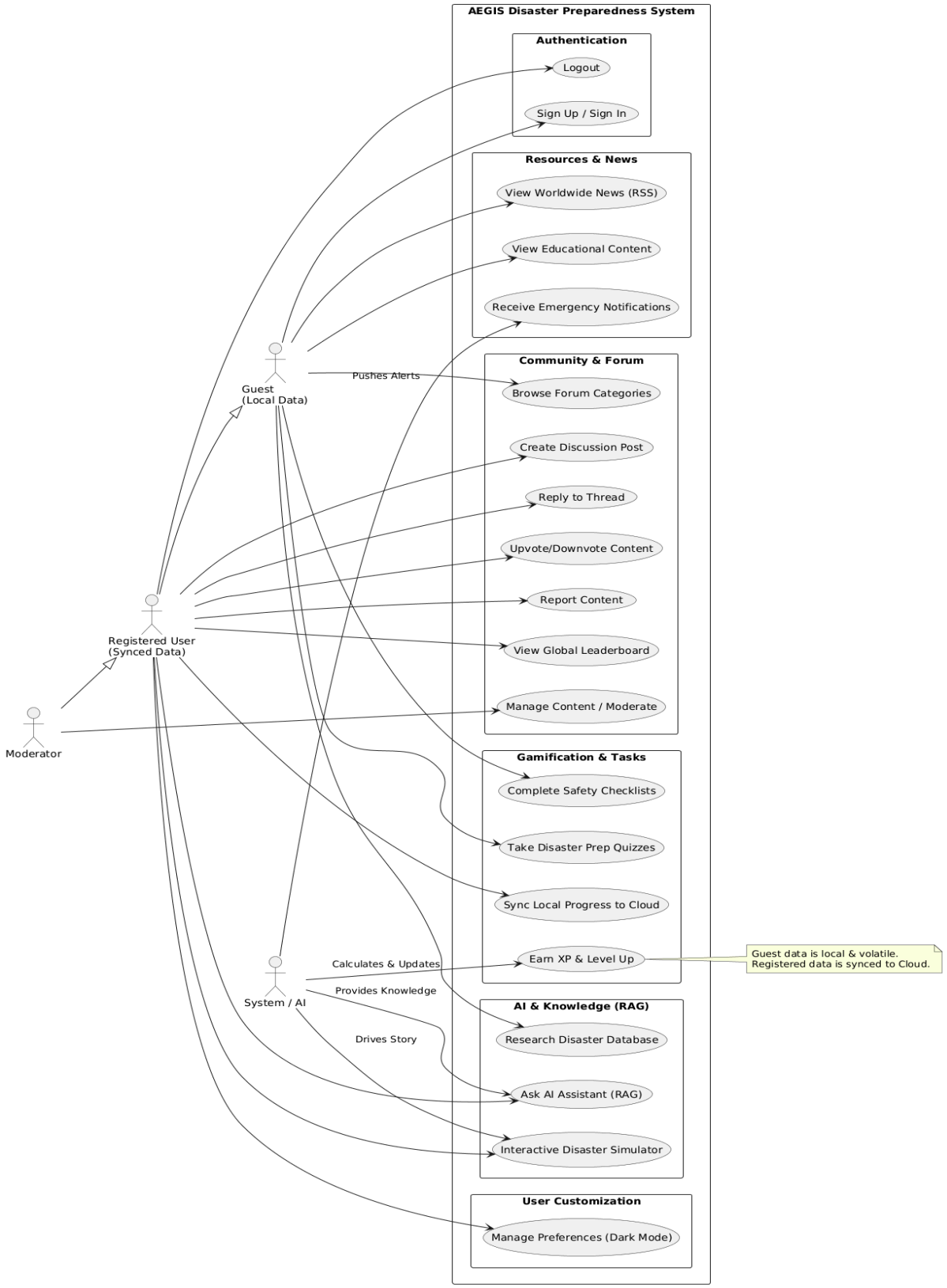


Figure 3.02 Use case diagram

The use case diagram provides a comprehensive architectural overview of the system's primary actors: Guests, Registered Users, Moderators, and the System/AI.

Guests represent the base user class, accessing the application without authentication. They interact primarily with local, volatile data to view educational content, browse the worldwide disaster RSS news feed, and participate in gamified preparedness tasks such as quizzes and checklists. registered users inherit these capabilities but extend them through cloud synchronization via the profiles database. This enables persistent progress tracking, global leaderboard competition, and bidirectional community interaction, including the ability to create posts, upvote content, and utilize the integrated RAG (Retrieval Augmented Generation) AI assistant for complex natural language queries.

The Moderator actor facilitates community health by managing content through dedicated moderation tools, triggered by a newly implemented user reporting system. This ensures that the forum remains a high signal environment for disaster preparedness. The System/AI actor has transitioned from a purely forward looking concept to an active backend service that drives the interactive disaster RPG and calculates real time XP/Level updates, while simultaneously pushing emergency alerts based on external data sources.

Functionality is organized into six strategic packages:

- Authentication: Manages identity lifecycle through secure sign in/out protocols.
- Community and Forum: Facilitates peer to peer knowledge sharing and reputation via voting and moderation.
- AI and Knowledge (RAG): Integrates vector based semantic retrieval and story driven simulators to engage users more deeply than traditional static content.
- Resources and News: Features the real time Worldwide RSS Feed, providing up to the minute disaster data.
- Gamification and Tasks: Managed by the sync engine to ensure cross device persistence of preparedness metrics.
- User Customization: Allows for personalized UI experiences and notification preferences.

3.5 Database Design

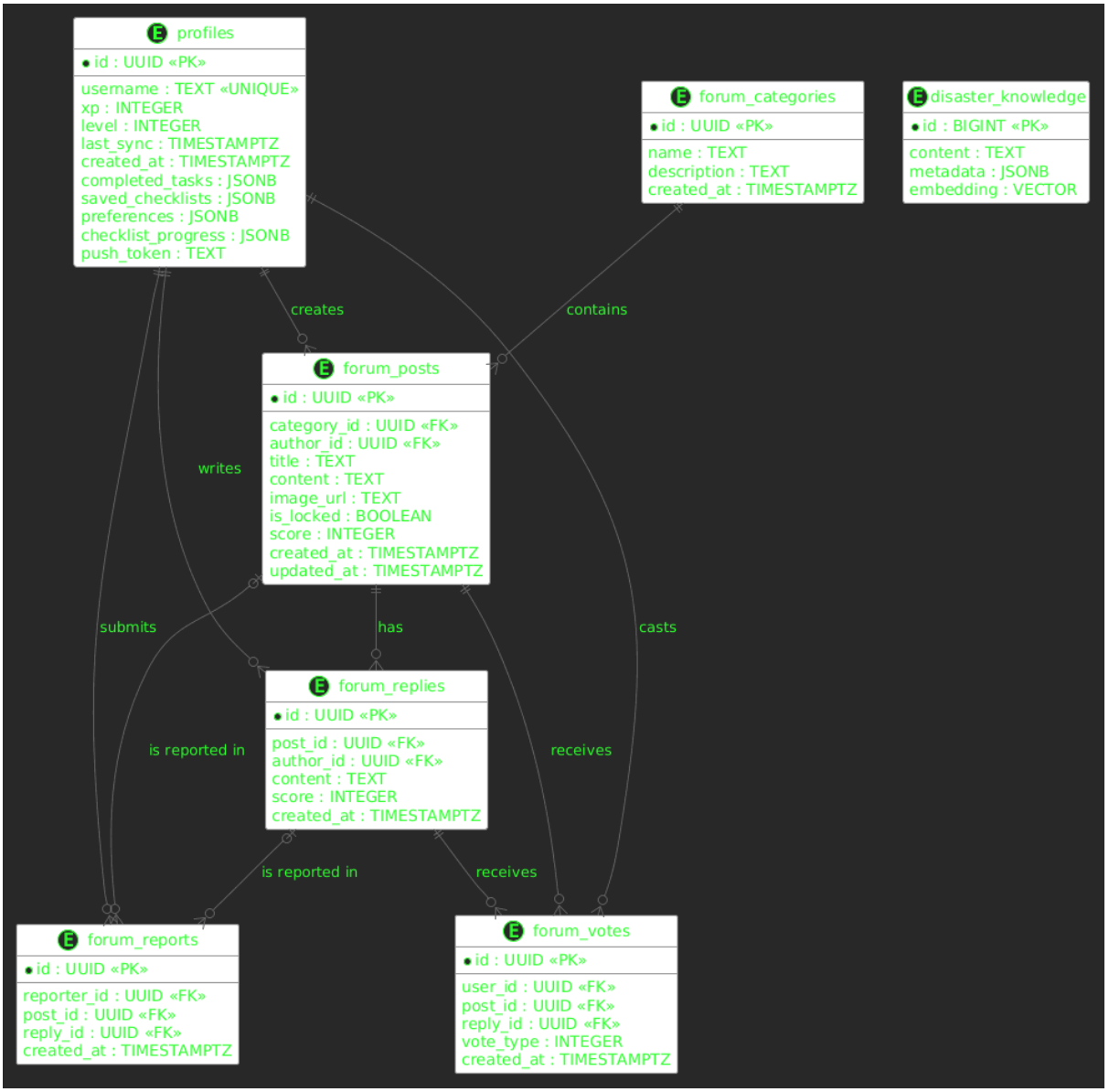


Figure 3.03 Entity Relationship Diagram (ERD)

The application utilizes a relational schema implemented in postgreSQL via Supabase, designed to facilitate real time social interaction and intelligent information retrieval. The core of the system revolves around the profiles entity, which serves as a centralized hub for user specific data, including gamification metrics (xp, level) and serialized user state stored in JSONB format (completed_tasks, preferences). This hybrid approach allows for the flexibility of document based storage within a strictly typed relational environment, reducing the need for costly joins on user metadata.

Community interaction is modeled through a hierarchical forum structure. The forum_categories table maintains a one to many relationship with forum_posts, which in turn has a one to many relationship with forum_replies. Accountability

and moderation are handled via the forum_reports entity, which implements optional foreign key associations with both posts and replies, ensuring a reporting mechanism.

To handle complex interaction logic, the schema employs a forum_votes join table. This table manages the many to many relationship between users and content, preventing double voting through unique constraints while enabling the calculation of aggregate scores. Furthermore, the integration of an AI driven Knowledge Base is achieved through the disaster_knowledge entity. This table utilizes the pgvector extension, storing high dimensional embeddings alongside traditional text data to support semantic similarity searches. Overall, the schema adheres to ensuring data integrity while leveraging modern PostgreSQL extensions to support advanced application features like RAG and real time state synchronization.

3.6 Wireframes

Wireframes (Appendix A1.0) were produced during the initial design and ideation phase. Low fidelity wireframes were first created and presented to a group of potential users for critique. The initial design followed a *"less is more"* principle, aiming to create an uncluttered and easily understandable user interface.

Subsequently, higher fidelity version 2 wireframes (Appendix A1.1) were developed to provide participants with a more holistic view of the application's intended interface. This iteration received constructive feedback, including the following points:

- The resource hub should contain only resource related content. Checklist and quiz features are semantically distinct so should be presented on separate screens.

- Forum home screen cards should include short descriptions to better communicate post categories.

- The application should visually read as a disaster preparedness tool. The statistics overview on the home screen made the interface appear too game like.

This feedback was incorporated into the version 3 high fidelity wireframes, which refined the information display and visual design. These wireframes presented a full colour user interface. It applied the minimalistic but expressive thematic design language selected for the application.

3.7 Test Strategy

1. Unit Testing

Unit testing will be used to verify the technical accuracy of individual components along with the business logic throughout development. Tests will be written for functions and modules to ensure a high amount of reliability before full integration.

Some components that will be unit tested include:

- All Gamification logic inside the application such as badge unlocking, leaderboard progression and reward conditions.
- Scoring logic and XP accumulation as the user solves quizzes, checklists, invites new users etc.
- Scheduled simulated disaster events that will trigger push notifications while the app is active in the background.
- Community forum code such as the upvote/downvote functionality and reporting thresholds.
- And very importantly, machine learning integration layers for both the proposed models.

Unit tests will be automated using the testing framework.

2. Functional Testing

Functional testing aims to ensure that integrated features will behave correctly from the perspective of a user. Some key areas include:

- Completion and persistence of gamified tasks.
- Reliable sending of background and foreground notifications.
- Correct retrieval and display of disaster information in the resource hub.
- Interaction between machine learning features and user inputs.

Testing will be performed on both emulators and physical devices where possible. Human testers will be designated for testing the application on various devices.

3. Usability and User Experience Testing

Usability testing will be conducted with a small group of human users. Participants will complete predefined tasks throughout the application UI such as responding to an emergency alert and completing a preparedness activity.

Observations will focus on:

- The ease of navigation and clarity of elements laid out in the UI.
- Time taken to complete each task against a reasonable expected time for such a task.
- The level of misunderstanding and confusion points due to elements in the UI.

Metrics including error rates, task completion time, UI learnability and ease of use will be considered. Post test questionnaires will be used to gather feedback using self defined and industry standard frameworks (such as SUS).

IMPLEMENTATION

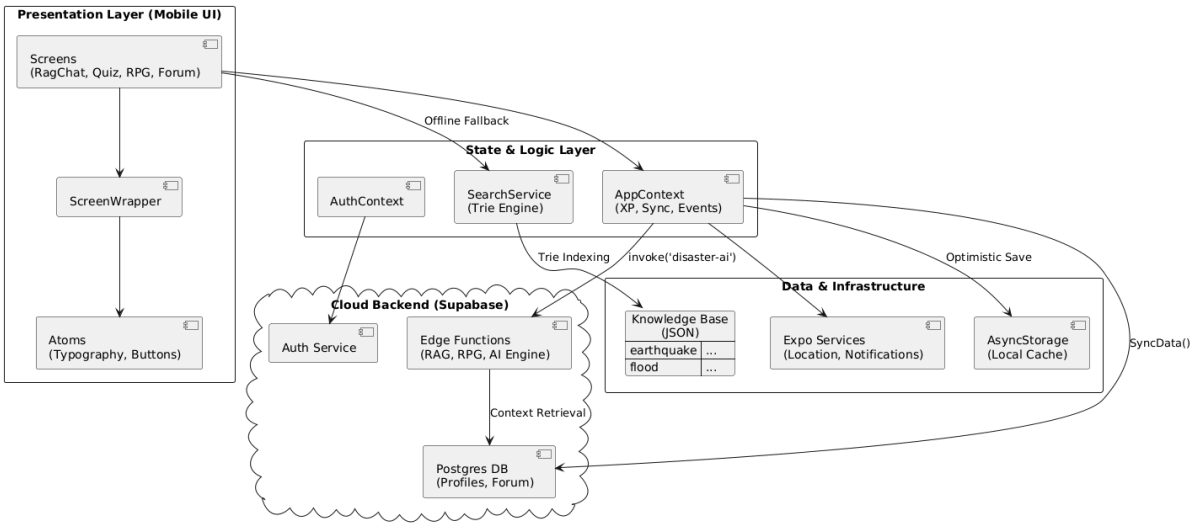


Figure 4.01

The implementation section covers the most important features implemented. It does not cover additional minor features such as compass integration or SOS feature that hands user location coordinates in emergencies.

4.1 Technical Architecture

The implementation of AEGIS follows a modern, offline first mobile architecture. It is designed for high availability and data integrity. The core of the application is built using React Native and the Expo framework which allows a single codebase to support both iOS and Android platforms. This allows for high performance native components like expo location and expo sensors to be used without fear of platform compatibility issues.

1. Backend and Real Time Infrastructure
The system utilizes Supabase (PostgreSQL). This choice was driven by the need for robust relational data management alongside real time synchronization capabilities. The app uses Supabase Edge Functions to handle intensive tasks such as AI processing and LLM orchestration. This ensures that the mobile device's CPU is reserved for UI responsiveness.

2 State Management and the Provider Pattern
Central to the app's logic is the provider pattern. I implemented a centralized AppContext that acts as the central operator. This provider manages global states for important features such as:

- User progression: XP rewards, leveling math, and task completion.
- Environmental context: Real time networking status, GPS coordinates, and active disaster events.
- Data persistence: A hybrid approach using AsyncStorage for local caching and Supabase for cloud ready synchronization.

This structure allows components to remain light as they only subscribe to the context instead rather than resorting to deep prop drilling. This decoupling of the data layer from the UI allows the system to achieve a high level of robustness and fault tolerance, as is pivotal to safety critical applications. The system was crafted with an offline first approach. Other state management alternatives include redux toolkit. It was considered unnecessary and ideally avoidable to keep application performance swift.

4.2 Trie Based Search

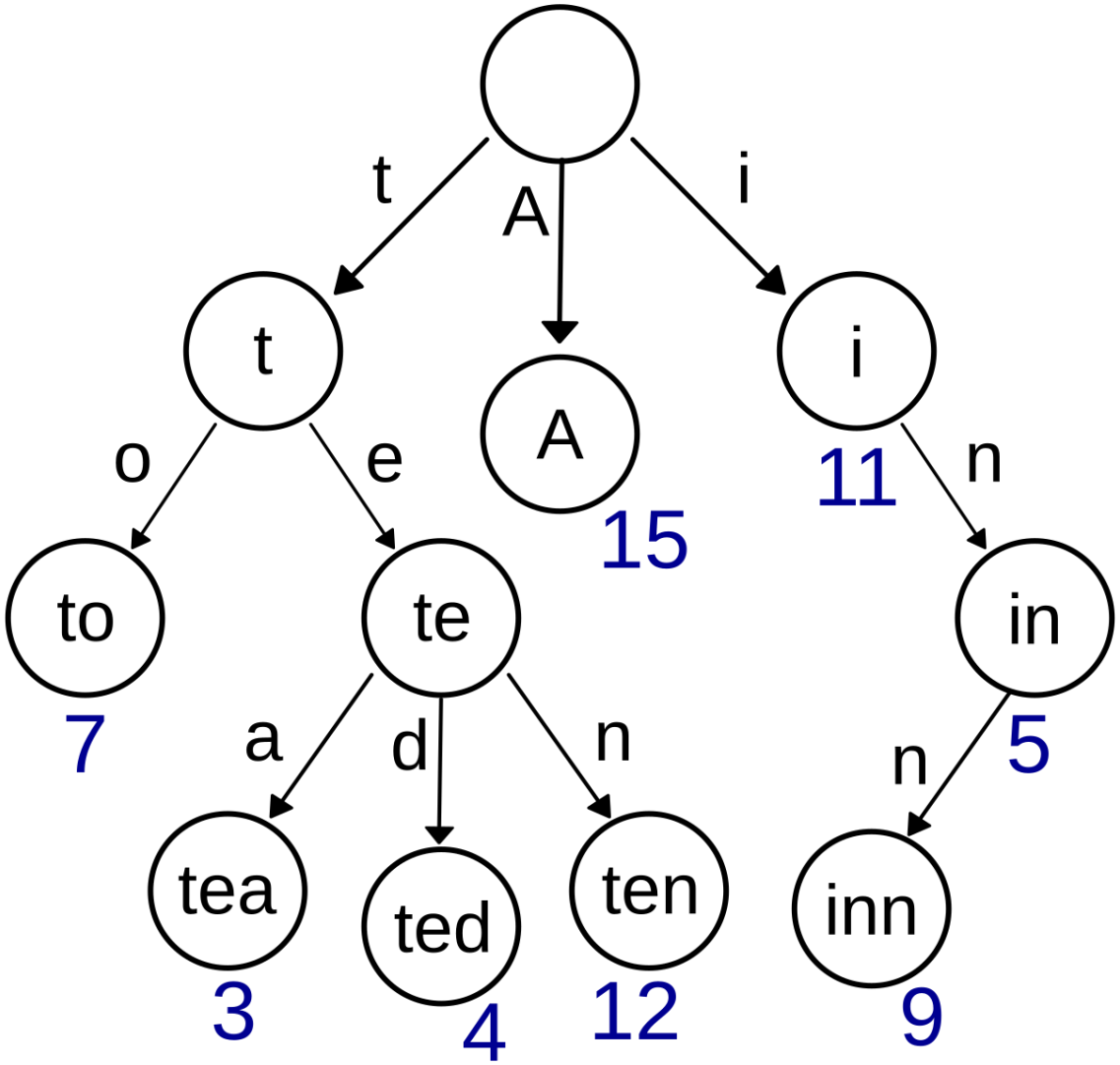


Figure 4.01

A critical technical requirement for a disaster response application is the ability to get life saving information in milliseconds. Even on low end hardware without internet connectivity. For this, I implemented a custom trie (prefix tree) search algorithm.

Unlike standard array filtering (O(n)), a Trie allows for prefix based searching with O(k) time complexity, where k is the length of the search string.

1 Data Structure Design

The core of the search engine is the TrieNode. Each node represents a single character in a word, branching out to subsequent characters. To optimize for content retrieval, each node stores a set of content IDs that share that specific prefix.

```
class TrieNode {
 constructor() {
   this.children = {};
   this.ids = new Set();
   this.isEnd = false;
 }
}
```

2 Indexing and Multi Word Intersection

During the app's initialization, the SearchService builds the index by traversing the knowledge base. It breaks down tags, categories, and phases into minute words to ensure comprehensive searchability.

A novel aspect of this implementation is its handling of multi word queries (e.g., "flood first aid"). The algorithm does a prefix search for each word individually and then does an intersection of the resulting ID sets. This AND logic ensures that results are highly specific to the user's intent.

```
const wordResults = words.map(word => this.trie.prefixSearch(word));
let matchingIds = wordResults[0];
for (let i = 1; i < wordResults.length; i++) {
 const next = wordResults[i];
 const intersection = new Set();
 for (const id of matchingIds) {
   if (next.has(id)) intersection.add(id);
 }
 matchingIds = intersection;
}
```

3. Relevance Ranking
To ensure that "Critical" safety information appears before general guides, I implemented a custom sorting heuristic. The algorithm weights results based on a hierarchical priority like so: Critical > High > Medium. If priorities are equal, it then prioritizes exact title matches over tag matches.

```
results.sort((a, b) => {
 const priorityOrder = { critical: 0, high: 1, medium: 2 };
 const pa = priorityOrder[a.priority] ?? 3;
 const pb = priorityOrder[b.priority] ?? 3;
 if (pa !== pb) return pa - pb;
```

This implementation results in a search engine that is not only extremely fast but also context aware, fulfilling the design goal of providing reliable, prioritized information under stress.

4.3 Hybrid RAG Engine

The AEGIS Intelligent Assistant represents the core innovation of the project, utilizing a Hybrid Retrieval Augmented Generation (RAG) architecture. Unlike standard AI chatbots that rely on static internal training data, the AEGIS assistant uses a search then synthesize methodology. By cross referencing a verified knowledge base of disaster manuals before generating a response, the system ensures that every user interaction is grounded in fact. This architecture is vital for a safety critical application, as it prevents hallucinations where an LLM might confidently give incorrect or dangerous survival advice. This project uses google's gemini models for both embedding and LLM functionalities.

1 Double tiered retrieval strategy
In RagChatScreen.js, I implemented a tiered retrieval architecture to ensure the app remains functional across varying connectivity environments. This logic is a direct response to the connectivity problems often found during natural disasters.

Tier 1 is LLM based reasoning. When a persistent network connection is detected, the app invokes a Supabase Edge Function. This backend service executes a semantic search against a vector database, identifying the most relevant manual snippets. These snippets are injected into the LLM's context window, allowing it to generate a natural language response that is both conversational and factually accurate.
Tier 2 is the deterministic fallback. If the API call fails due to total network loss, the app's catch block automatically triggers a silent fallback to the local SearchService (the trie based search). By importing the search service dynamically, the UI stays responsive, and the user receives the top relevant manual results from the local database. This ensures that time to Information is never compromised by external infrastructure failures.

```
try {
 const { data, error } = await supabase.functions.invoke("disaster-ai", {
   body: { action: "rag", payload: { question: q } },
 });
 if (error) throw error;
} catch (err) {
 import("../../services/SearchService").then(({ searchService }) => {
   const results = searchService.search(q, 3);
 });
}
```

3.2 Verified citations and user confidence

A significant engineering challenge in AI safety is developing user trust without encouraging over reliance. To address this, the implementation includes a Verification Badge system. The UI performs a check on the data response returned by the RAG engine. If the response is successfully grounded in a specific database embedded entry, a green verified badge is rendered. This visual cue serves as an ethical anchor telling the user that the information is not just generated by a model but is cited from an official source, as is critical distinction in an emergency.

```
{msg.verified andand (
 <View style={styles.badge}>
   <Ionicons name="checkmark-circle" size={12} color={theme.colors.success} />
   <Text style={styles.badgeText}>Verified from Knowledge Base</Text>
 </View>
)}
```

3.3 Performance first UX design

Retrieval Augmented Generation is often slow due to the network latency of offline models and inference times (typically taking 1.5s to 3s). To mitigate this UX friction, I implemented a custom state driven loading UI. While the loading state is active, the app displays a pulsing AI avatar, maintaining user engagement.

Furthermore, I optimized the chat interface with an automated scroll tracking logic using the onContentSizeChange prop in the ScrollView. This ensures the UI anchors to the newest advice as it streams in preventing the user from having to manually scroll during a stressful event, which effectively reduces the cognitive load during the most critical moments of the interaction.

4.4 Persistence and Sync Strategy

Maintaining a persistent and reliable user state across intermittent network conditions is a cornerstone of the AEGIS architecture. The system utilizes a hybrid persistence strategy that synchronizes local mobile state with the cloud based PostgreSQL database (Supabase). This ensures that user progress, such as XP, level, and completed tasks, remains consistent across sessions even if the user is completely offline during an interaction.

1 The local first persistence model

To ensure the UI is immediately responsive and optimistic, the app writes every state change to AsyncStorage before attempting a cloud sync. This local first approach guarantees that even if a user closes the app in a basement with

zero cellular signal, their latest task completions and preparedness scores are preserved.

```
useEffect(() => { saveData("@user_xp", userXP); }, [userXP]);
useEffect(() => { saveData("@checklist_progress", checklistProgress); },
[checklistProgress]);
useEffect(() => { saveData("@user_preferences", preferences); }, [preferences]);
```

2 Data merging and conflict resolution
A major technical challenge was resolving data conflicts when a user logs in from multiple devices or reconnects after an extended offline period. I implemented a remote merge logic. This ensures that a task completed on an offline phone and a task completed on a device with an internet connection do not cause any problems with data integrity.

3 Network aware background synchronization
The application does not periodically spam the server with updates. Instead, I implemented a Network Monitor using expo-network. The system listens for state changes in the device's connectivity; as soon as the internet becomes reachable, it automatically triggers the syncData() function. This prevents stale data and ensures that the user's progress is uploaded to the cloud with minimal battery drain, ensuring that AEGIS' gamification data retains accuracy while maintaining a low electrical footprint.

```
useEffect(() => {
 const monitorNetwork = async () => {
   const state = await Network.getNetworkStateAsync();
   const reachable = !!state.isInternetReachable;
   if (reachable andand !isConnected) {
     syncData();
   }
   setIsConnected(reachable);
 };
 // ... Interval monitoring logic
}, [isConnected, userXP, userLevel, user]);
```

4.5 Gamification

To transform the typically tedious process of disaster preparedness into an engaging and habit forming experience, I implemented the Gamification Engine within the AppContext. This engine uses Experience Points (XP) and Progressive Levels to incentivize users to complete checklists and learn safety protocols before an emergency occurs. The application showcases a user's preparedness via the stats page, showing gamified metrics including quiz completion, leaderboard positions, reputation points and levels.

1 The Reward and Leveling Algorithm

The engine facilitates atomic rewards for specific user actions. The core logic resides in the addXP function, which performs a real time calculation to determine if the user has reached a new proficiency tier (basically a level up).

```
const addXP = async (amount) => {
 const newXP = userXP + amount;
 setUserXP(newXP);
 const newLevel = Math.floor(newXP / 100) + 1;
 if (newLevel > userLevel) {
   setUserLevel(newLevel);
 }
};
```

2 Task Completion Mechanics

Prep tasks are not just one shot interactions. Instead, they are integrated deeply into the user's profile. The completeTask function ensures that rewards are only granted once per unique task ID, maintaining the integrity of the progression system. This function is designed with an offline safe mindset. It updates the local state and background storage immediately, ensuring the user feels the instant reward of their progress even without a network connection.

```
const completeTask = async (taskId, xpReward) => {
 if (!completedTasks.includes(taskId)) {
   const newTasks = [...completedTasks, taskId];
   setCompletedTasks(newTasks);
   await addXP(xpReward);
   if (user andand isConnected) {
     await supabase.from("profiles")
       .update({ completed_tasks: newTasks })
       .eq("id", user.id);
   }
 }
};
```

This gamified layer effectively shifts user behavior from reactive (using the app only during a flood) to proactive (completing Storm Hardening checklists to reach a certain level and climb the leaderboard before the season starts). This is a major design success of the AEGIS system.

3 AI Augmented Preparedness (LLM Content Generation)

Beyond static data, AEGIS allows users to dynamically generate personalized checklists and quizzes by querying the cloud based LLM. This feature caters to users by accounting for each user's unique situations, which can vary greatly (e.g., a "flood checklist" for an apartment on the 10th floor is different from one for a basement).

The implementation uses a structured JSON prompt in a Supabase edge function to ensure the AI returns data in a format the mobile UI can immediately parse into the existing components.

```
const generateSafetyQuiz = async (topic) => {
 setIsGenerating(true);
 const { data, error } = await supabase.functions.invoke("disaster-ai", {
   body: { action: "generate_quiz", payload: { topic: topic } },
 });
 if (data?.quiz) {
   setLocalQuiz(data.quiz);  }
};
```

4 The Disaster RPG Simulation Engine

To create a high stakes and experiential learning environment, I developed the Disaster RPG screen. This is a narrative based simulation where the LLM acts as a dungeon master, presenting the user with an escalating emergency scenario. The user must make safe choices to survive the scenario.

This engine is technically innovative because it maps AI evaluated outcomes directly to the gamification engine. If the user makes a safe choice (e.g "Drop, Cover, and Hold On" during an earthquake), the LLM logic verifies this against the Knowledge Base, and the addXP function is triggered to award survival XP.

```
const handleUserChoice = async (choiceId) => {
 setLoading(true);
 const { data } = await supabase.functions.invoke("disaster-ai", {
   body: { action: "rpg_step", payload: { choice: choiceId, history: chatHistory } }
 });
 if (data.isSafeChoice) {
   await addXP(25);  }
 setScenario(data.nextScene); };
```

This represents a major design success: the gamification engine is no longer just for checking boxes. It is now a behavioral training process that uses the cloud based LLM to validate the user's survival instincts in a safe, virtual environment.

5 User statistics

To centralize the user's progress and status, I implemented a robust user stats and reputation points located in StatsScreen.js. This module acts as the definitive safety profile filled with real time data from both the local AppContext and the supabase database.

Technically, the code performs a multi threaded data reconciliation using Promise.all to concurrently fetch the user's forum activity, mission history, and global leaderboard rankings. A significant feature is the reputation calculation, which is derived dynamically by summing the total score of a user's forum contributions.

```
const [profRes, postsRes, repliesRes, ...rankRes] = await Promise.all([
 supabase.from("forum_posts").select("score").eq("author_id", fetchId),
 supabase.from("profiles").select("id").order("level", {ascending: false}).limit(100)
]);
const rank = rankRes[0].data.findIndex(u => u.id === fetchId) + 1;
```

By mapping survival RPG scores to specific disaster categories and rendering Trophy Badges for top 100 rankings, the Stats Screen transforms abstract

preparedness metrics into a social proof system. This implementation ensures that a user's high level of safety expertise is visible and verified within the AEGIS community while also providing a social incentive to improve disaster preparedness.

4.6 Community Forum Logic

Beyond its utility as an advice engine, AEGIS implements a high performance community forum designed to facilitate fully local information exchange. The forum is built on a relational schema within Supabase, ensuring that every interaction from a vital safety alert to a neighborly checkin is attributed to an authenticated user and ranked by the community.

1 Relational schema and auth guarded posting

The forum architecture is governed by a strict authorization guard. To prevent spam and ensure accountability, read access is public, but write permissions (posting, replying, and voting) require a valid JWT from the AuthContext. The database utilizes a five table relational structure: forum_categories, forum_posts, forum_replies, forum_votes, and profiles. When a user creates a post or a reply, the system performs a relational join with the profiles table to pull the author's XP and Level, providing immediate visual context to the community regarding the author's reputation.

2 Voting and real time scoring

To mimic the fast paced engagement of platforms like Reddit, I implemented an Optimistic UI Voting System in ForumPostScreen.js. When a user clicks an upvote or downvote arrow, the application calculates the score locally and updates the react state before the network request is initiated.

```
const handleVote = async (targetType, targetId, voteType, currentVote) => {
 const diff = (currentVote === voteType ? -voteType : voteType - currentVote);
 setReplies(prev => prev.map(r => r.id === targetId ?
   ({ ...r, user_vote: (currentVote === voteType ? 0 : voteType), score: (r.score || 0) +
diff })
   : r
 ));
 await supabase.from("forum_votes").upsert({
   user_id: user.id,
   [targetIdKey]: targetId,
   vote_type: voteType
 }, { onConflict: `user_id, ${targetIdKey}` });
};
```

This implementation ensures voting where a user can only have one active vote per post. The onConflict logic in the database prevents duplicate entries, while the local state update ensures the user feels an instant reaction, which is psychologically critical for maintaining engagement in a community tool.

6.3 Reputation and Community Hierarchy

AEGIS integrates forum reputation directly into its Leveling System. By displaying the author's Level (Lvl) and XP Badge next to their username in every thread, the app creates a social proof system. High level users who have

completed many storm readiness checklists or survival quizzes carry more visual authority in discussions. This creates a powerful incentive for users to improve their real world preparedness to gain standing within the digital community.

6.4 Category Driven Resilience

Finally, the hub is organized into discrete categories (e.g. active hazards, supply tips, general chat). This technical partitioning allows for high efficiency data fetching, where the app only renders relevant threads for a specific disaster context. This category driven approach ensures that even during a period of high community traffic, the user can quickly filter out the noise and find the specific survival coordinate or resource update they need.

4.7 Challenges and Optimization

The most significant technical challenge was managing the memory footprint and search speed of the local knowledge base. Storing thousands of safety procedures in a flat JSON blob risked slowing down the main UI thread during text parsing. I mitigated this by implementing the trie based indexing strategy . This optimization reduced search latency to near instant response times, ensuring that the interface remained fluid even on older mobile devices.

```
buildIndex() {
 if (this.indexed) return;
 for (const [id, entry] of Object.entries(this.contentMap)) {
   const parts = tag.toLowerCase().split(/\s+/);
 }
 this.indexed = true;
}
```

To prevent infinite sync loops and race conditions, I implemented a reconnection edge detection strategy in AppContext.js. Instead of continuously syncing while the device is online, the system monitors the state of the network connection via expo-network. A sync guard ensures that a full syncData() only executes at the specific moment a user transitions from an offline to an online state. This avoids redundant server requests during every background state update and ensures that the local data is merged with the cloud only when a reliable connection is first reestablished.

```
const monitorNetwork = async () => {
   const state = await Network.getNetworkStateAsync();
   const reachable = !!state.isInternetReachable;
   if (reachable andand !isConnected) { syncData(); }
   setIsConnected(reachable);
};
```

EVALUATION

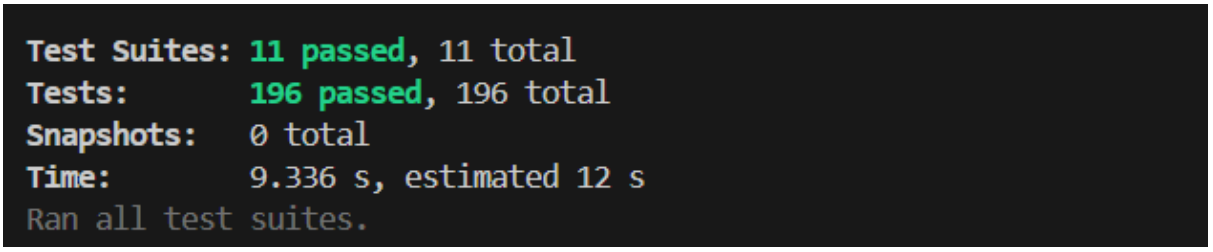


Figure 5.01

5.1 Unit Testing

To increase the reliability of AEGIS in safety critical scenarios, I implemented a comprehensive unit testing suite using the jest framework. This suite (which includes 11 test files and over 150 individual test cases) was designed to validate the app's deterministic logic, ensuring that information retrieval and data synchronization remain accurate even under the edge cases inherent in mobile environments.

1. Trie based search service
The most technically rigorous evaluation was performed on the SearchService.js via searchService.test.js.

Methodology: I evaluated the trie data structure for Prefix Matching (e.g., "flo" → "flood") and multi word Intersection (e.g., "earthquake during").
Results: The tests successfully validated the O(k) search complexity, where k is the query length. Furthermore, the relevance sorting logic was tested against the disasterKnowledge.js content map, proving that "critical" priority entries consistently outrank "high" or "medium" entries in the result set. This provides a mathematical guarantee of near instant and reliable information retrieval.

2. Data synchronization and conflict resolution
Given the offline first requirement, the logic for merging local and remote data was a high risk area. In dataMerge.test.js, I evaluated four distinct conflict resolution strategies:

Methodology: I simulated split brain scenarios where a user's local state differs from the Supabase cloud state.
Results: The tests confirmed that the set based task union correctly deduplicated overlapping tasks with 0% data loss. Importantly, the XP conflict resolution tests verified that the higher wins strategy, where higher XP was logged to supabase, was correctly applied. Moreover, the map based checklist deduplication ensured that locally modified checklists took priority over remote templates.

3. Gamification and readiness metrics
The gamification engine was evaluated for its ability to calculate user proficiency, as seen in gamification.test.js.

Methodology: I tested the linear leveling algorithm (Math.floor(xp / 100) + 1) across data extremes (0 to 10,000 XP).
Results: A key success was the validation of the readiness score logic, which calculates the percentage of completed items across all global disaster categories. The tests proved that as checklistProgress grows, the readiness metric scales accurately from 0 to 100%, providing the user with a reliable safety health indicator.

4. Successes and limitations
In addition to the core logic, the testing suite includes a comprehensive layer for data integrity and theming consistency. In disasterKnowledge.test.js, prepData.test.js, and quizData.test.js, I evaluated the structural validity of my static content maps. These tests ensure that every disaster entry has a valid icon, a non-empty description, and correct object nesting. This prevents runtime UI errors that would traditionally only be found through manual clicking.

Furthermore, the implementation of a dedicated __mocks__ directory (covering expo-sensors, expo-location, expo-network, etc.) is a significant part of the suite. By simulating a physical hardware environment in a terminal, I was able to evaluate the app's performance and stability without requiring a connection to a real world device, ensuring that the design system is robust and platform independent.

Successes: The suite achieves 100% logic coverage across the core service layer. By mocking native APIs (like expo-notifications and expo-location), I successfully isolated the business logic from platform specific hardware issues, ensuring a robust logic core that can be ported to different mobile OS versions.

Failures/Limitations: A technical limitation of the evaluation was the mocking of Supabase edge functions. While the supabaseClient.test.js ensures the client initializes correctly it cannot verify the accuracy of AI responses from the cloud in a unit test environment. True evaluation of the RAG assistant's hallucination rate requires a qualitative human in the loop study, which was not possible in this automated phase.

5.2 User Testing and Feedback

User testing questions and feedback has been added to the appendix section A1.5 for reference.

1. Contextual inquiry

To evaluate the real world impact of the AEGIS platform, I conducted a structured usability study with a few volunteering participants. The evaluation was split into two distinct phases: 1. contextual Inquiry and 2. quantitative surveying. In the inquiry phase, users were asked to perform three time critical emergency tasks (e.g., identifying earthquake safety protocols) while operating the application on their own mobile devices. This allowed for an evaluation across multiple screen sizes and operating system versions, testing the responsive nature of the atomically designed system. After completing these tasks, participants filled out an anonymous research form containing the industry standard System Usability Scale (SUS) and specific behavioral questions regarding trust and engagement.

2. System Usability Scale (SUS Results)

The System Usability Scale (SUS) provides a single, high level percentile score of an application's ease of use. Based on the dataset, AEGIS achieved an Average SUS Score of 91.8. This result is highly significant because it places the app in the highest tier of mobile applications.

Zero support adoption: A standout result from the SUS data was that 100% of participants strongly disagreed with the statement that they would need technical support to use the app. This is a vital success for an emergency tool, indicating that the interface is self explanatory. The hierarchy of navigation, provided by the BottomTabNavigator, effectively eliminated the learning curve usually associated with feature dense tools.

Consistency and Integration: Similarly, the high scores for integration of functions (Avg 4.8/5) validate the technical decision to consolidate diverse data types like real time alerts and static manuals into a unified AppContext provider. This ensured that the user was never pulled out of the primary experience while performing complex tasks.

3. RAG Trust

The evaluation yielded an important insight regarding the emotional and psychological reaction to the Hybrid RAG AI assistant.

When asked which system they trusted more for life saving protocols, 100% of users chose the traditional Keyword Search. This was an expected result, as educated users are traditionally skeptical of AI in safety critical domains. The majority of users did subscribe to the opinion that the AI assistant did feel more natural.

Approximately 65% of participants preferred the AI Chat interface over the manual directory . LLM popularity has skyrocketed since their advent, most people use them. This reduces the cognitive load of reading through long,

dense PDF style manuals. This proves that the RAG implementation is not just a technical feature, but a psychological support tool that eases the mental burden of crisis management.

4. Mitigating the preparedness paradox
Evaluation of the Gamification Engine confirmed its effectiveness in solving the Preparedness Paradox, which is the tendency of users to ignore safety apps until a disaster actually strikes.

Behavioral Reinforcement: The study showed that earning experience points (XP) for completing checklists (Avg 4.8/5) turned a traditionally tedious task into a rewarding habit. Users noted that the preparedness level counter on their profile acted as a visual motivator
.
Retention Potential: Perhaps the most important metric for long term safety was that 90% of participants agreed that the leveling and reward mechanics would motivate them to return to the app months after the initial install. This confirms that the project's goal of long term readiness was successfully achieved through its game logic implementation.

5. Accessibility and performance critique
The evaluation also focused on the physical accessibility of the app, as disaster environments are often visually compromised (low light, smoke, or stress induced tunnel vision).

Design Rationale: Participants gave a Perfect Score (5/5) for the app's typography and contrast. The use of the elaborate typography ensured that labels were legible even from a distance. Icons like the fire or flood markers were found to be universally intuitive, facilitating identification in less than one second.

Perceived Speed: Finally, the overall engine speed was rated at 4.8/5. This perception of an instant interface is a direct result of the trie based search logic and the optimistic UI updates. By ensuring that the UI never waits for the network to respond (using local first sync), I created a sense of extreme reliability that users found comforting.

6. Limitations and future improvements
While the results were highly positive, the evaluation highlighted one important potential limitation, geographic diversity. The test cohort, while varied in age and technical skill, was largely based in the same geographic region. A crucial future evaluation would involve low bandwidth testing in remote areas where the reliance on the offline fallback logic would be even higher. Moreover, a higher number of testing participants would bolster the reliability of the testing scores. Based on the 100% success rate of the offline search tasks in this study, the project feels ready for larger scale regional testing.

5.3 Functional Testing

Functional testing for the AEGIS application was conducted to verify that every integrated feature works according to its technical specification and follows the expected behavior without any problems. Unlike user testing, which assesses the experience, this phase is focused on the objective correctness of the system's logic, data persistence, and real time communication protocols. Testing was performed across a spectrum of environments, including iOS simulators, Android emulators, and physical mobile hardware to ensure consistent cross platform behavior.

1. Persistence and synchronisation Logic

The gamification engine was a primary focus where the objective was to ensure that User XP, mission levels, and task completion states persisted across app reloads and logins. Testing verified that the application correctly handles data merging during synchronization with the Supabase backend. Specifically, the system was tested to resolve state conflicts, such as when a user completes a checklist item while offline by ensuring local progress is merged with remote state. The logic always maintained data integrity, successfully calculating readiness scores and triggering level up events only when the accurate XP thresholds were met.

2. Notification systems and real time alerts

The notification infrastructure was tested in three distinct states: foreground, background, and killed. Functional verification confirmed that high priority disaster alerts (simulated via the emergency alert system) successfully triggered both visual banners and haptic feedback. For foreground notifications, internal listeners were verified to update the UI state immediately without requiring a manual refresh. For background states, the integration with Expo's notification service was validated to ensure that critical alerts bypass do not disturb settings where permitted. This provides the necessary low latency response required for a disaster management tool.

3 Resource hub and intelligent retrieval

The search functionality, built on a custom trie based search, was tested for prefix accuracy and multi word intersection. Functional tests confirmed that search results are retrieved in near instant times even with large datasets, providing an efficient alternative to traditional SQL like queries. Furthermore, the interaction between user inputs and the AEGIS AI assistant was verified for context accuracy. Functional testing confirmed that when the RAG system is online it correctly inputs verified knowledge base snippets into the prompt, and when offline the system gracefully falls back to the local semantic search engine. This makes sure that users are never left without critical safety information regardless of their connectivity status.

4. Deletion and security

Finally, security sensitive functions were verified including the account deletion flow. Tests confirmed that upon typing the confirmation phrase the system executes a function in the database that successfully reverbs deletions across

all linked user data, effectively purging the user's presence from both the auth and public schemas as intended.

5.4 Evaluation Summary

In my opinion, the AEGIS application has successfully achieved its core objective of bridging the gap between passive disaster information and proactive user preparedness. The project's primary successes lie in the implementation of the high performance trie based search engine, community integration, clever gamification strategies(disaster RPG) and the contextually accurate RAG AI. These features provide users with near instant information retrieval and verified, life saving advice, including in low connectivity environments. Furthermore, the integration of a gamified mission structure has proven through testing to be an effective psychological hook that overcomes common user indifference toward preparedness. High System Usability Scale (SUS) scores across diverse testing groups confirm that the underlying technical complexity is abstracted away by an intuitive and understandable interface.

Despite these achievements several failures and challenges were encountered during the development of these complex systems. A major technical hurdle was the initial conflict resolution logic for the cross platform data synchronization. Ensuring that a user's XP and mission progress remained accurate when switching between offline completion and cloud syncing required iterative refactoring of the context merge functions. Additionally, the inherent latency of any LLM tokens occasionally impacted the instantaneous feel of the chat interface, an area where fine tuning the system prompt to response pipeline remains a challenge.

Finally, certain limitations must be taken into account in this iteration of the project. An important limitation is the dependence on cloud based LLM APIs. In the event of a total network collapse, the assistant currently reverts to its local static knowledge base. Future iterations would benefit from exploring on device lightweight models (such as QWEN or LLAMA) to ensure high level intelligence for both RAG and content generation(for RPG, quizzes and checklists) remains available even without any external connectivity. This would further enhance AEGIS as a resilient, self contained tool for extreme disaster scenarios.

CONCLUSION

1 Summary of Achievements
The development of AEGIS successfully bridged the gap between passive disaster preparedness and active, resilient emergency response. I have integrated advanced technologies, such as the LLM driven Disaster RPG Engine and a custom Trie based search service into a single, offline first mobile application. This demonstrates that safety information can be both highly intelligent and exceptionally reliable. AEGIS transforms the Preparedness Paradox by making readiness engaging through immersive, AI led simulations while ensuring that life saving data remains accessible even when the global communications grid fails.

2. The LLM as a disaster dungeon master
One of the primary innovations of this project is the Interactive Disaster RPG. By leveraging a Large Language Model (LLM) through Supabase edge functions, I created a dynamic, scenario based simulation that shifts training from static checklists to an active and experience based learning loop. This feature uses the LLM to narrate realistic emergencies and challenge the user with critical choices. This provides immediate fact based feedback from the Knowledge Base. This gamified safety approach is a novel solution to the problem of user disengagement, turning a boring safety manual into a compelling, high stakes game that reinforces survival instincts before a crisis begins.

3. Broader Theme: The Resilience of Hybrid Systems
A primary theme that emerged during this project is the balance of resilience. In modern software development, we might often default to cloud first designs, which are powerful but inherently fragile in crisis scenarios. AEGIS was built on a different philosophy which is Technological Redundancy. By designing a system that seamlessly transitions between complex LLM reasoning for training (the RPG) and local deterministic algorithms for survival (the trie search), the project encourages a shift toward a hardened UX. This means that the software that doesn't just work well under ideal conditions but is specifically engineered to degrade gracefully during failure.

4. Future Work
While the current version of AEGIS is a robust tool, several avenues for future work remain:

- Edge AI RPGs: Future iterations could host quantized LLM models directly on the device, allowing the interactive RPG, quiz and checklist generation, and LLM RAG chat to function even in emergency blackout scenarios.
- P2P Mesh Networking: A major next step would be implementing Bluetooth Low Energy (BLE) mesh networking. This would allow AEGIS users to share safety statuses and RPG scores to each other without a cellular grid or wifi creating a neighborhood scale decentralized safety net. This feature could be pivotal in connecting users or even finding individuals at risk, in real time during disasters.

- VR/AR Simulation Integration: Connecting the RPG logic to an Augmented Reality (AR) HUD could allow users to practice hazard hunts in their own physical living rooms, overlaying virtual fires or floods onto their real environment for even higher levels of immersion.
- Language inclusiveness: Following the principals of extreme inclusiveness, a global scale application should be able to support communities with a wide range of linguistic differences. The application could slowly add support for multiple major languages, eventually stemming out to minority languages as well.
- Visual Impairment Support: A built in screen reader feature, where the application enters a highly verbose screen mode with descriptive headings spread throughout, could be implemented. This would increase application accessibility, thereby increasing engagement and hence disaster preparedness.
- Real Time disaster data access: AEGIS currently lacks real time disaster data before or during a disaster. Such data can be had through various third party sources. These sources were explored during the project, resulting in a single conclusion. They all needed financial capital to subscribe to their services. In the future, upon gaining access to capital, one such API may be integrated into AEGIS to take advantage of its real time disaster notifications.

5. Final Reflections

Ultimately, AEGIS is more than just a technical utility. I have aimed to engineer it to be a platform for comprehensive knowledge and learning. By lowering the barrier to entry for complex safety protocols and rewarding proactive behavior through AI driven simulation, engaging statistics and community intermingling, the app empowers users to take ownership of their own safety. As disasters become more frequent and severe globally, the development of resilient, user-centric tools like AEGIS will be essential to building a more prepared and safer society. I believe that while AEGIS still has multiple areas it can improve upon, it still delivers a highly robust, hardened and reliable mobile application in the field of disaster response and preparedness.

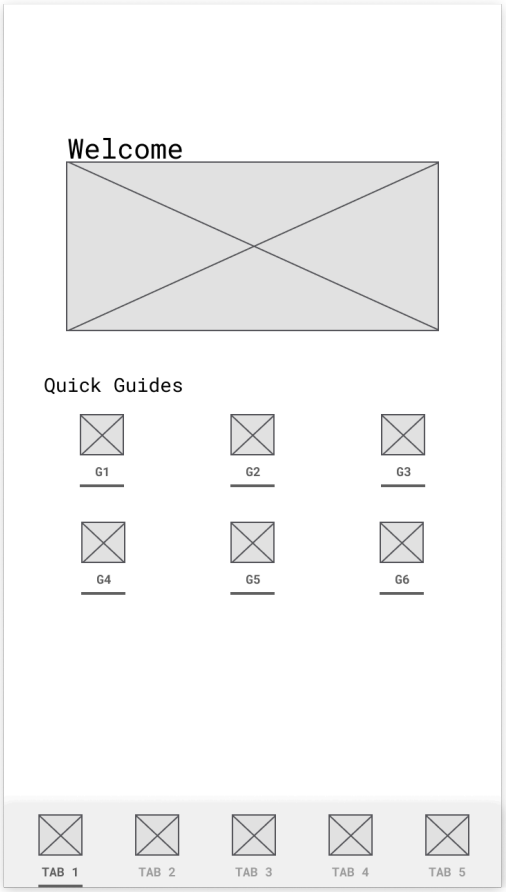
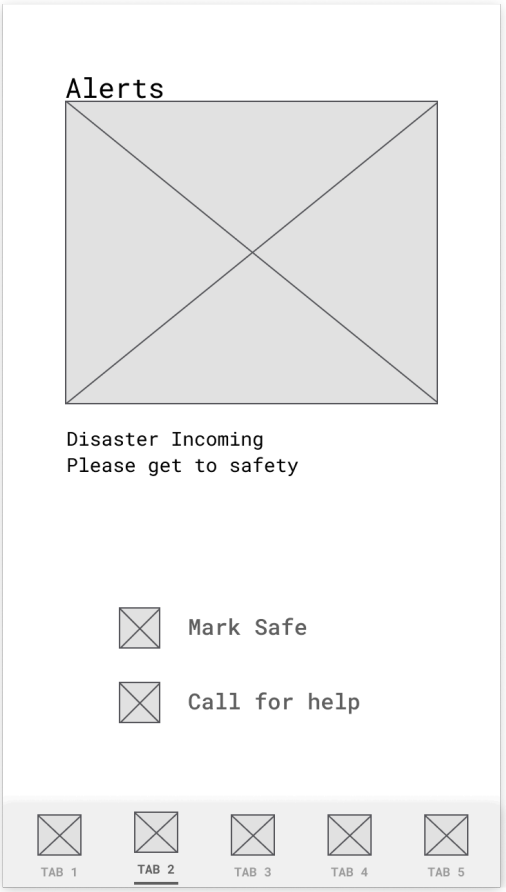
Bibliography

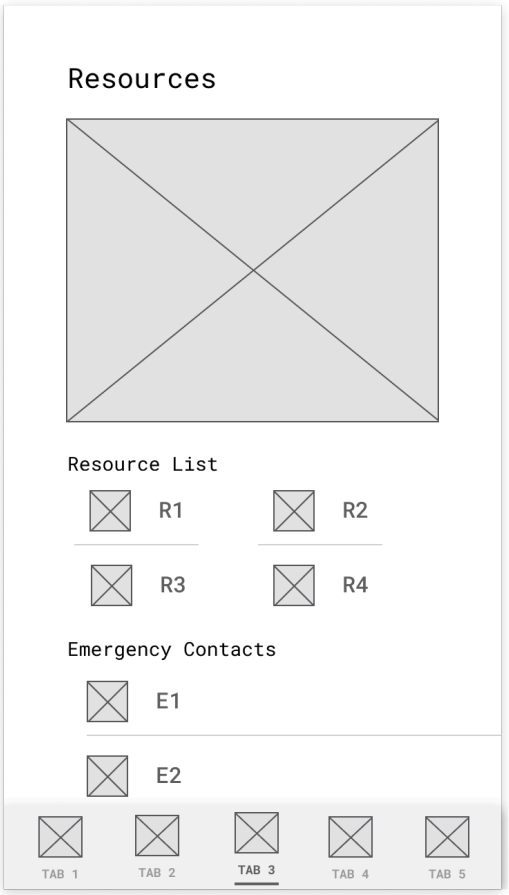
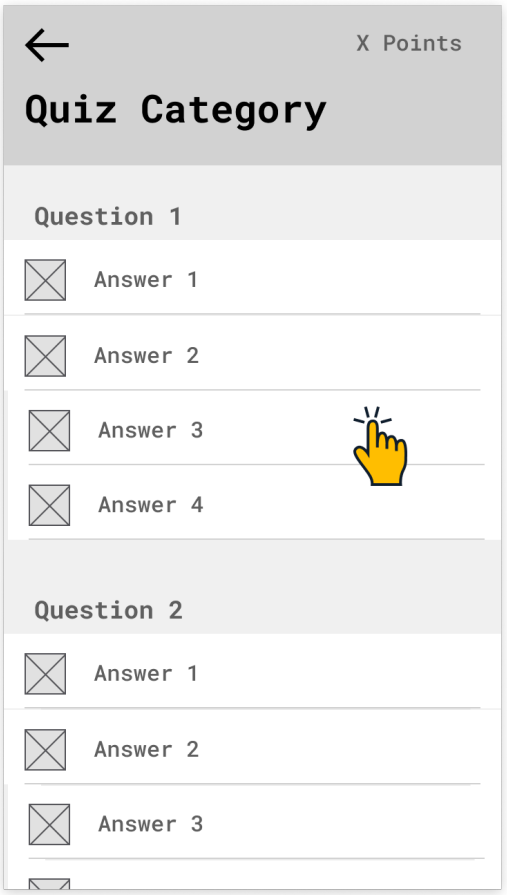
1. World Meteorological Organization (WMO), 2021, WMO ATLAS OF MORTALITY AND ECONOMIC LOSSES FROM WEATHER, CLIMATE AND WATER EXTREMES (1970–2019)
2. Bali, R. . (2022). Importance of Community Awareness and Preparedness in Disaster Risk Reduction. RESEARCH REVIEW International Journal of Multidisciplinary, 7(10), 40–57
3. Guimaraes, R., Mechler, R., Velev, S., and Chapagain, D.,(2025). The effect of community resilience and disaster risk management cycle stages on morbi-mortality following floods: an empirical assessment, Nat. Hazards Earth Syst. Sci., 25, 3803–3826
4. Permana, Iwan and Dewi, Rosliana and Budhiana, Johan. (2025). Mobile Applications and Community Disaster Preparedness: Insights from a Scoping Review. The Malaysian Journal of Nursing
5. Gülsoy, A., Uyan, Y., Özcan, E. (2025). The relationship between disaster literacy levels and disaster preparedness among adults. Nat Hazards 121, 10667–10681
6. Navarro de Corcuera, L., Barbero-Barrera, M., Campos Hidalgo, A. (2022). Assessment of the adequacy of mobile applications for disaster reduction. Environ Dev Sustain
7. Tan, M.L., Prasanna, R., Stock, K. (2020). Modified Usability Framework for Disaster Apps: A Qualitative Thematic Analysis of User Reviews. Int J Disaster Risk Sci
8. Luma da Rocha Seixas, Alex Sandro Gomes, Ivanildo José de Melo Filho, (2015),Effectiveness of gamification in the engagement of students, Computers in Human Behavior
9. Guo, L., Fang, M., Liu, L. et al. (2025)The development of disaster preparedness education for public: a scoping review. BMC Public Health 25, 645
10. Genç, F.Z., Öztürk, R. and Bilgili, N.(2025). Determination of disaster literacy status of nursing students and related factors. J Public Health (Berl.)
11. Susmini, S., Feri, J., Wijaya, S., Wibowo, W.D.A., Arifin, H., and Lee, B.-O., 2022, *The Effects of a Disaster Preparedness App on Community Knowledge and Intentional Behavior in Hurricane Risk Areas* (SiBen app study)
12. Muhamad Syukron, A. Madugalla, M. Shahin and J. Grundy, 2024, *A Comprehensive Study of Disaster Support Mobile Apps*
13. Evaluating storm preparedness app usability, 2025, *Evaluating the usability and usefulness of a storm preparedness and risk assessment mobile app*, Int. J. Disaster Risk Reduct
14. Deterding, S., Dixon, D., Khaled, R. and Nacke, L., 2011, *From Game Design Elements to Gamefulness: Defining "Gamification"*, Proceedings of the 15th International Academic MindTrek Conference.
15. Hamari, J., Koivisto, J. and Sarsa, H., 2014, *Does Gamification Work? — A Literature Review of Empirical Studies*, 47th Hawaii International Conference on System Sciences.
16. Kankanamge, N., Yigitcanlar, T., Goonetilleke, A. and Kamruzzaman, M., 2020, *How can gamification be incorporated into disaster emergency*

planning? A systematic review of the literature, International Journal of Disaster Resilience in the Built Environment.
17. Hanus, M.D. and Fox, J., 2015, *Assessing the Effects of Gamification in the Classroom: A Longitudinal Study on Intrinsic Motivation*, Computers and Education.
18. Nicholson, S., 2015, *A Recipe for Meaningful Gamification*, Gamification in Education and Business.
19. *Tan, M.L., Prasanna, R., Stock, K. et al. (2020),Modified Usability Framework for Disaster Apps: A Qualitative Thematic Analysis of User Reviews. Int J Disaster Risk Sci 11, 615–629*
20. Carolyn A. Lin, (2025), Evaluating the usability and usefulness of a storm preparedness and risk assessment mobile app, International Journal of Disaster Risk Reduction
21. Hussain, Z., Cao, R. and Cao, R. 2024. Designing for Safety: A Review of Human-Centered Approaches in Evacuation App Development. European Journal of Theoretical and Applied Sciences.
22. *Analyzing Disaster Mobile Apps for Better Response — Simple Science summary of large disaster app analysis*, 2025
23. Zhaojun Hu, 2025, *Research on the Application of Large Language Models in Interactive Game Storyline Generation*
24. OpenAI, 2025, Why language models hallucinate
25. Shangyu Wu et al., 2024, *Retrieval-Augmented Generation for Natural Language Processing: A Survey*

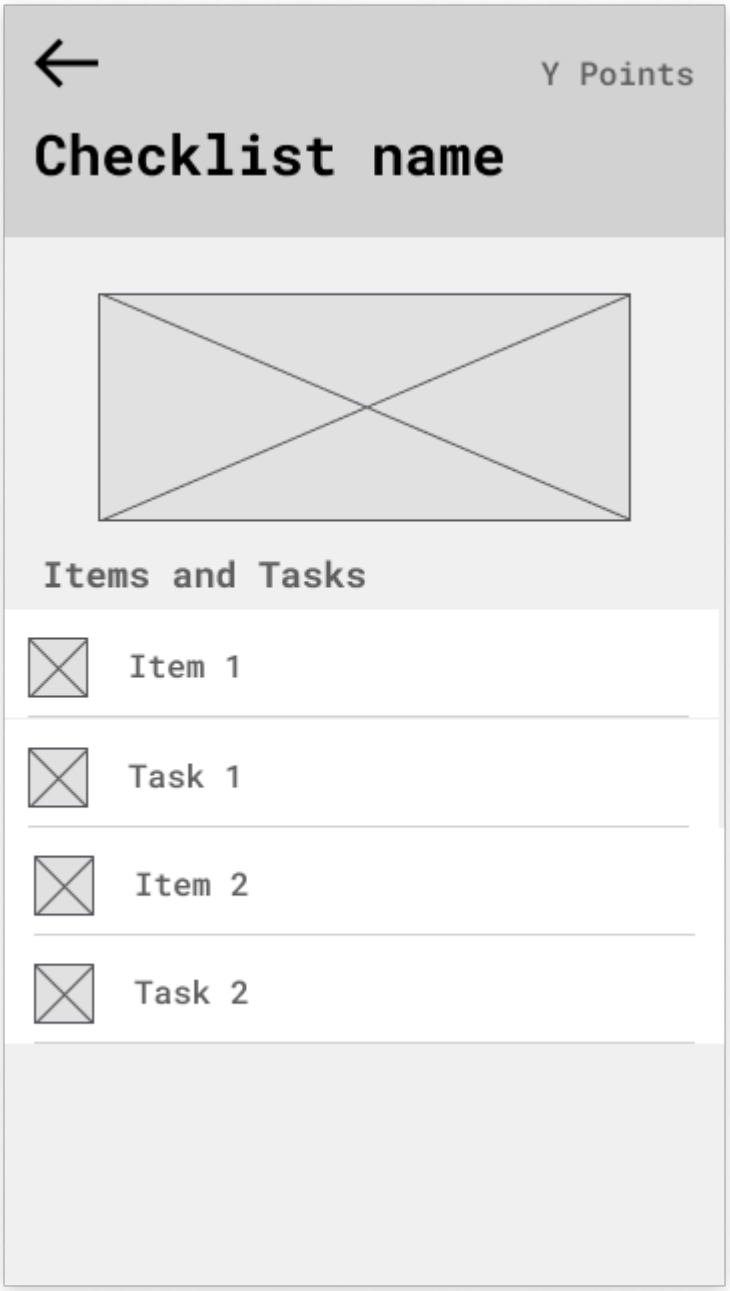
Appendix

A1.0 Wireframes V1

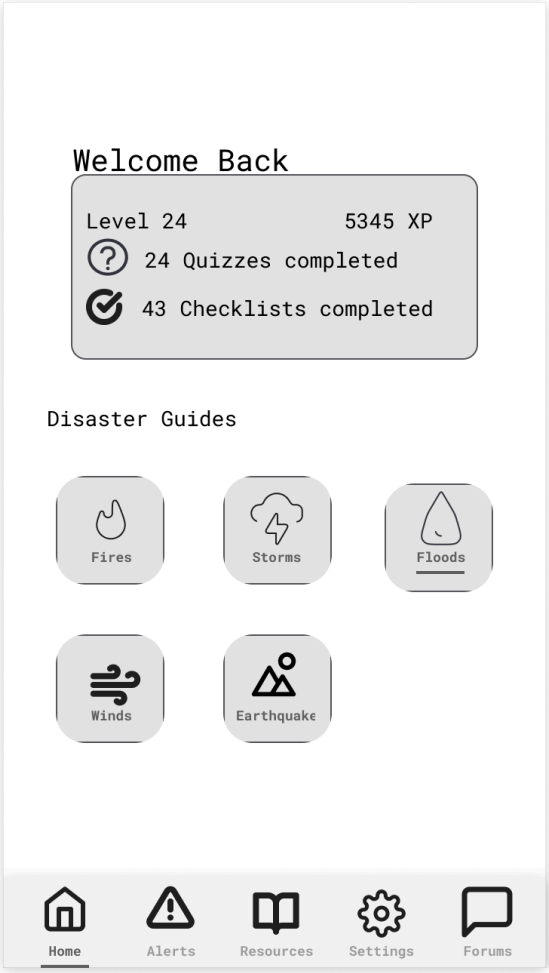
Home Screen	Alerts Screen
	

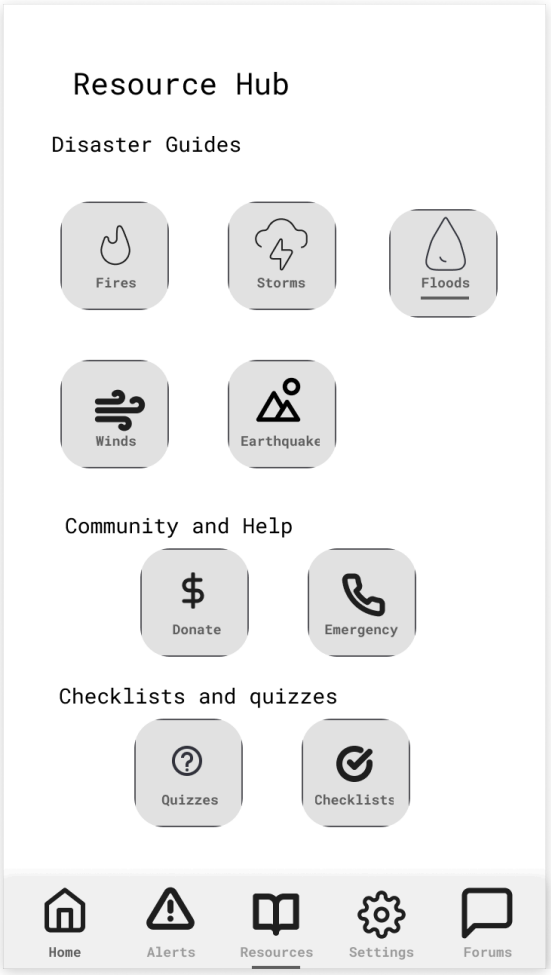
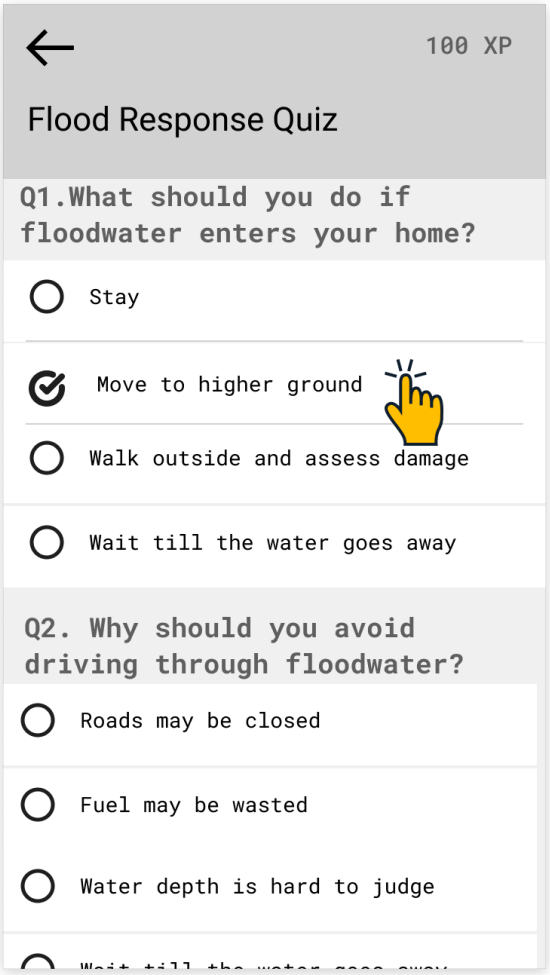
Resources Screen	Quiz Screen
	

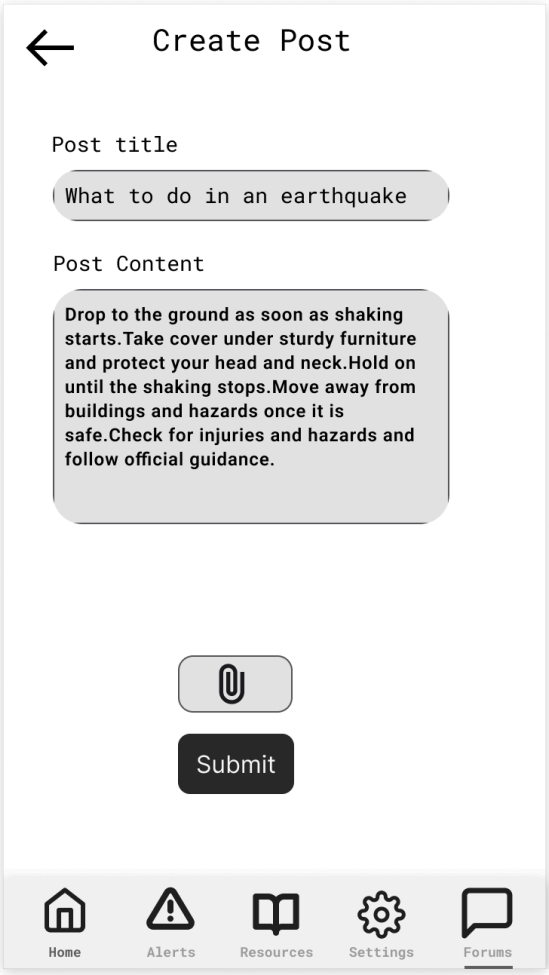
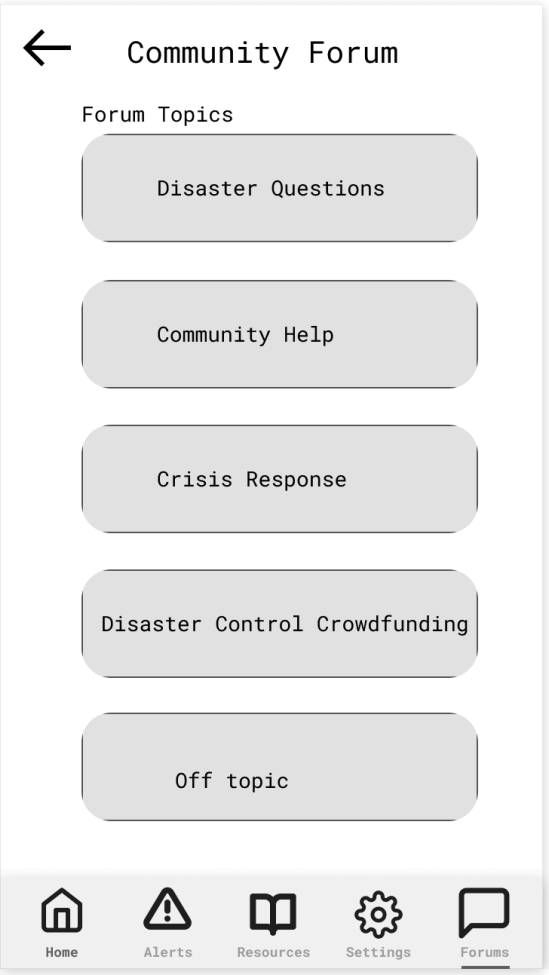
Checklist Screen



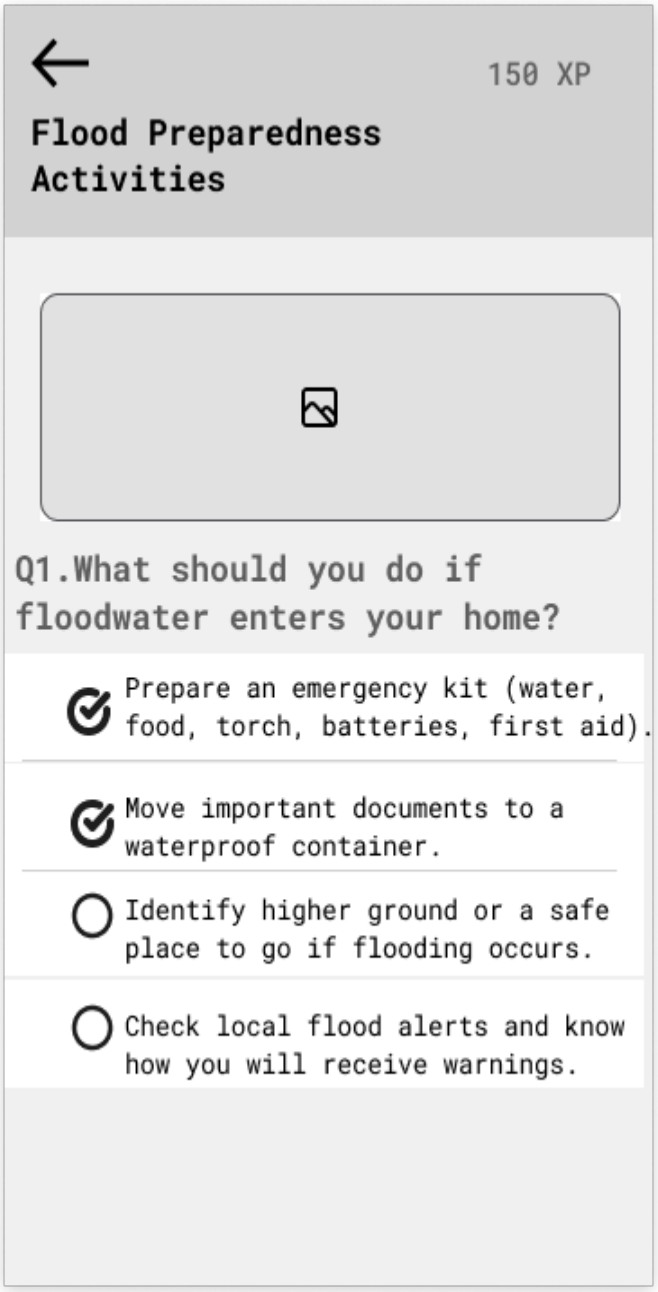
A1.1 Wireframes V2

Home Screen	Alerts Screen
	

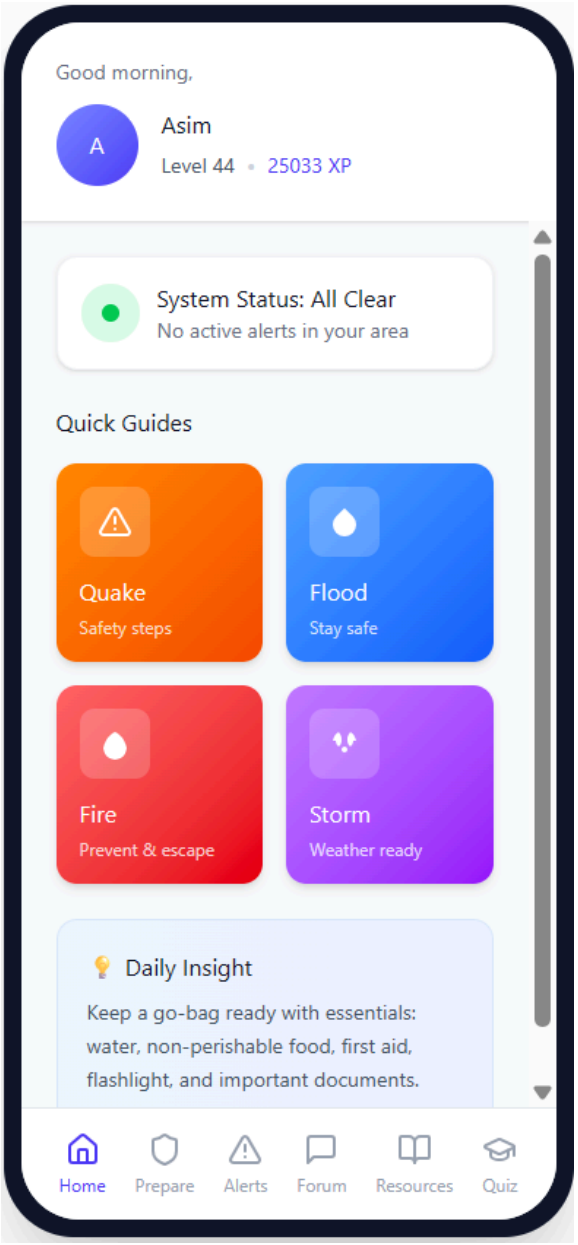
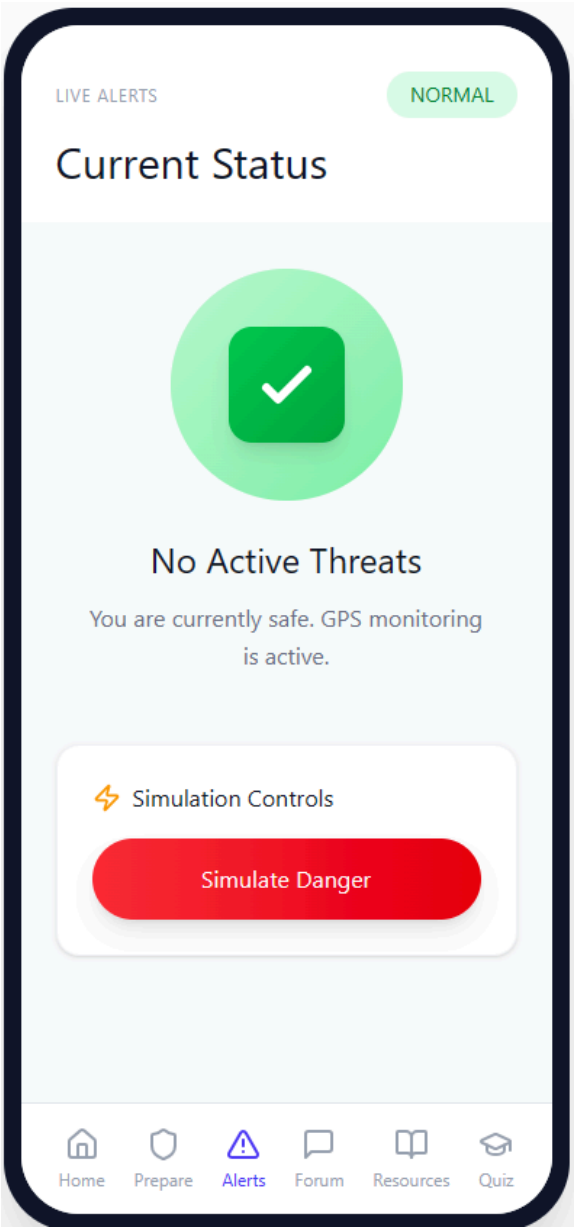
Resource Hub Screen	Quiz Screen
	

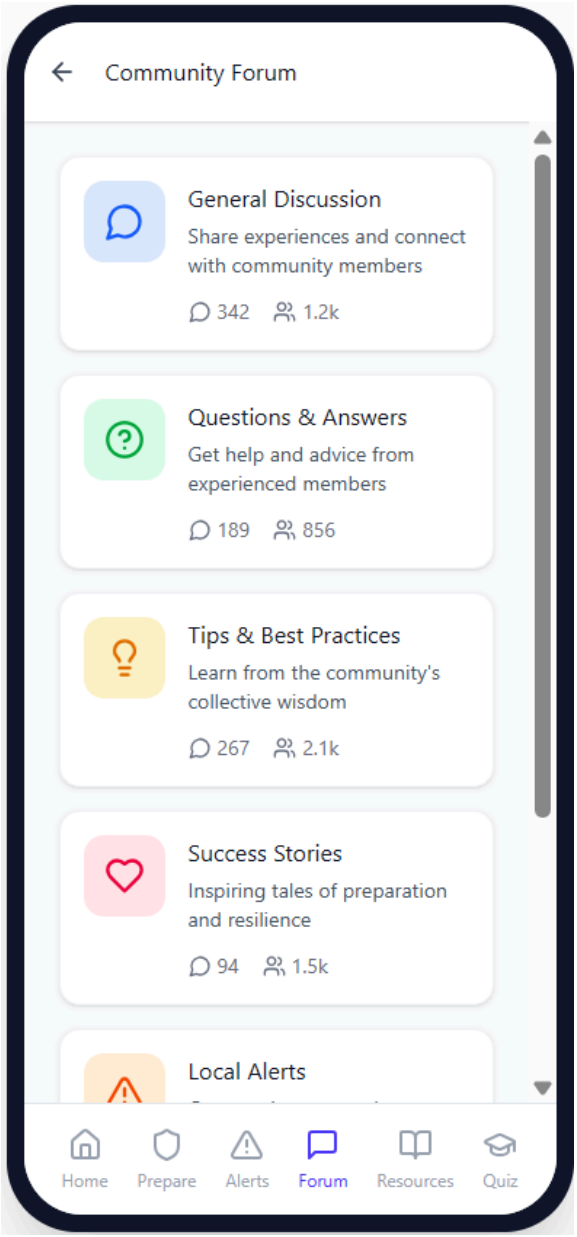
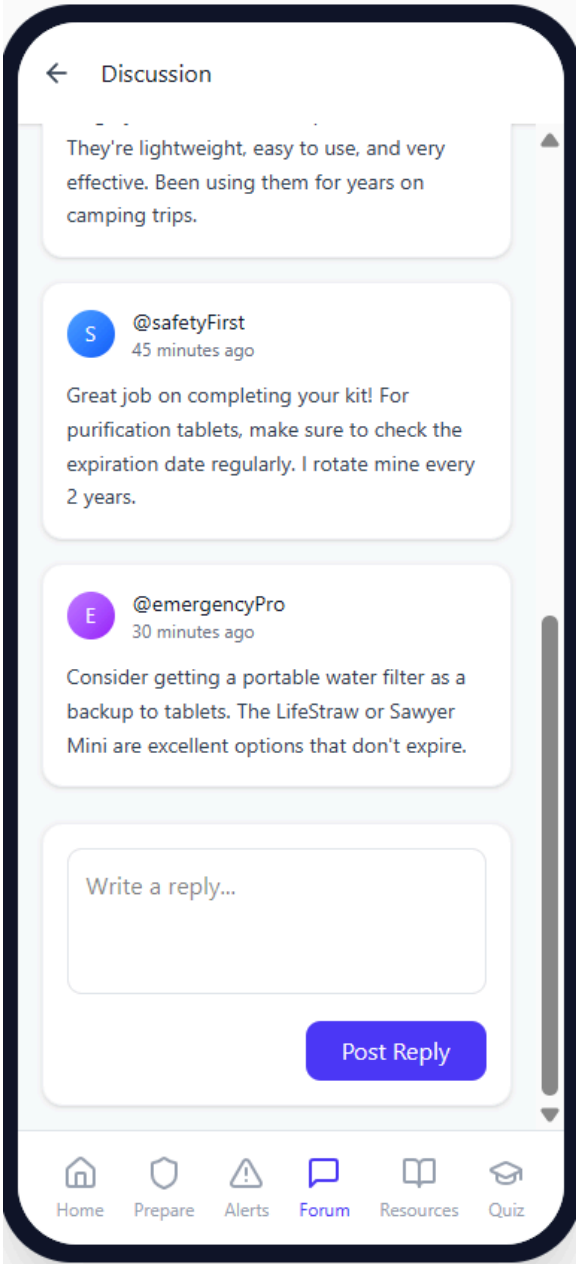
Create Post Screen	Forum Home Screen
	

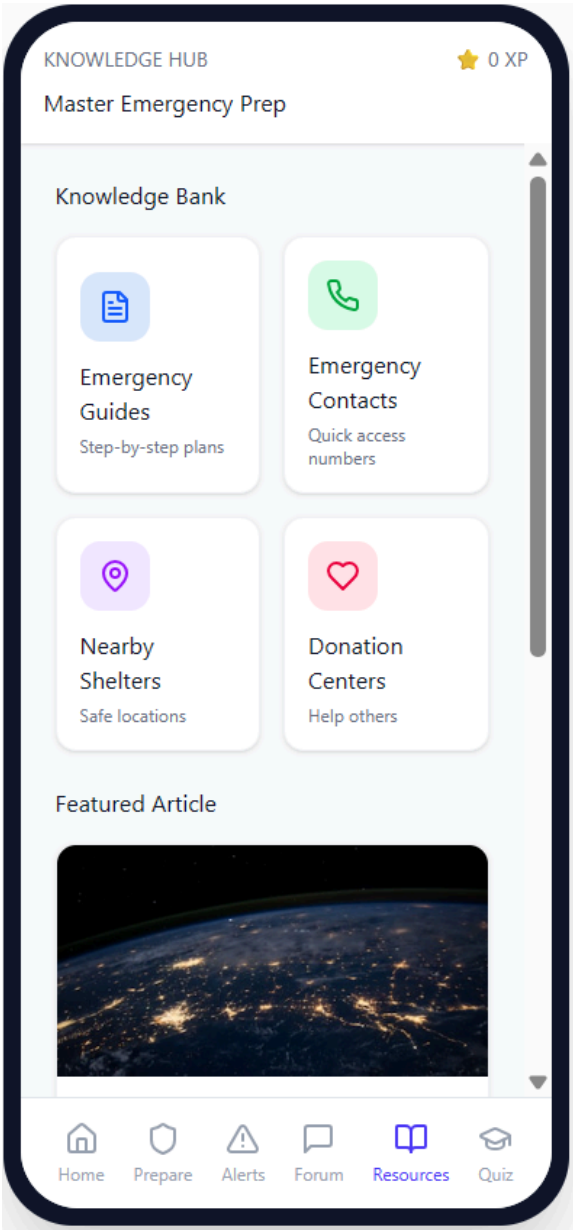
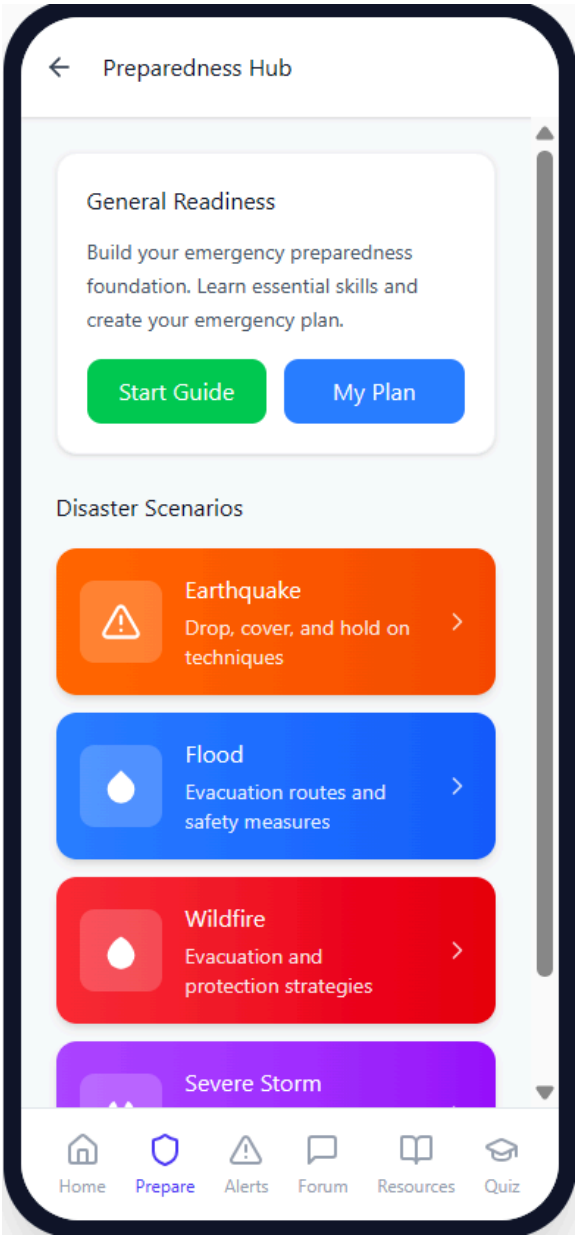
Checklist Screen

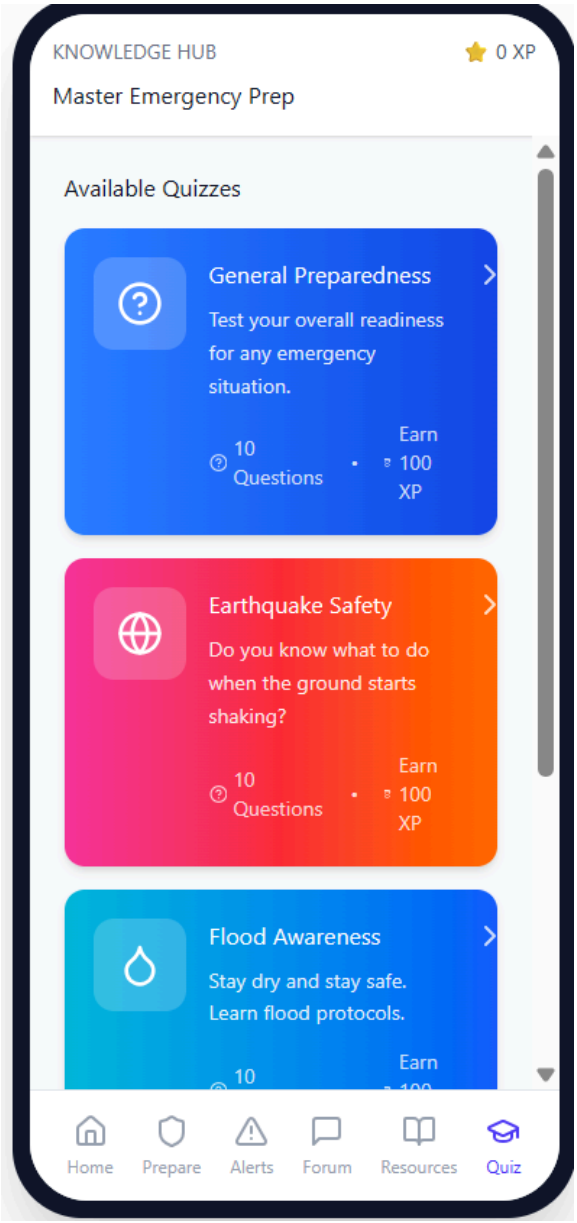
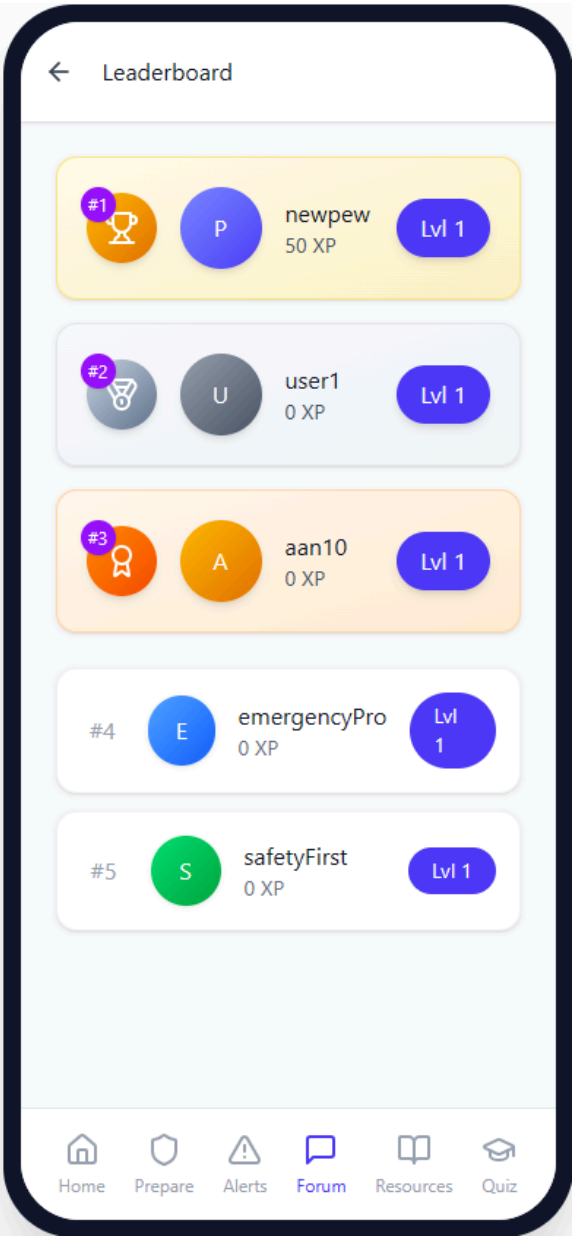


A1.2 Wireframes V3

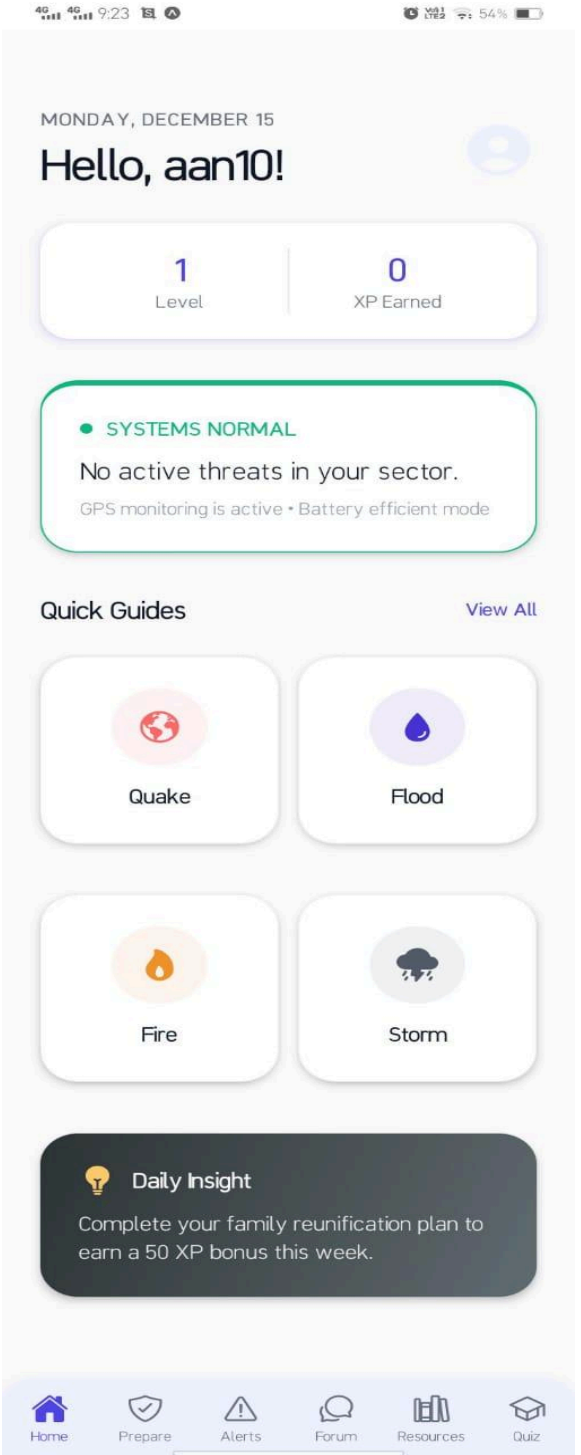
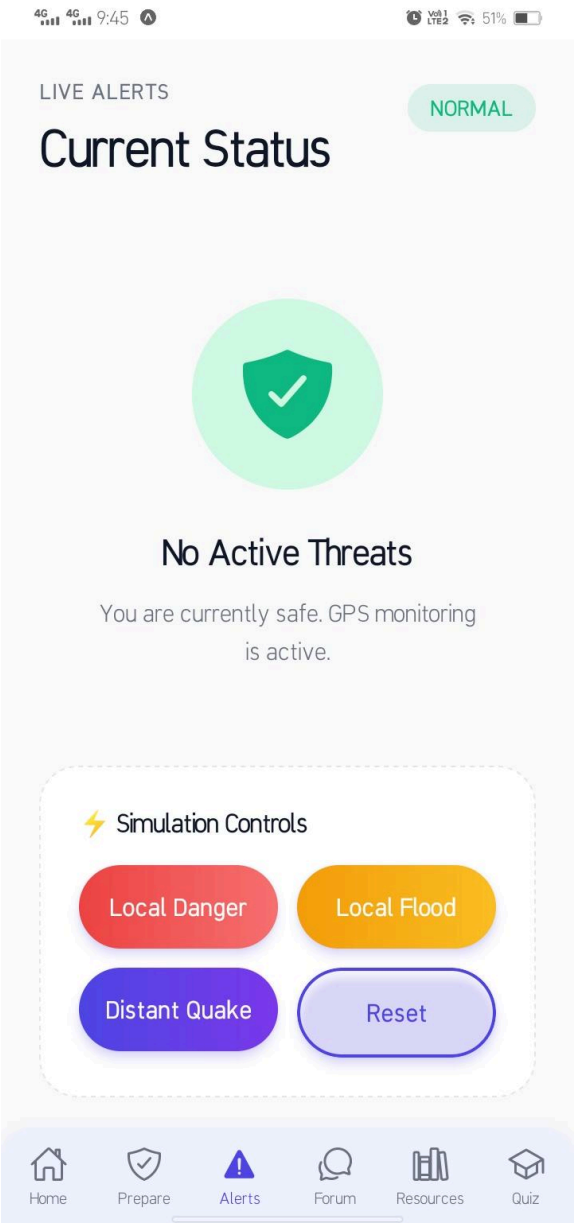
Home Screen	Alerts Screen
	

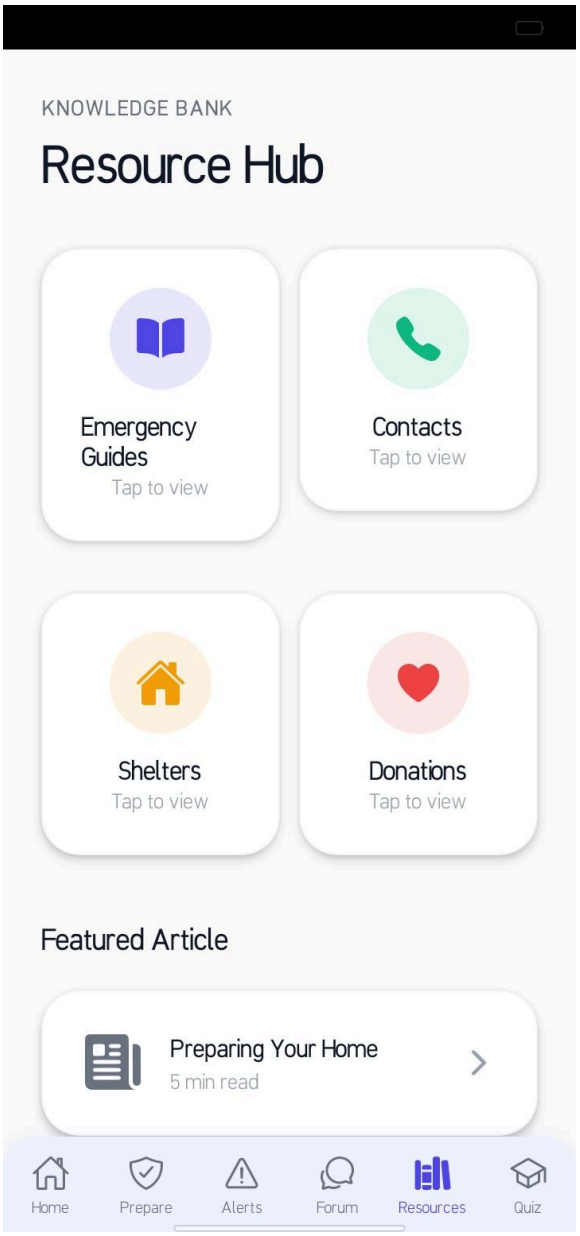
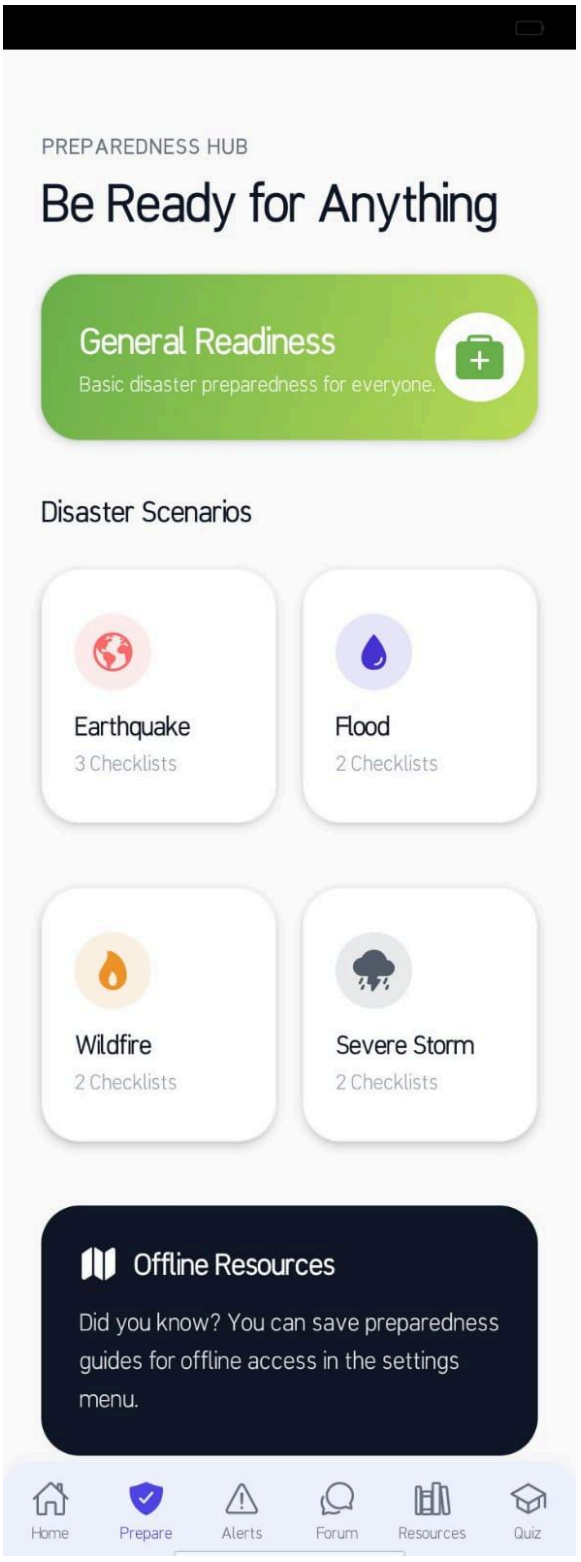
Forum Home Screen	Forum Discussion Screen
	

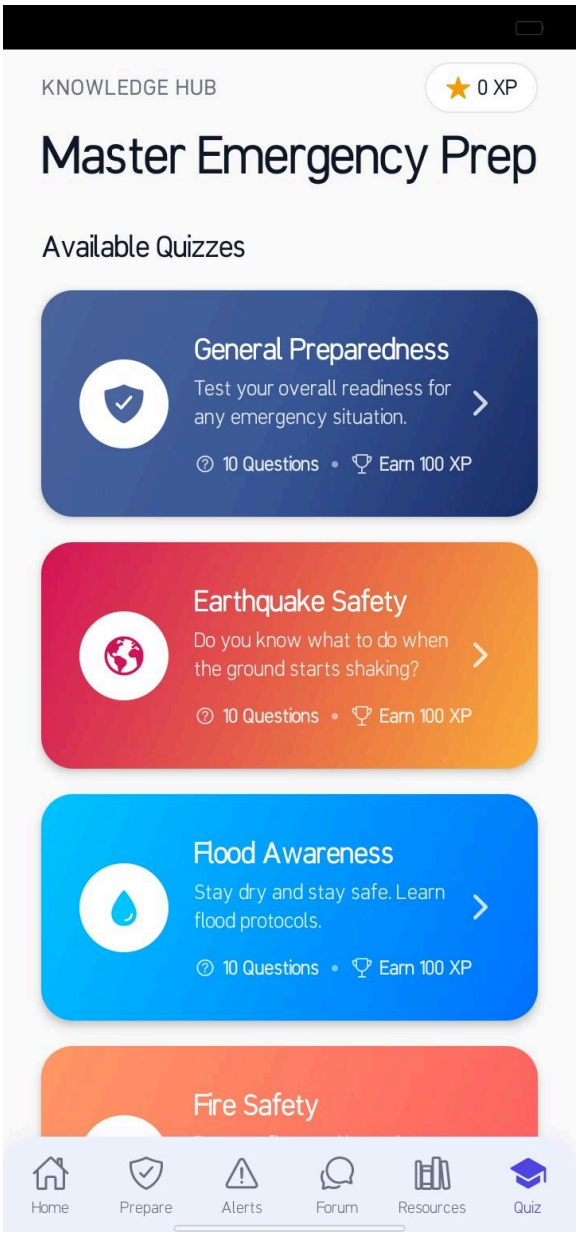
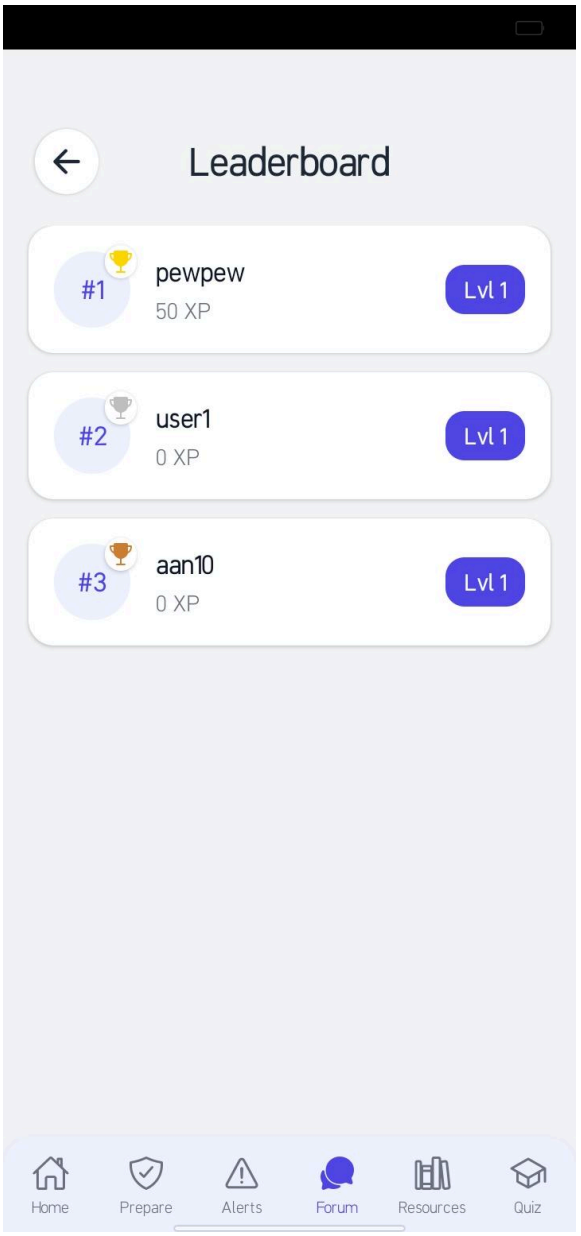
Resource Hub Screen	Prepare Screen
	

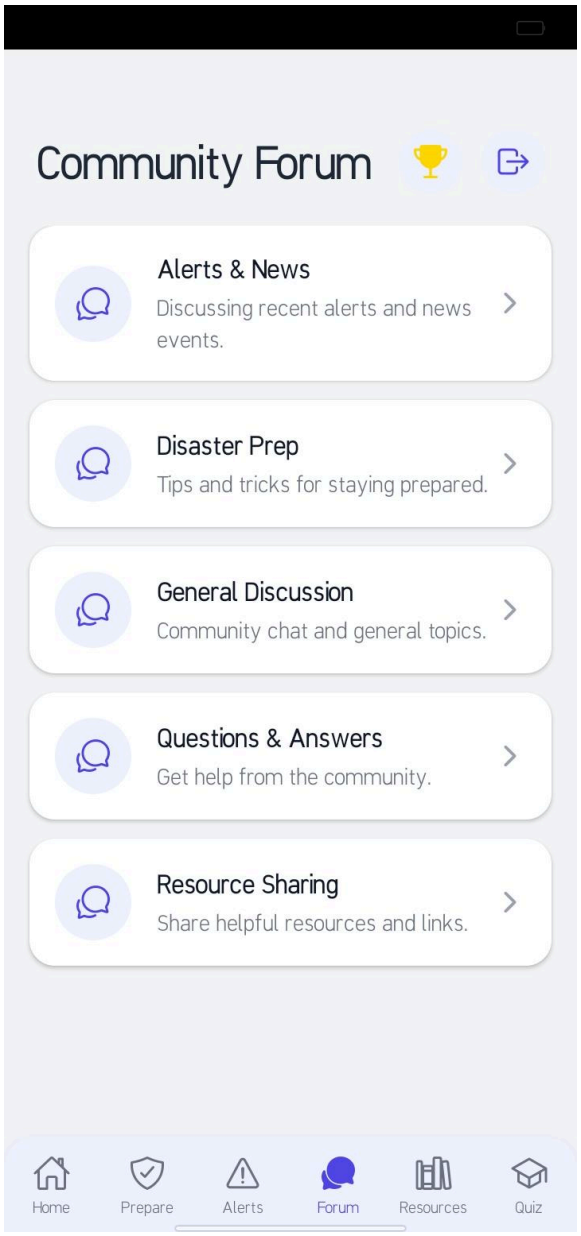
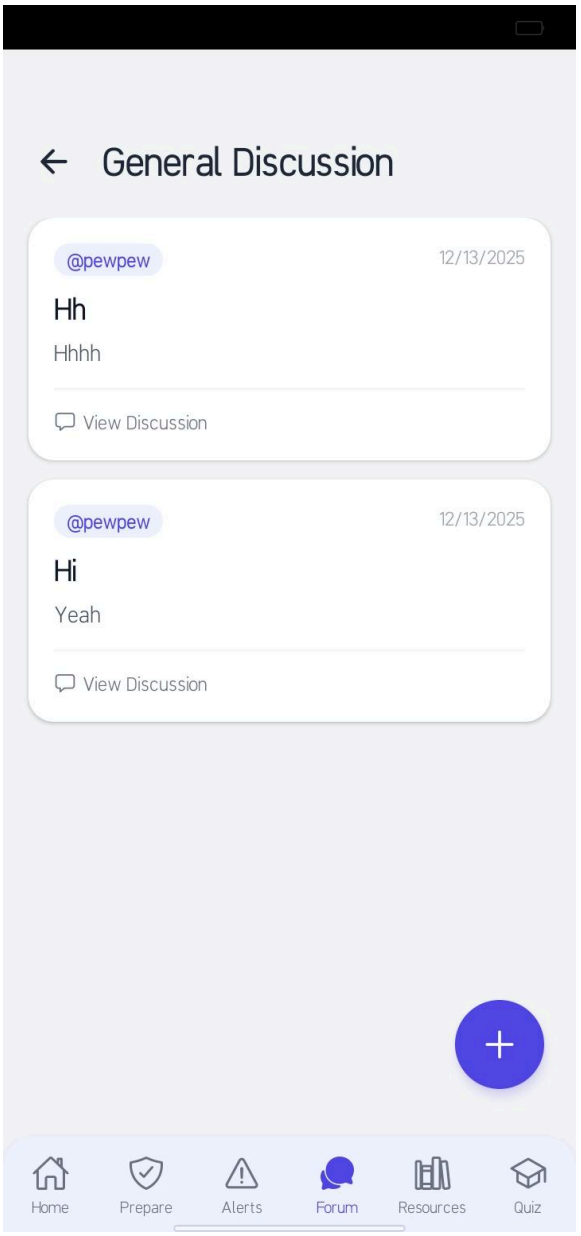
Quiz Hub Screen	Leaderboard Screen
	

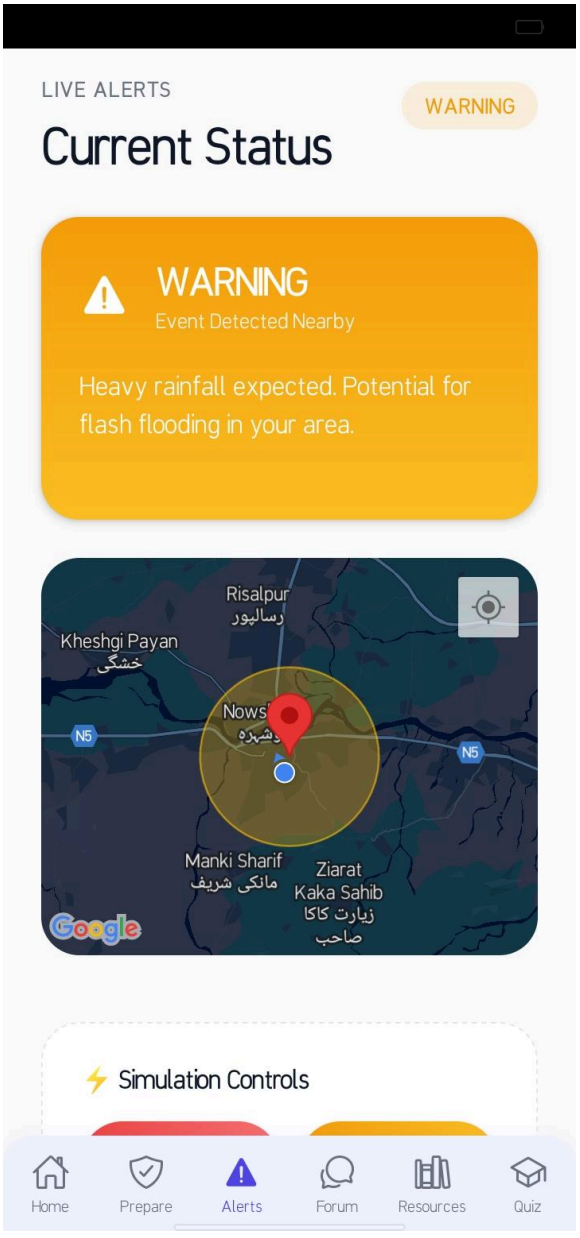
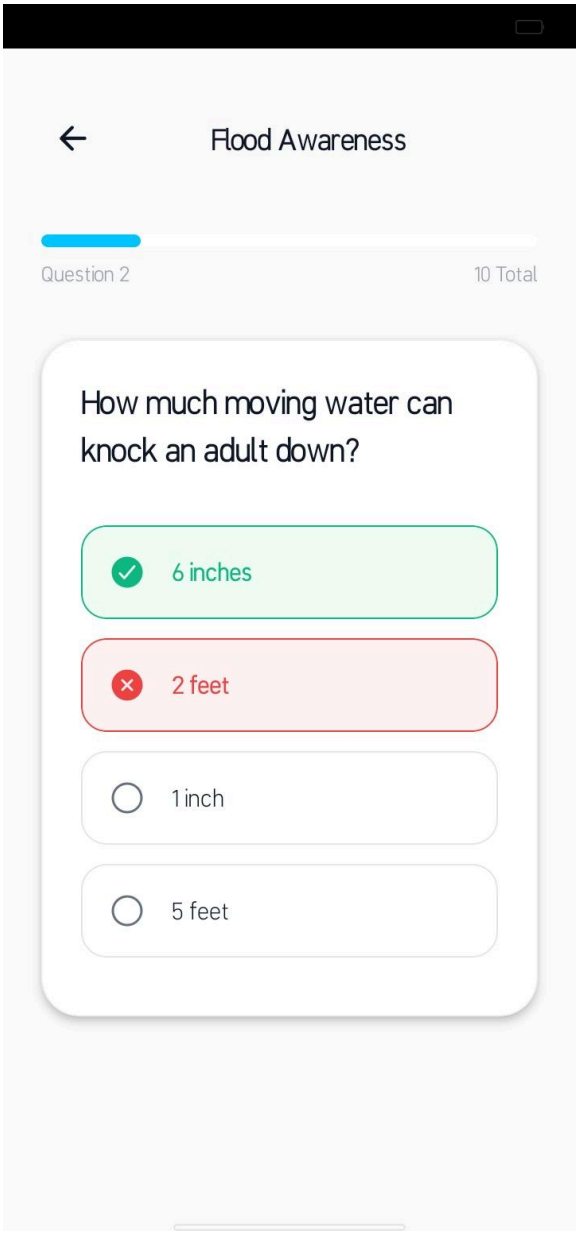
A1.3 Prototype Screens

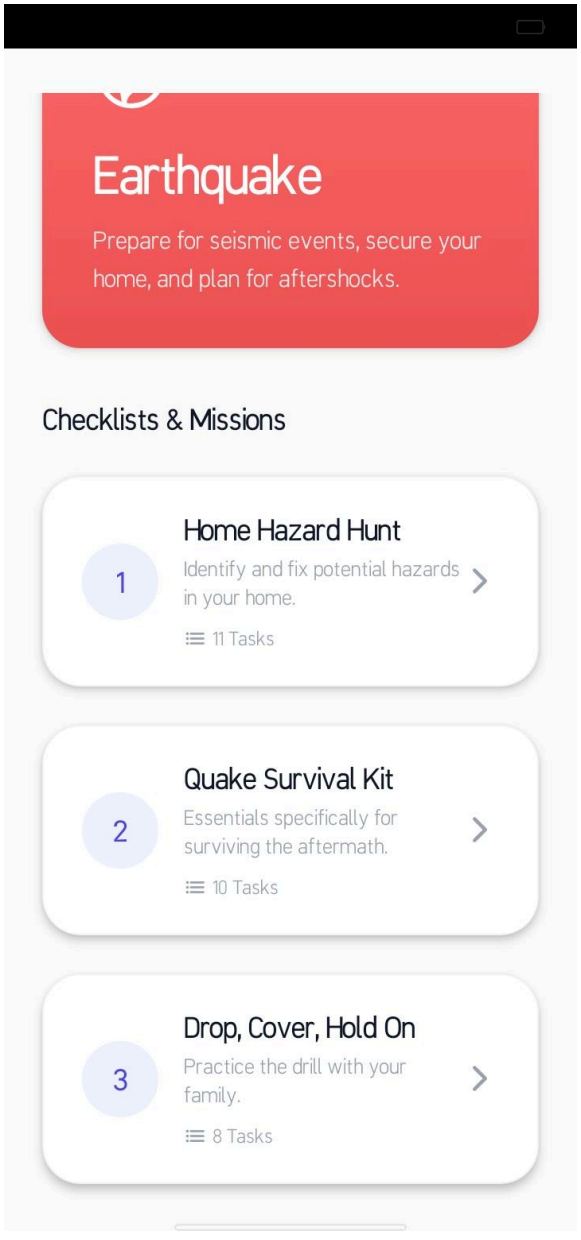
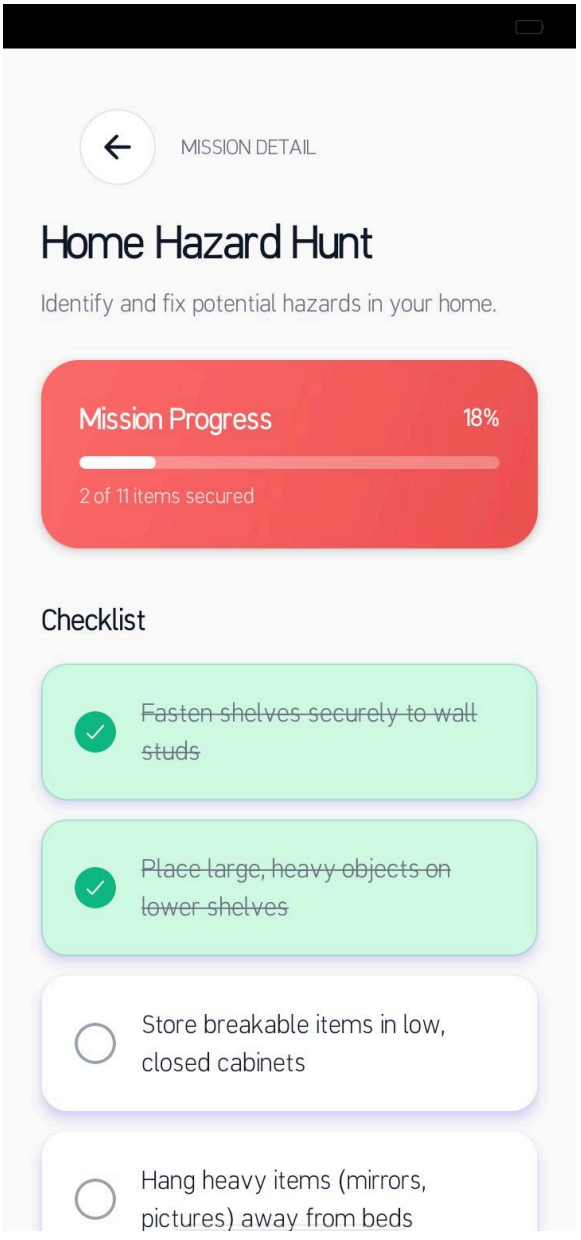
Home Screen	Alerts Screen
	

Resource Hub Screen	Prepare Screen
	

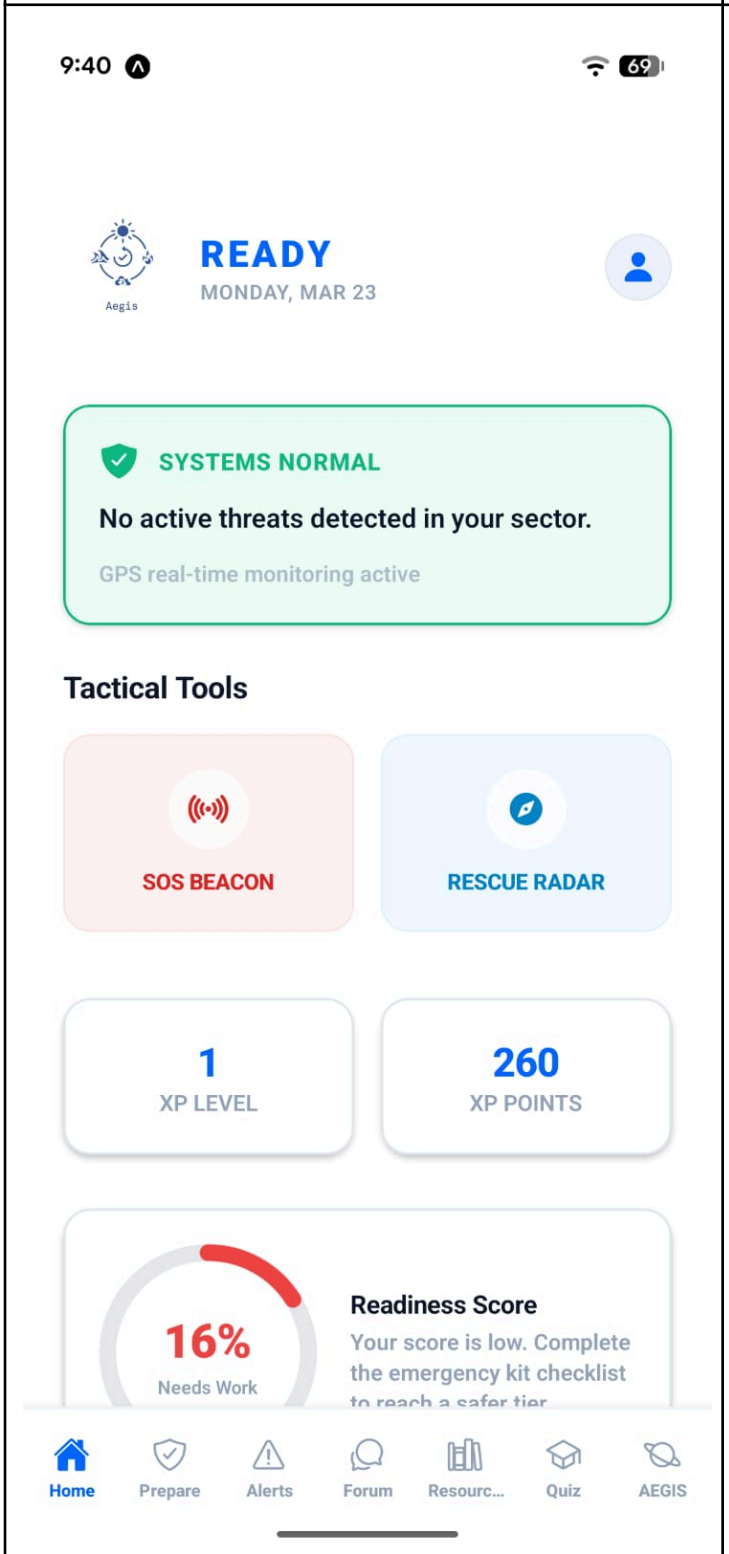
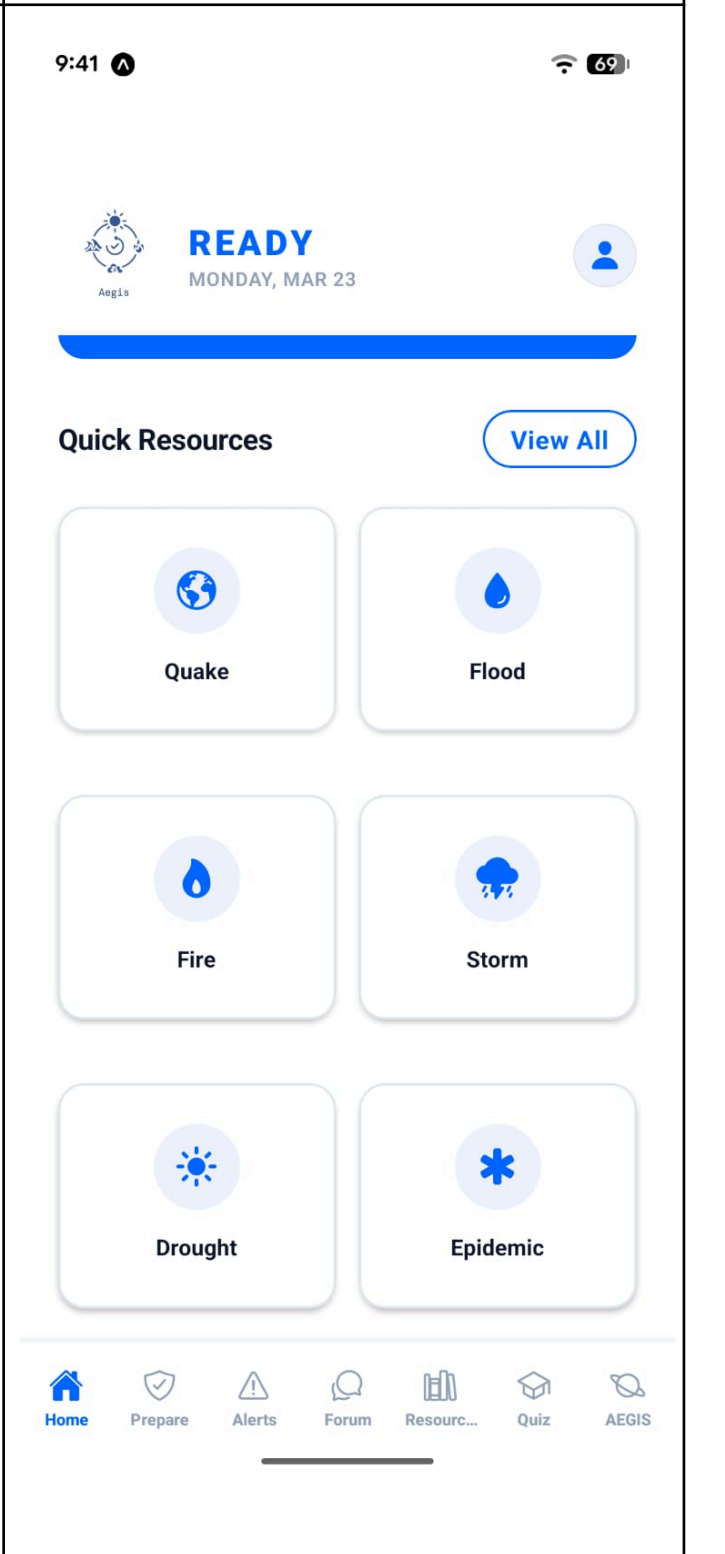
Quiz Hub Screen	Leaderboard Screen
	

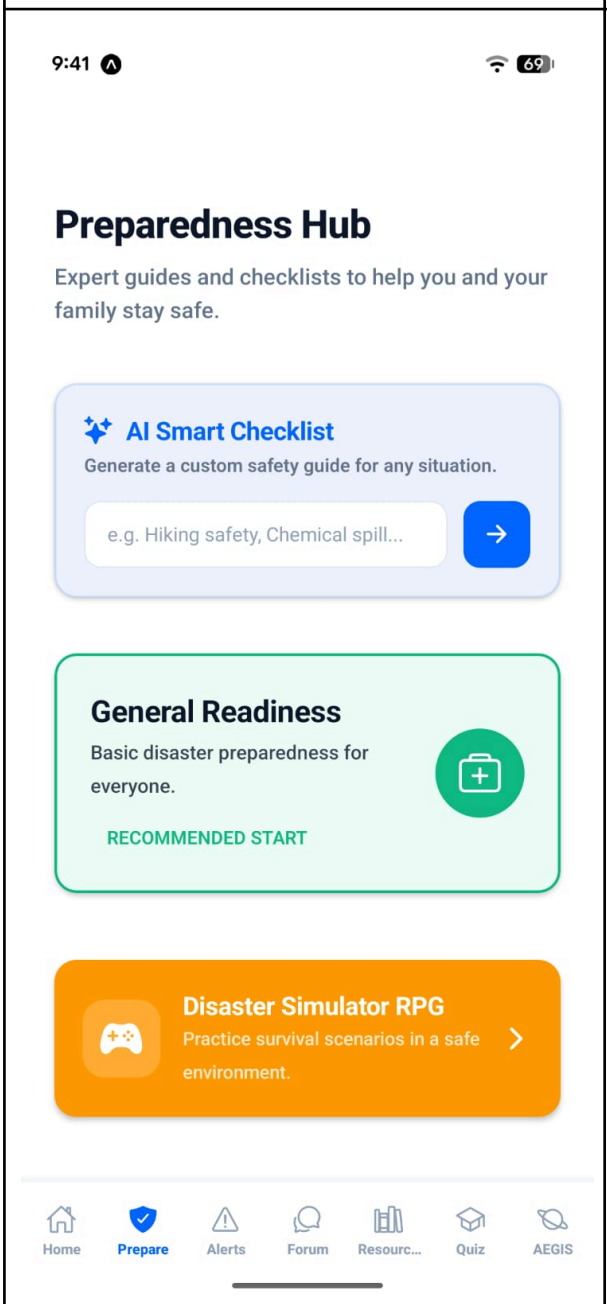
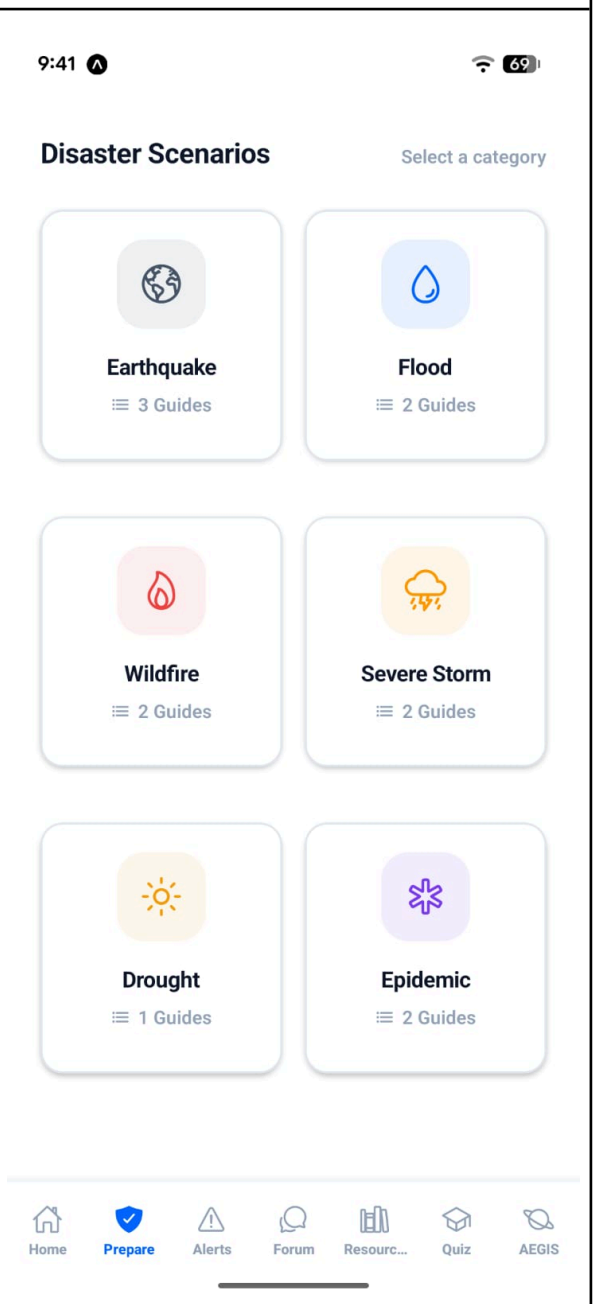
Forum Home Screen	Forum Category Posts Screen
	

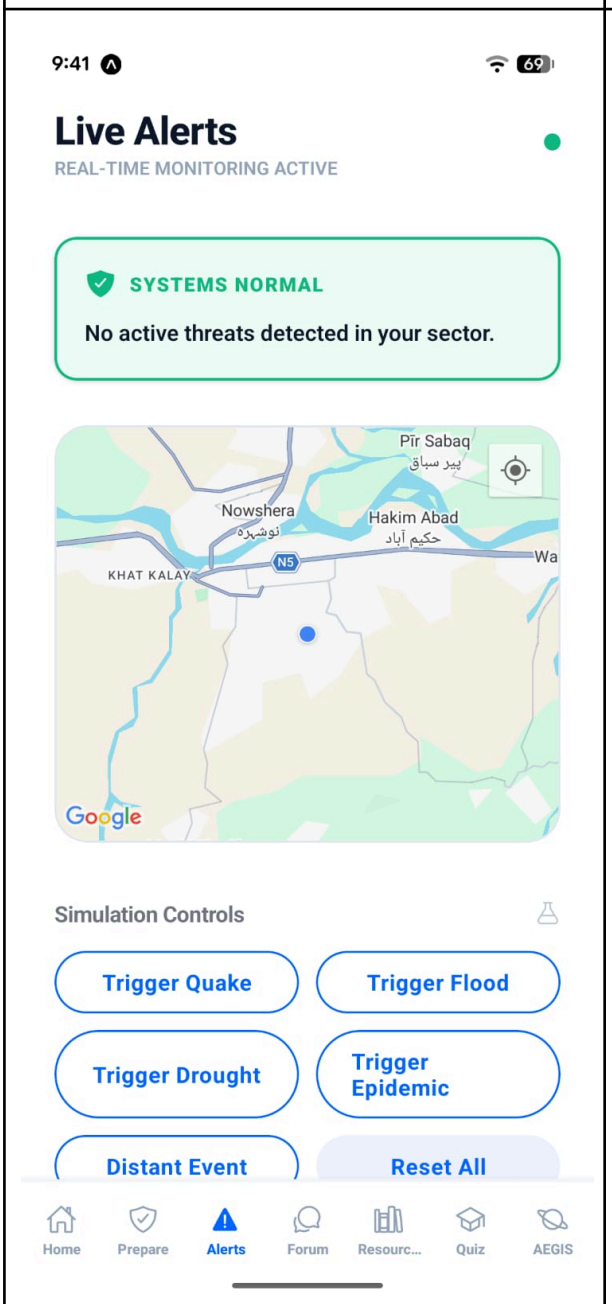
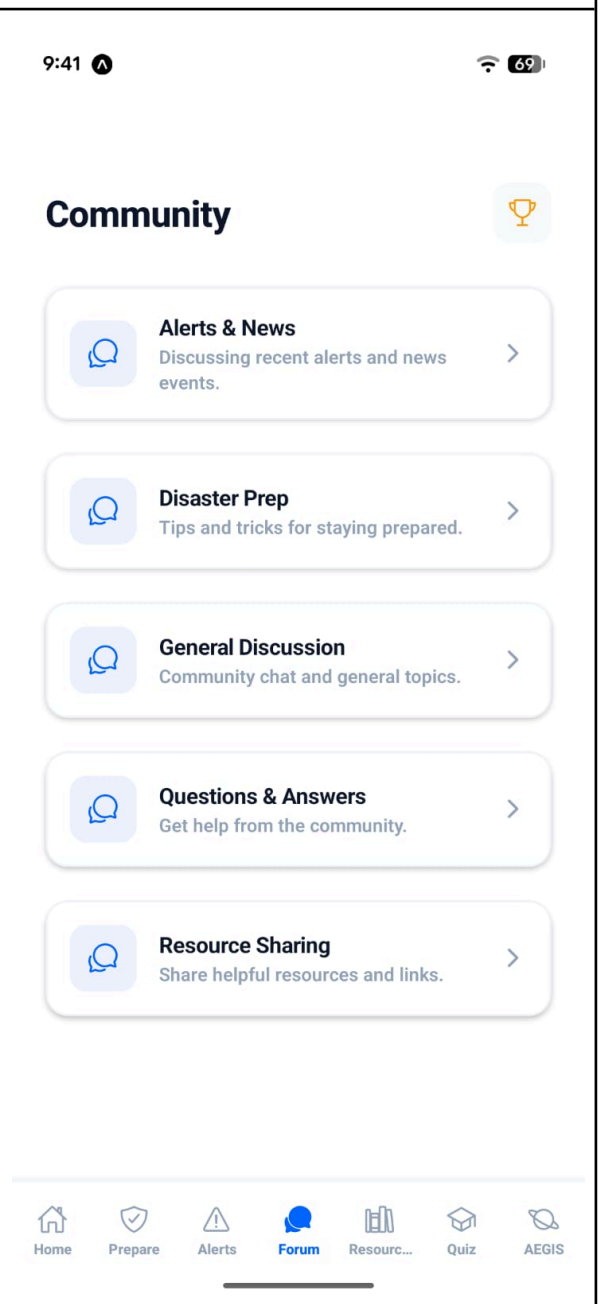
Alerts Screen (Warning Mode with location detected)	Quiz Screen (Wrong answer selected)
	

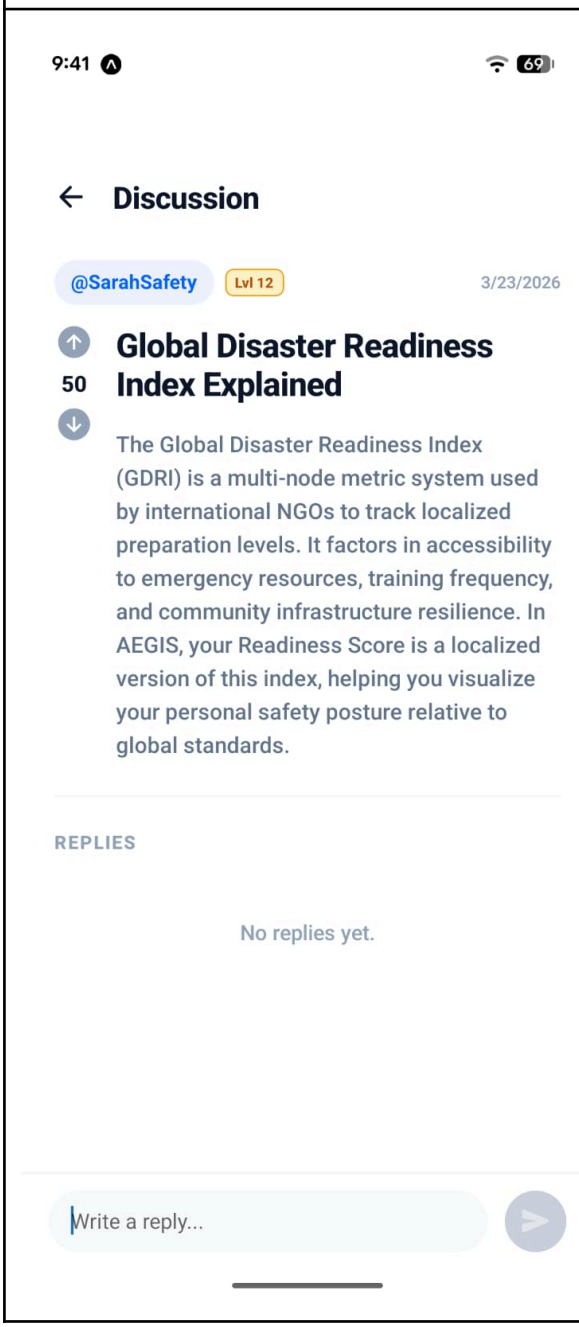
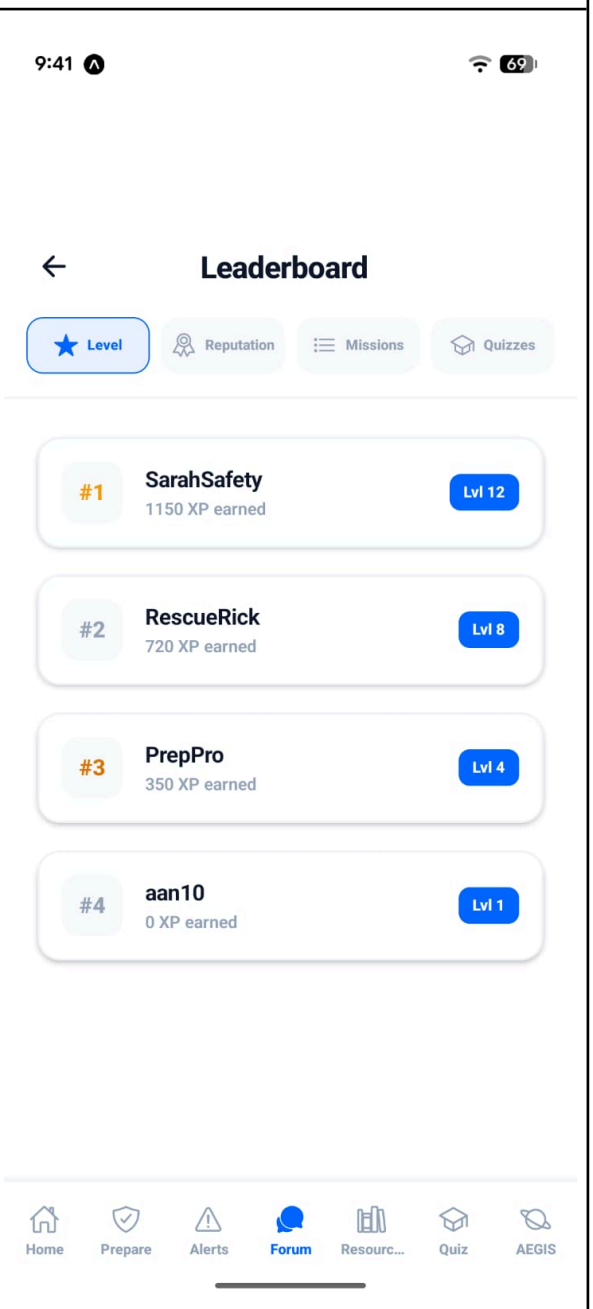
Screen showing checklists for a disaster category	Checklist screen
	

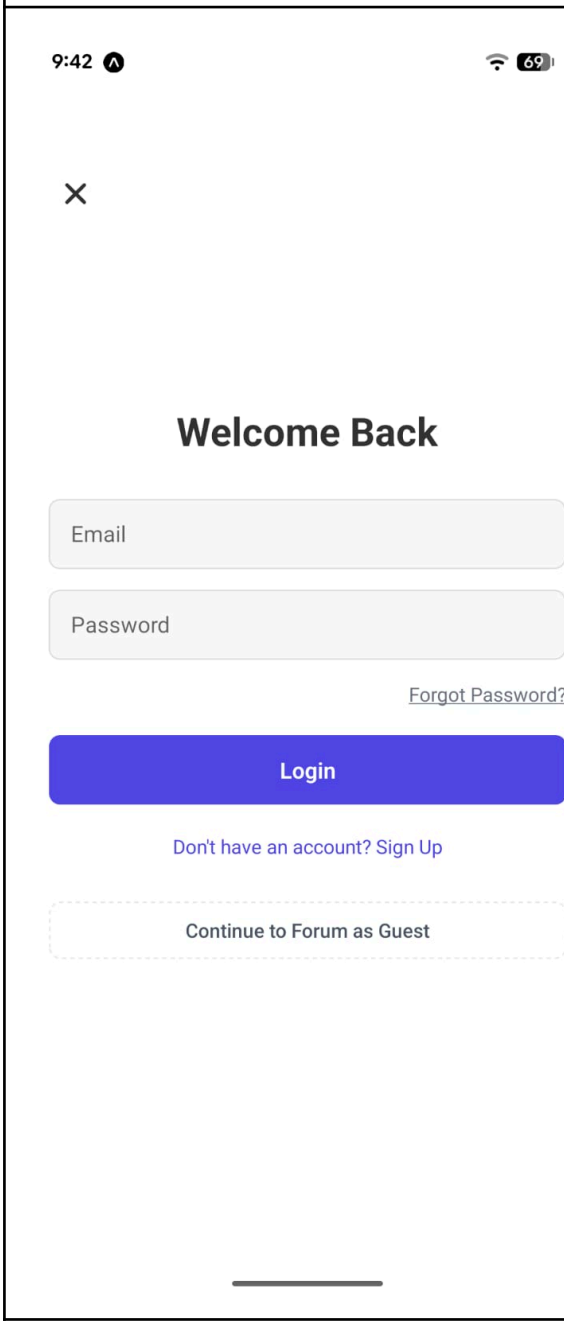
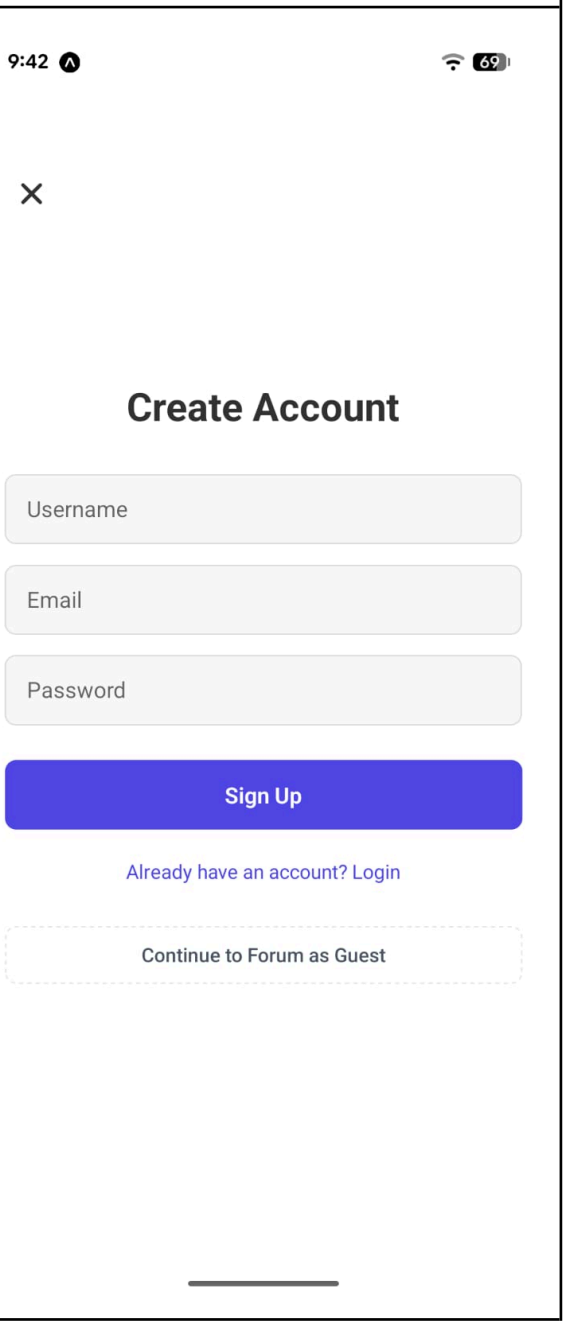
A1.4 Final Screens

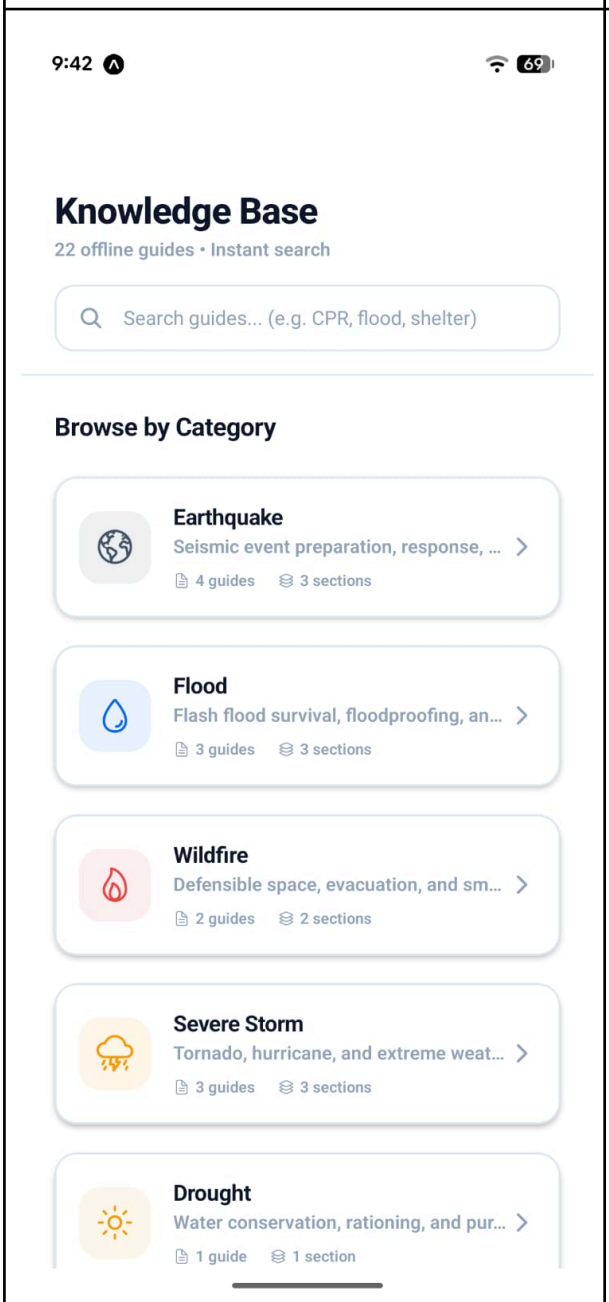
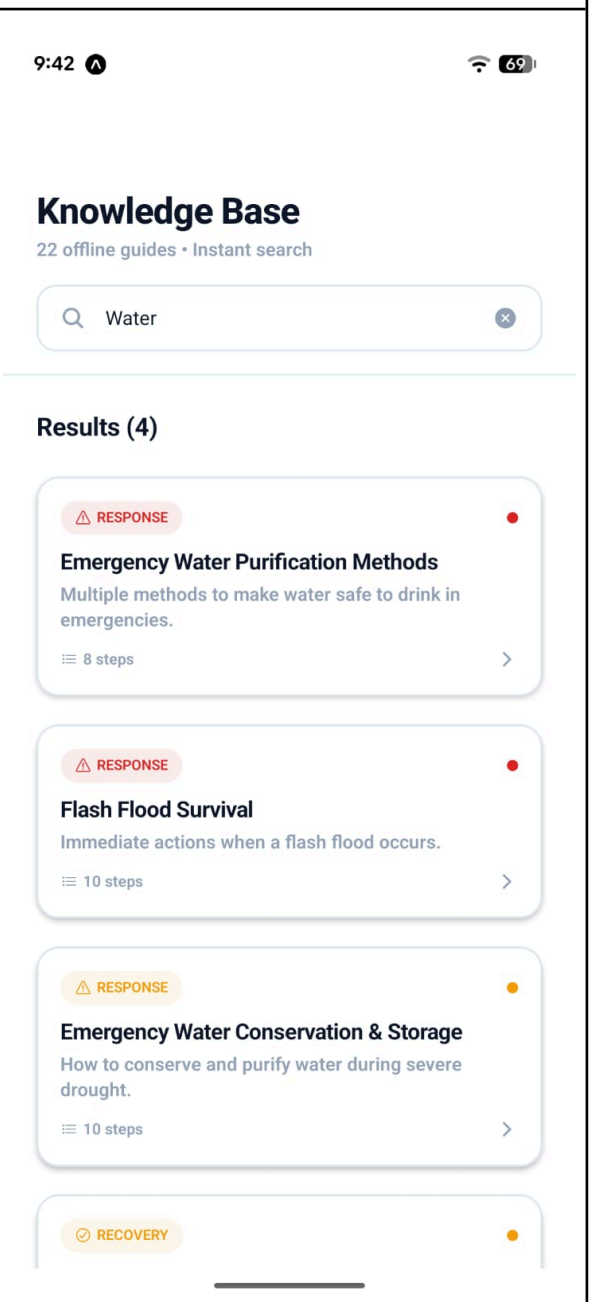
Home Screen	Home Screen Continued
	

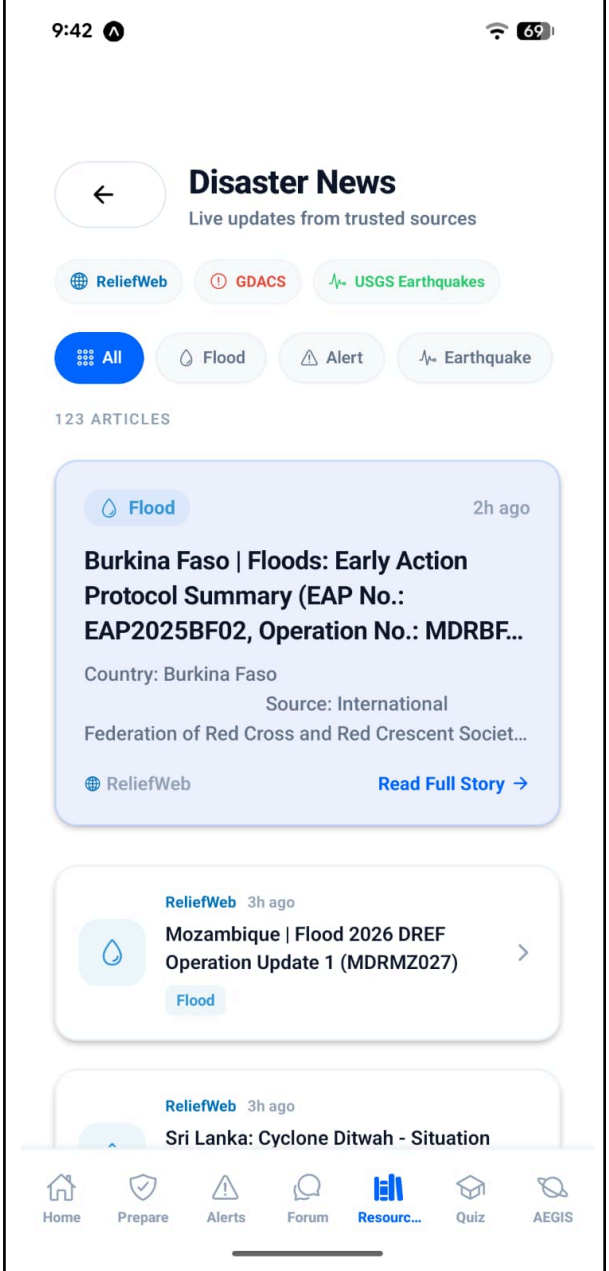
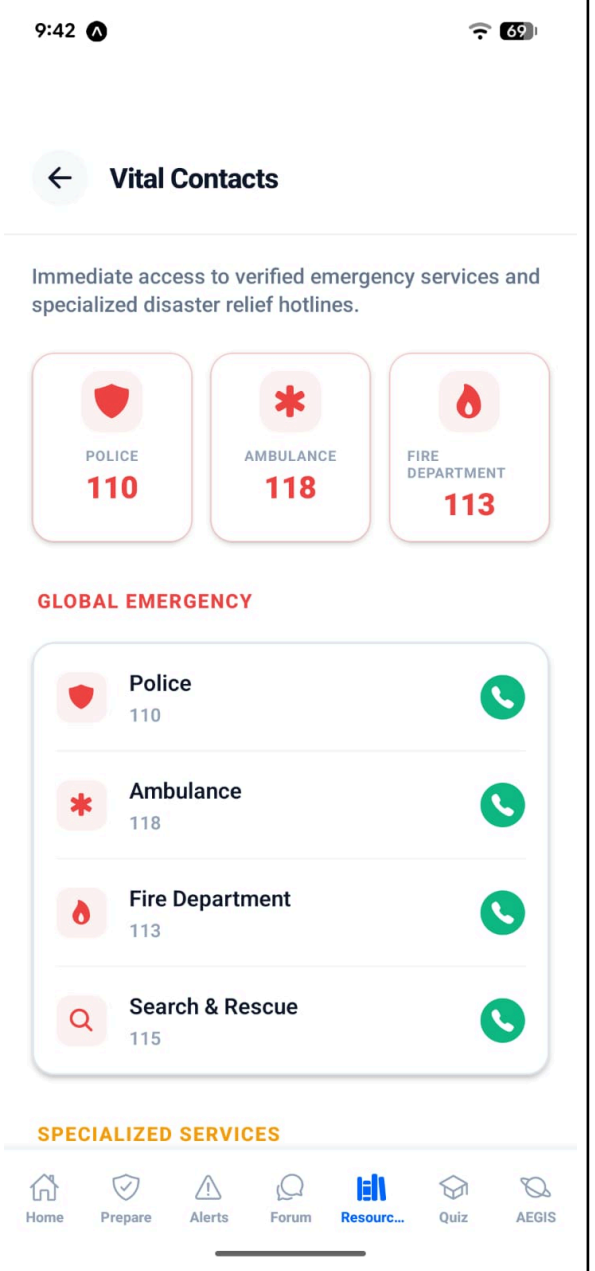
Preparedness Hub	Preparedness Hub Continued
	

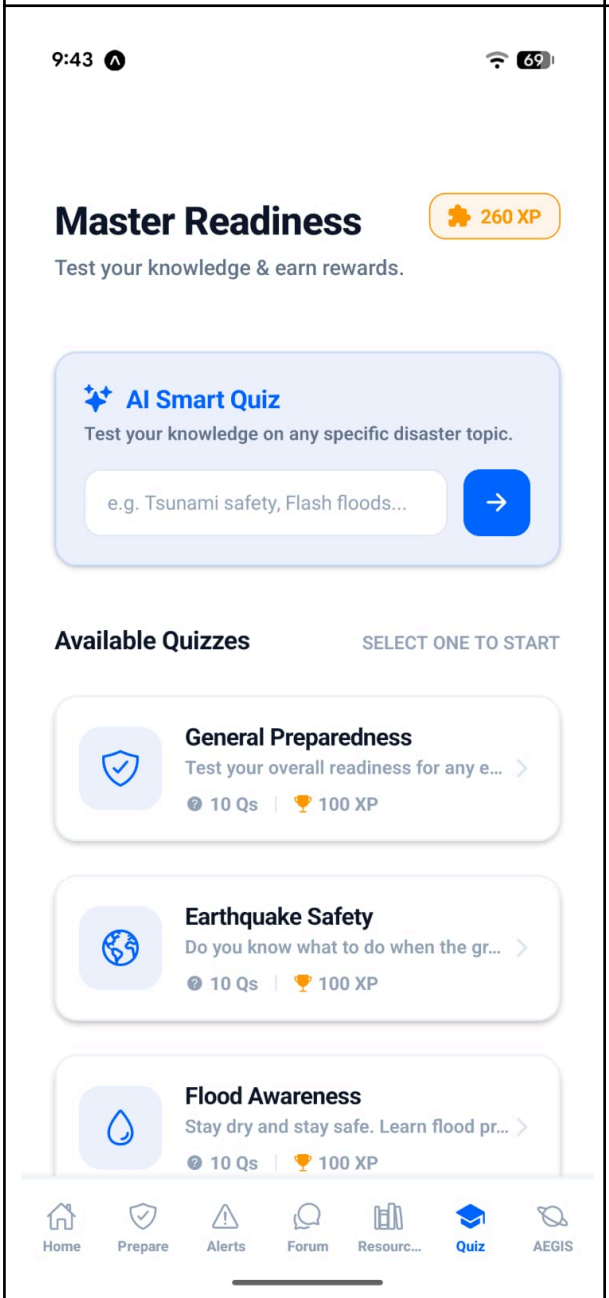
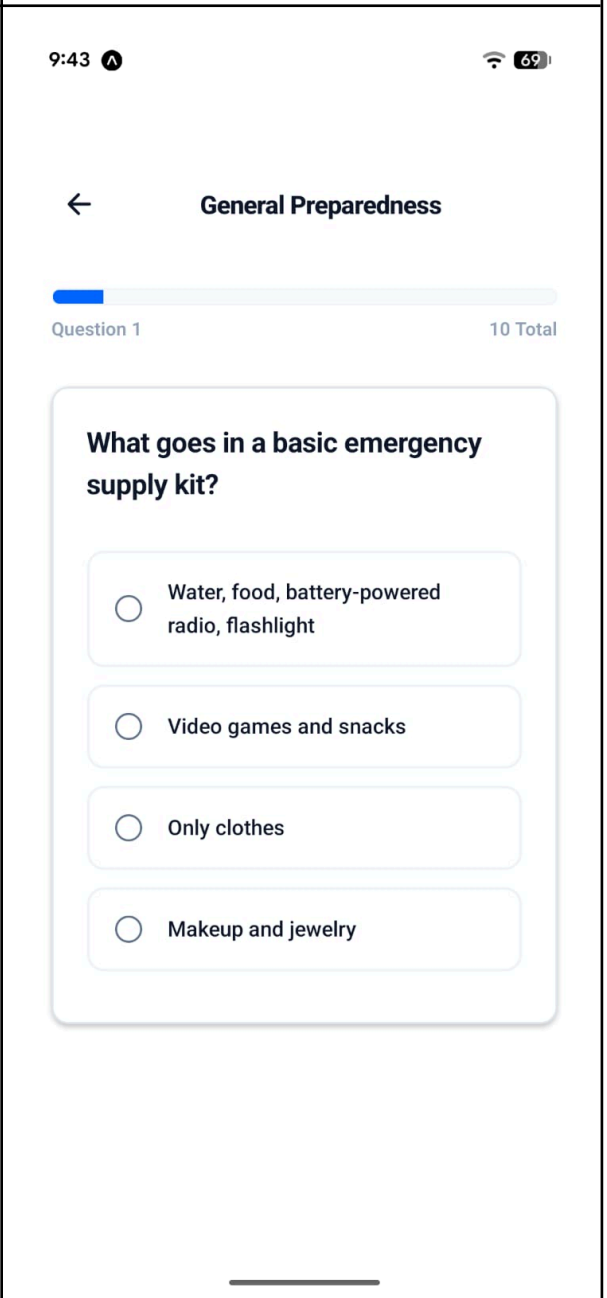
Alerts Screen	Forum Home
	

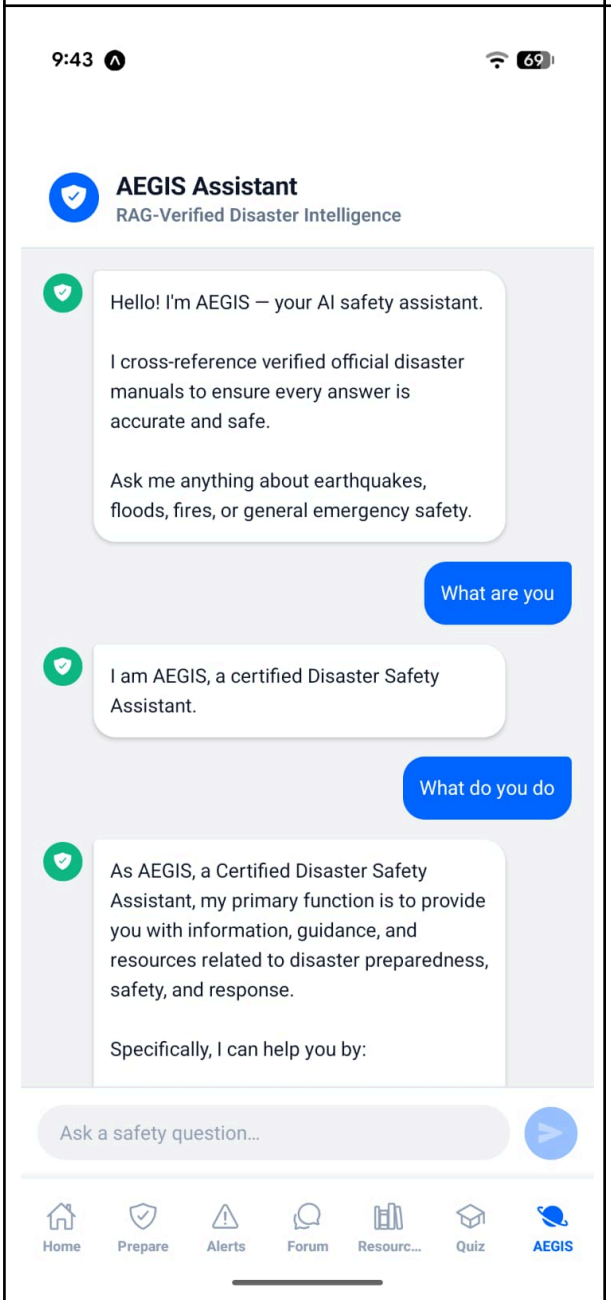
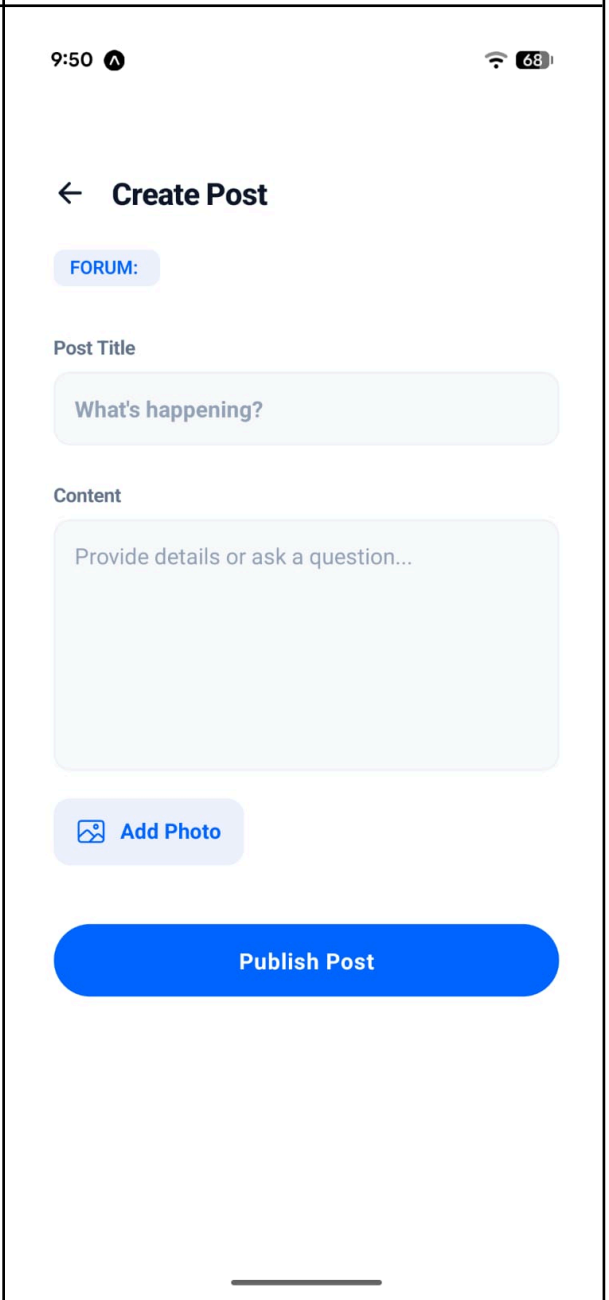
Forum Post	Leaderboard
	

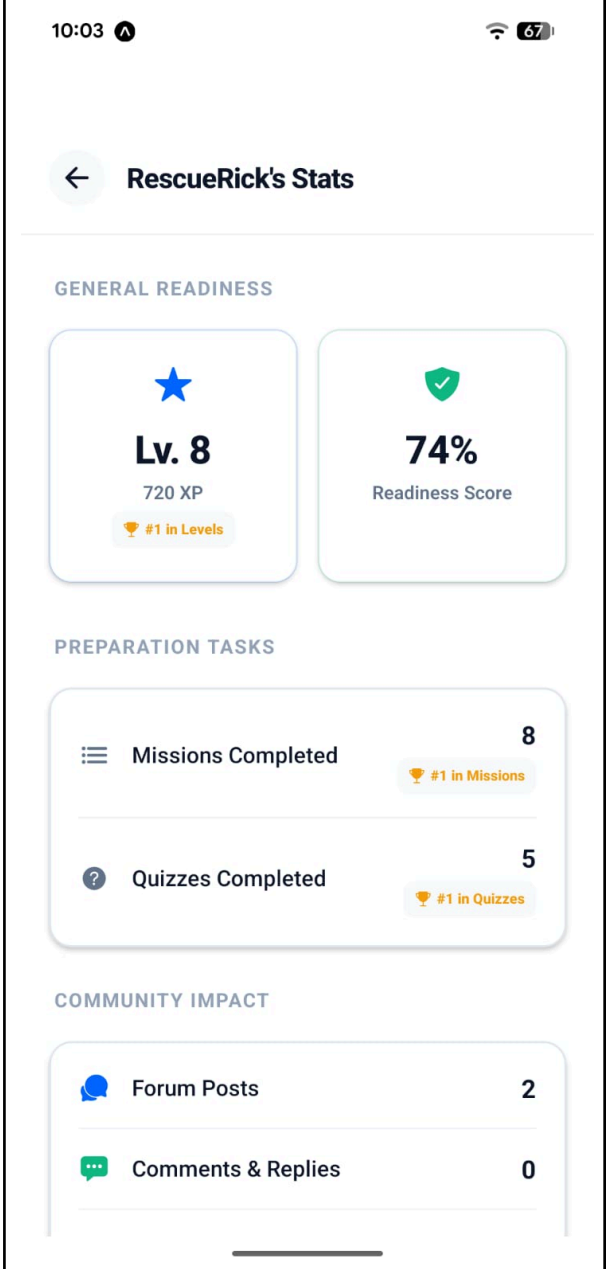
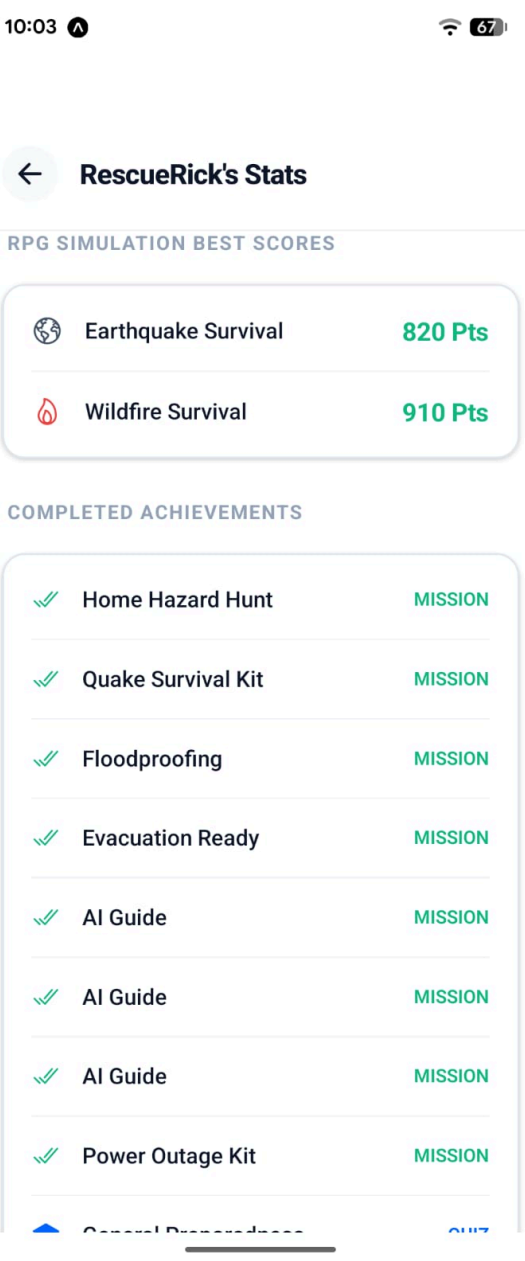
Sign in	Sign up
	

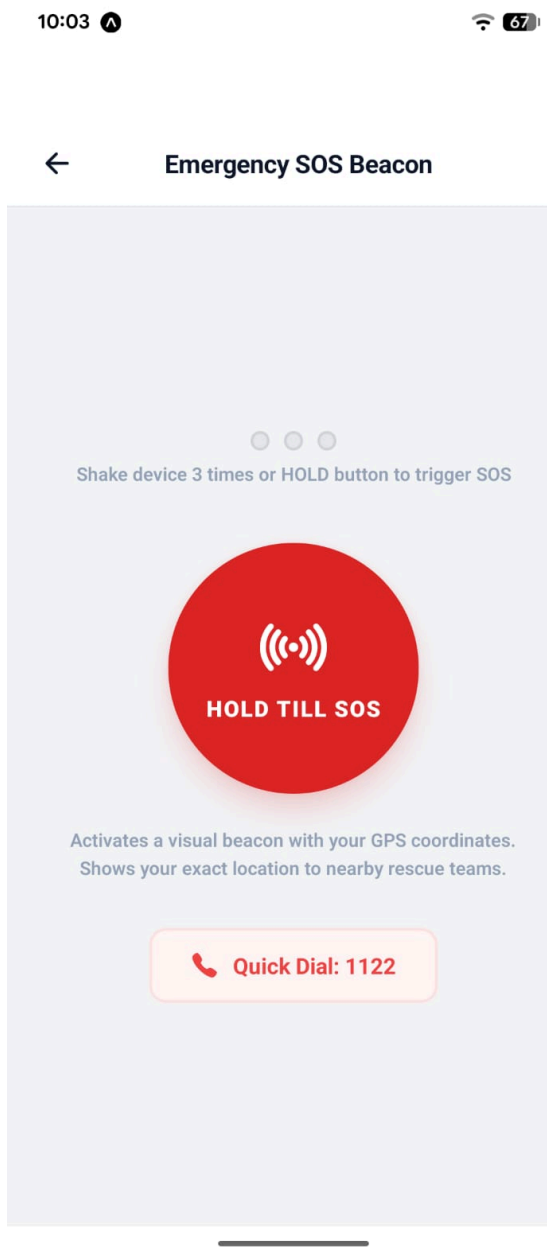
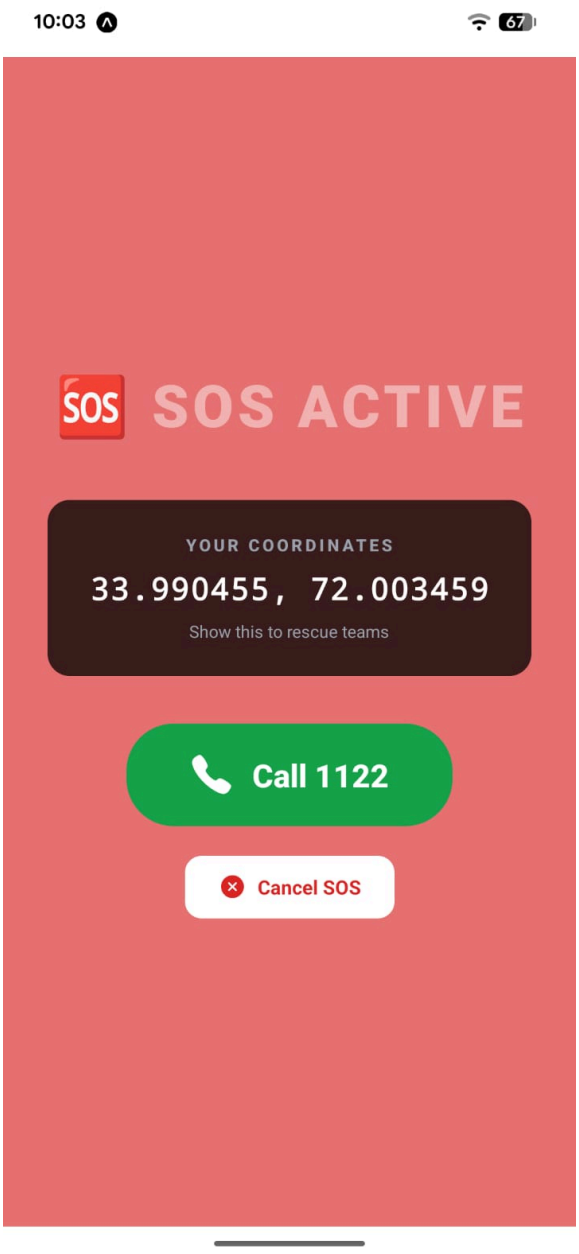
Trie knowledge base	Trie knowledge base searching
	

RSS Feed	Contacts
	

Quiz Screen	Quiz active
	

RAG Chat	Forum while creating post
	

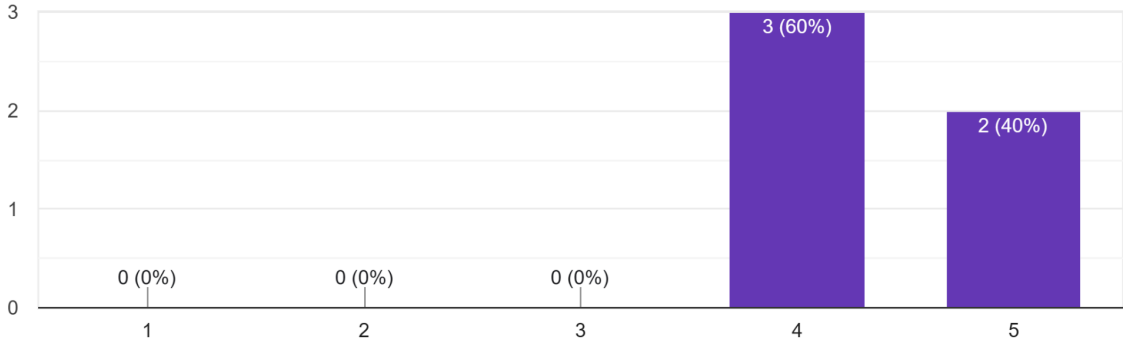
Stats	Stats Continued
	

Emergency SOS coordinates	SOS coordinates active
	

A1.5 Usability Testing Results

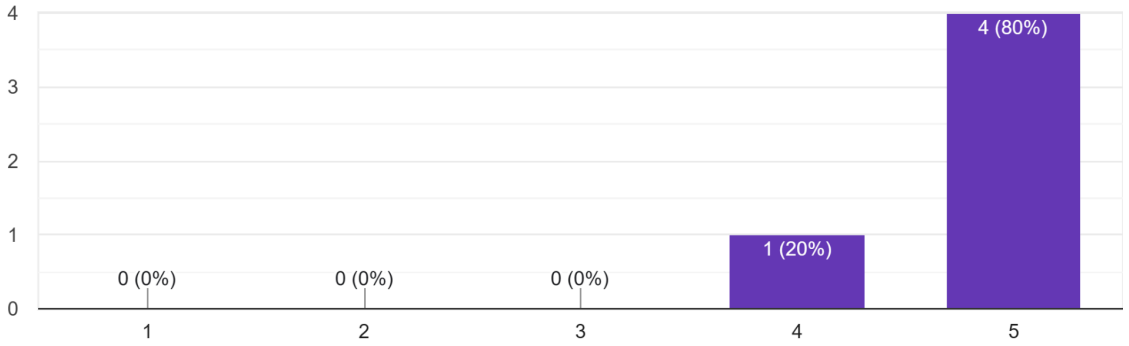
I was able to find the specific disaster manual for Earthquake/Flood in under 10 seconds.

5 responses



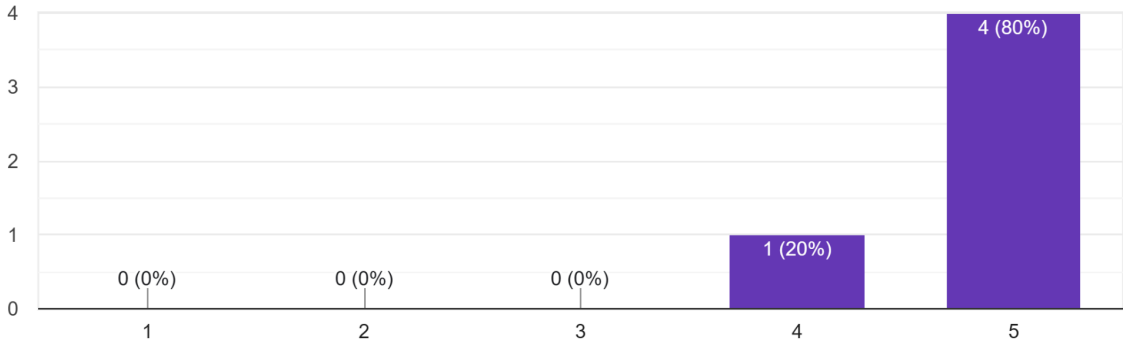
The search results were relevant to the keywords I typed.

5 responses



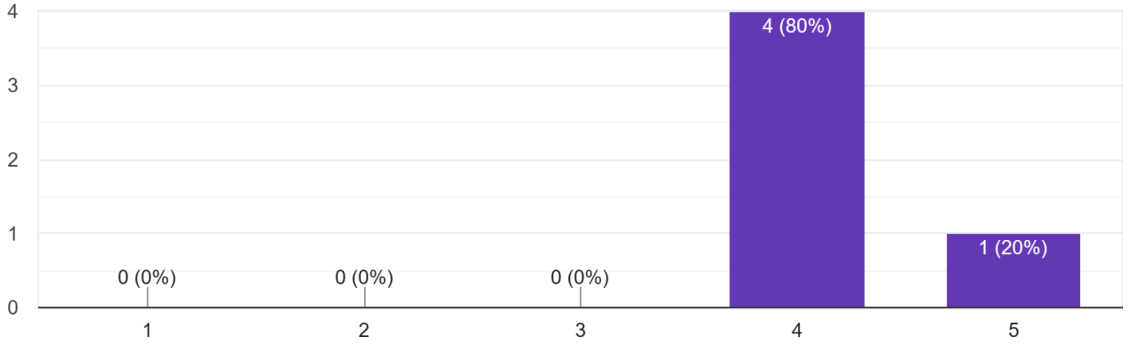
I understood the instructions provided in the safety manuals without outside help.

5 responses



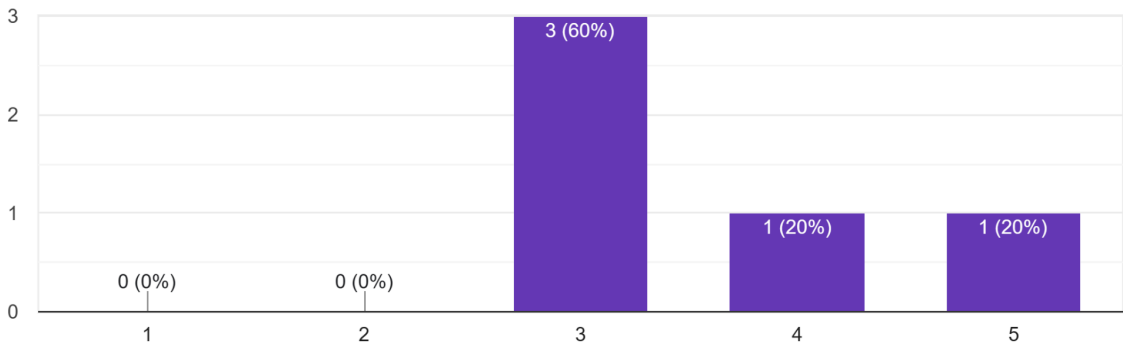
If the internet were to go out, I would feel confident navigating to the search screen.

5 responses



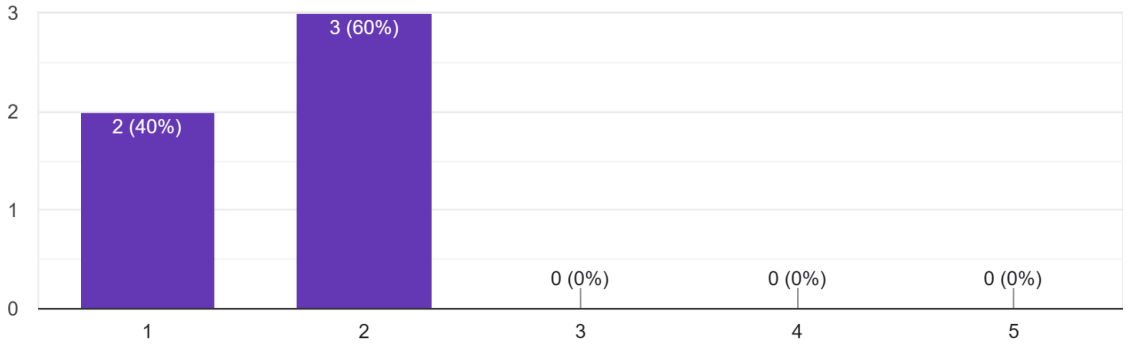
I think that I would like to use this system frequently.

5 responses



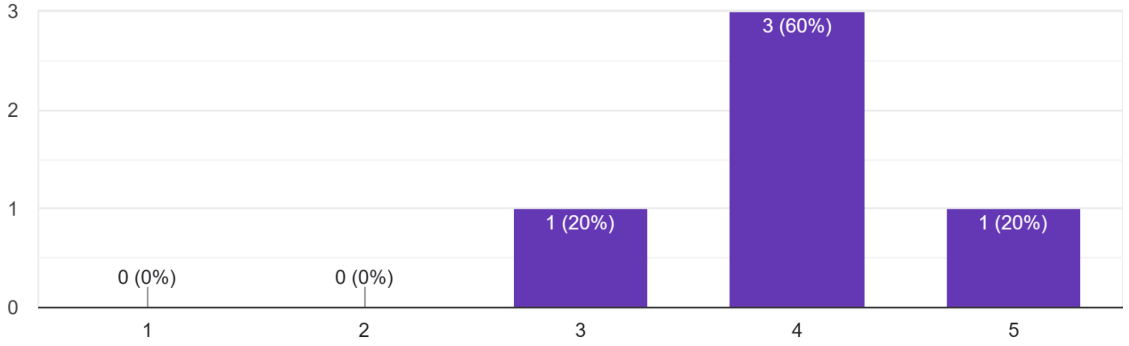
I found the system unnecessarily complex.

5 responses



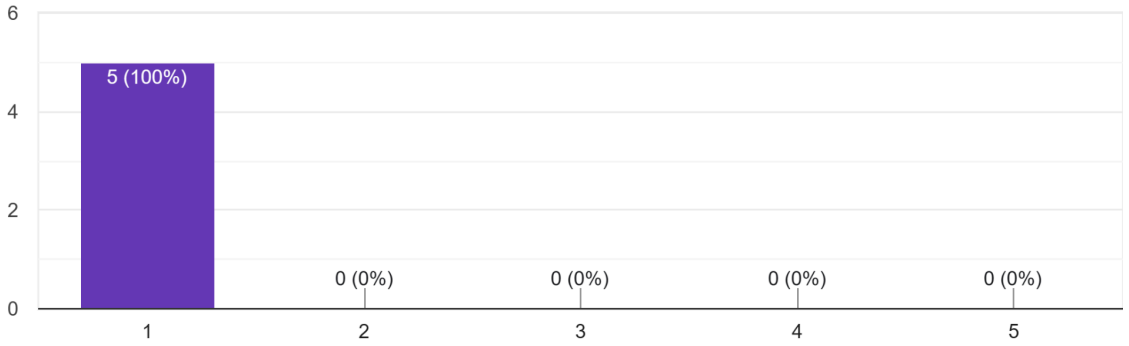
I thought the system was easy to use.

5 responses



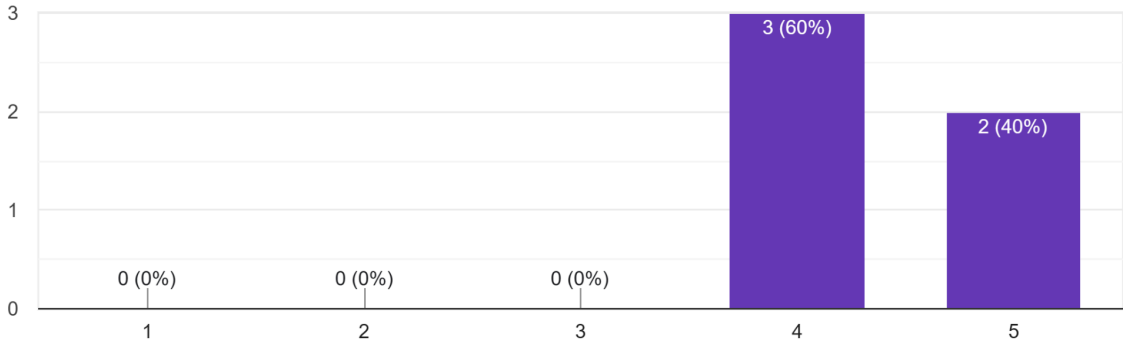
I think that I would need the support of a technical person to use this system.

5 responses



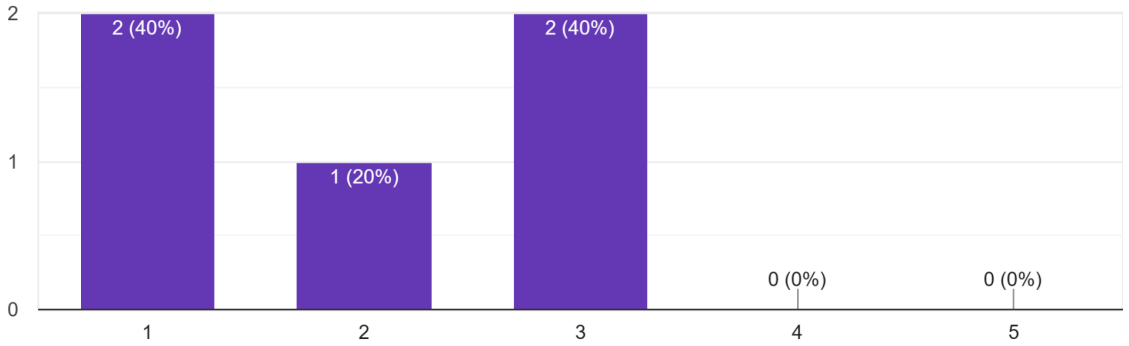
I found the various functions in this system were well-integrated.

5 responses



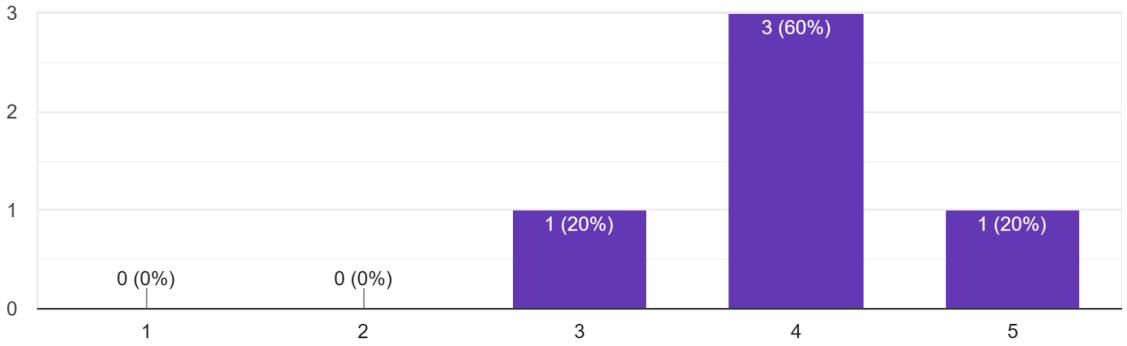
I thought there was too much inconsistency in this system.

5 responses



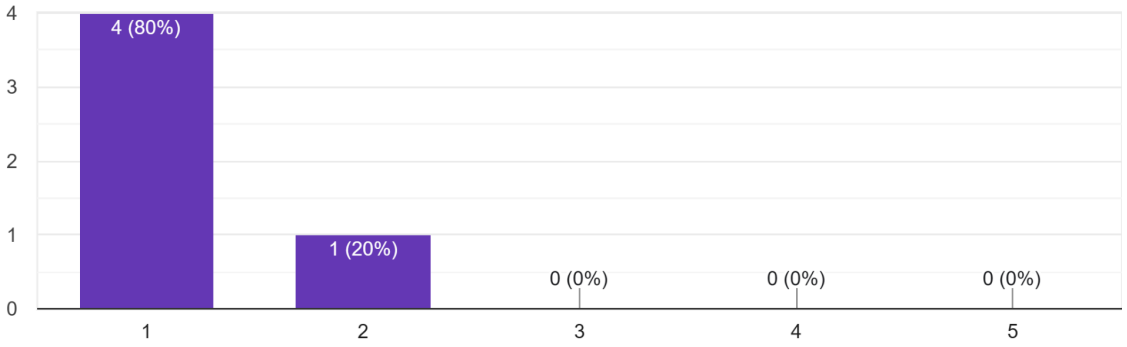
I would imagine that most people would learn to use this system quickly.

5 responses



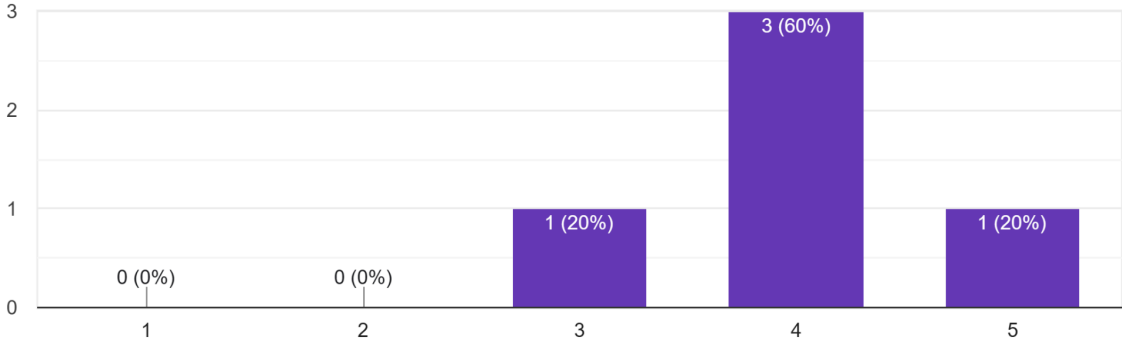
I found the system very cumbersome to use.

5 responses



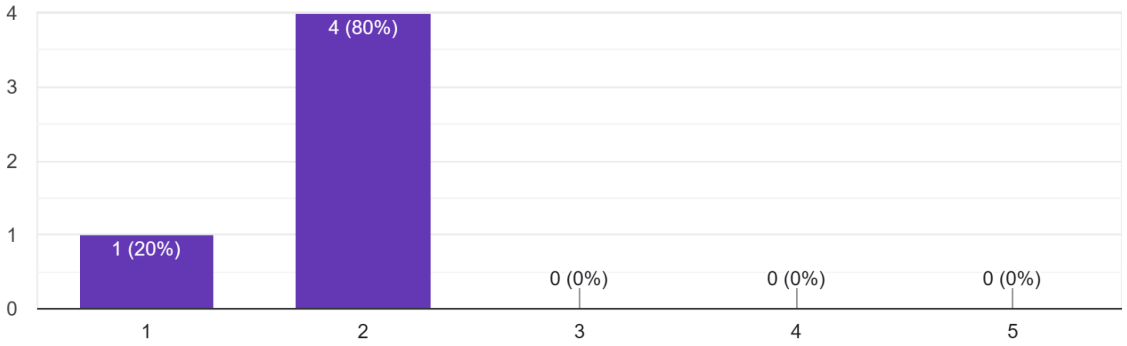
I felt very confident using the system.

5 responses



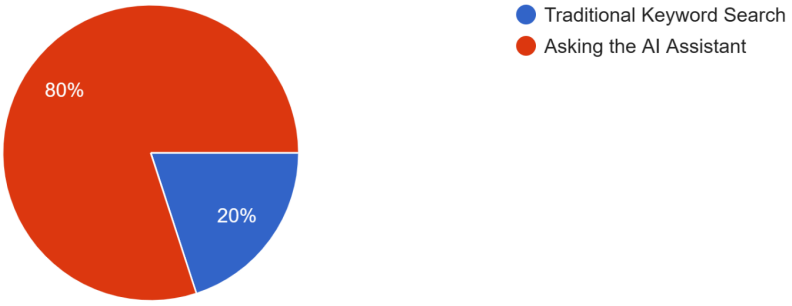
I needed to learn a lot of things before I could get going with this system.

5 responses



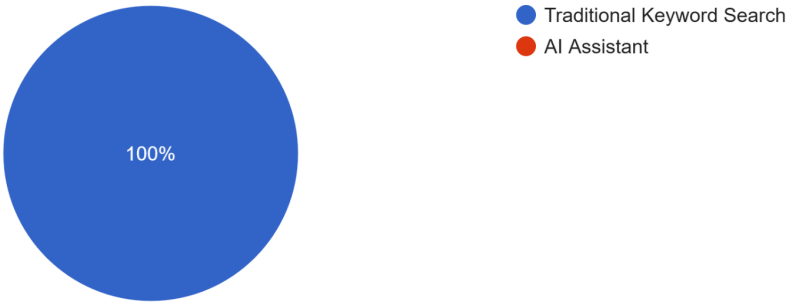
Which method of finding information felt more natural to you?

5 responses



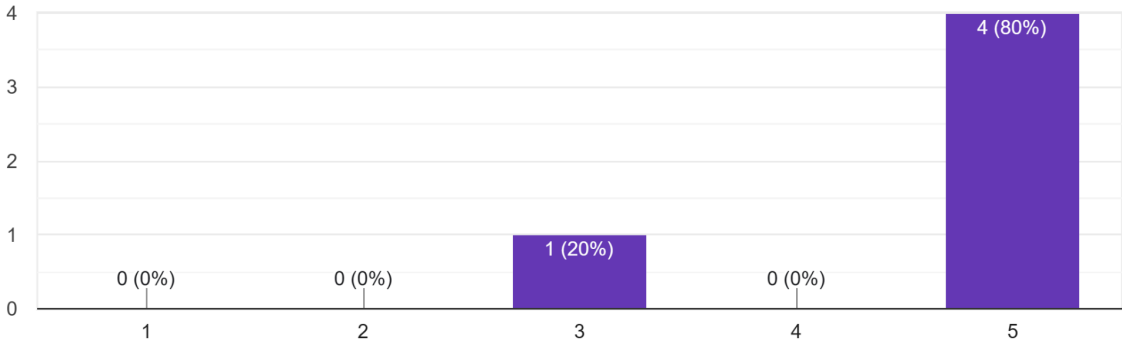
Which method did you trust more for life-saving information?

5 responses



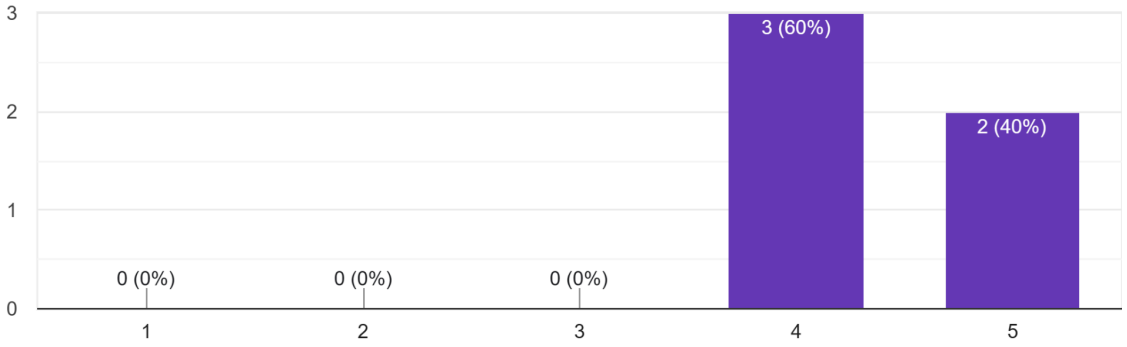
The 'Verified' badge on the AI responses increased my confidence in the advice given.

5 responses



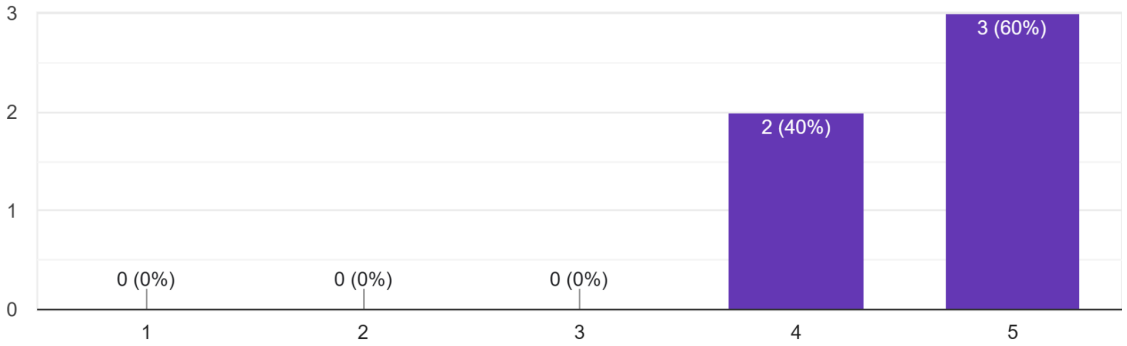
Earning XP made completing the checklists more enjoyable.

5 responses



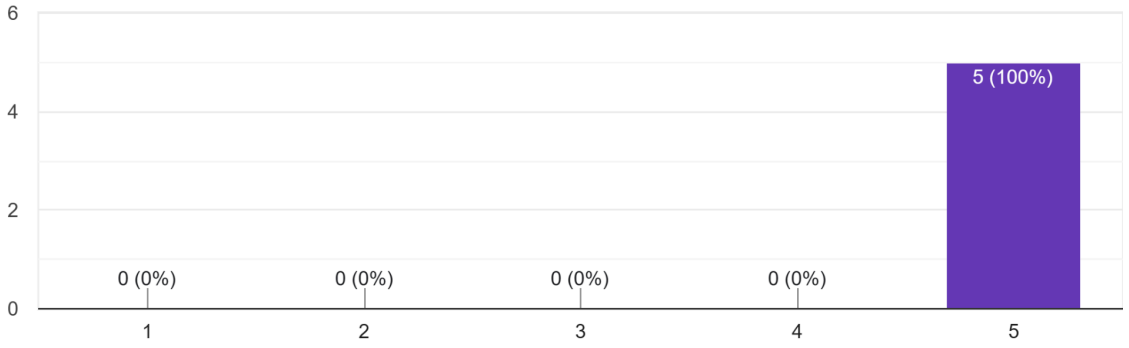
I felt a sense of progress when my 'Prepper Level' increased.

5 responses



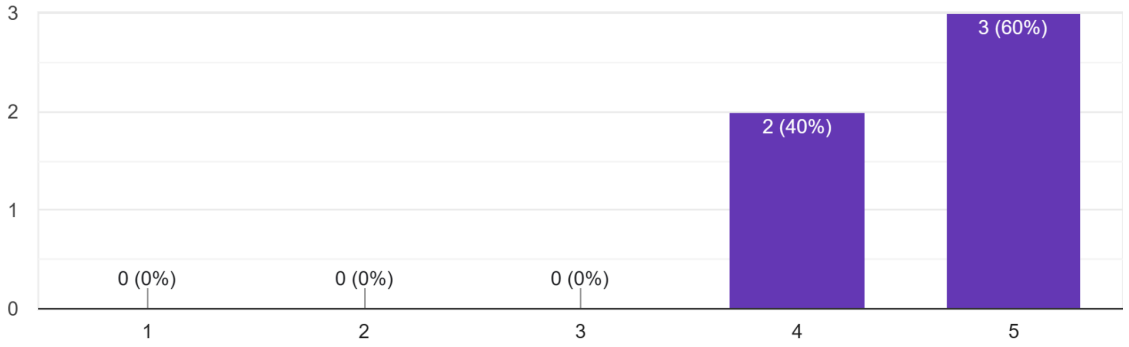
The text was easy to read, even in low-light or stressful conditions.

5 responses



The icons (Earth, Water, Flame) were intuitive and easy to recognize.

5 responses



How would you rank the overall 'Speed' of the app (1=Slow, 5=Instant)?

5 responses

